VERIQTA

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

DEVOPS TROUBLESHOOTING HANDBOOK

From Beginner to Production Troubleshooting

Enterprise Incident Response, Root Cause Analysis, and Production Recovery

DETECT Find the Problem → **DIAGNOSE** Understand the Cause → **MITIGATE** Stop the Impact → **RECOVER** Restore Service → **IMPROVE** Prevent it Happening Again

ENGINEER MINDSET

- ✓ Stay Calm
- ✓ Collect Facts
- ✓ Think Systemically
- ✓ Verify Everything
- ✓ Document Always
- ✓ Learn & Improve

— COVERS —

LINUX	DOCKER	KUBERNETES	AWS	AZURE	GCP
GIT	JENKINS	GITLAB CI	PROMETHEUS	GRAFANA	TERRAFORM
SHELL	NETWORKING	SECURITY	DATABASES	OBSERVABILITY	LOGS & TRACES

INSIDE THIS HANDBOOK

- ★ 60 Pages of Practical Troubleshooting
- ★ Real-World Scenarios and Case Studies
- ★ Step-by-Step Investigation Techniques
- ★ Command Reference & Quick Checklists
- ★ Production-Ready Playbooks
- ★ Build Your Incident Response Muscle

PRODUCTION SYSTEMS WILL FAIL.
GREAT ENGINEERS DON'T PREVENT ALL FAILURES,
THEY RECOVER FAST AND MAKE SYSTEMS STRONGER.

HANDBOOK CURRICULUM

60 PAGES

PRACTICAL. HANDS-ON. PROVEN.
TROUBLESHOOT LIKE A PRO.

Follow VERIQTA → YouTube /veriqta · GitHub /veriqta · LinkedIn /veriqta · X @veriqta · Instagram @veriqta · Telegram @veriqta

INTRODUCTION TO DEVOPS TROUBLESHOOTING

WHAT TROUBLESHOOTING REALLY MEANS

Troubleshooting is the systematic process of identifying why something is not working as expected, determining the root cause, and restoring the system with minimal impact.

IN PRODUCTION,
the goal is not just to fix the problem, but to restore trust in the system and prevent it from happening again.

TROUBLESHOOTING MINDSET

- **Be Curious:** Always ask "Why?" and "What changed?"
- **Gather Evidence First:** Observe, don't guess.
- **Think Systemically:** Understand how components interact.
- **Test Hypotheses:** Prove or disprove theory with data.
- **Document Everything:** For learning, knowledge sharing, and future you.

REACTIVE VS PROACTIVE OPERATIONS

REACTIVE (FIRE-FIGHTING)	PROACTIVE (PREVENTION)
✗ Fixes problems after users are impacted	✓ Detects issues early
✗ High stress, time pressure	✓ Prevents incidents
✗ Repeat issues	✓ Improves reliability
✗ Lost trust and reliability	✓ Builds trust
✗ Expensive in the long run	✓ Saves time and money

INCIDENT LIFECYCLE

1. **DETECT** — Identify that something is wrong.
2. **VERIFY** — Confirm the issue and avoid false alarms.
3. **SCOPE** — Understand the blast radius and who is affected.
4. **ISOLATE** — Pinpoint the component causing the issue.
5. **DIAGNOSE** — Find the root cause using data and logs.
6. **MITIGATE** — Apply a fix or workaround to reduce impact.
7. **RECOVER** — Restore service to normal and validate.
8. **LEARN** — Perform RCA and improve processes.

HANDBOOK OBJECTIVES

- ✓ Build a strong troubleshooting foundation across Linux, Networking, Containers, Kubernetes, Cloud, CI/CD, and more.
- ✓ Learn systematic incident response and root cause analysis techniques.
- ✓ Understand real-world production failures and how to recover from them.
- ✓ Develop repeatable workflows, checklists, and playbooks.
- ✓ Improve system reliability, performance, security, and resilience.
- ✓ Become a calm, confident, and trusted engineer during incidents.

REMEMBER

Every incident is an opportunity to learn. The best engineers are not those who never fail, but those who learn fast and prevent repeat failures.

TROUBLESHOOTING IS NOT JUST A SKILL. IT IS AN ENGINEERING SUPERPOWER.

BUILDING A TROUBLESHOOTING MINDSET

Great engineers don't guess. They observe, think, verify, and then act.

1. THINKING LIKE AN ENGINEER

- **Be curious:** always ask "Why is this happening?"
- **Look** at the system as a whole, not just the error.
- Understand how components depend on each other.
- Think in **flows:** request → service → dependency → data.
- **Prioritize impact:** what is broken, for whom, and how much?

2. GATHERING EVIDENCE BEFORE ACTING

- ☑ Never touch anything before you understand it.
- ☑ Collect facts, not opinions.
- ☑ Use commands, logs, metrics, traces, and events.
- ☑ Capture the time, symptoms, and environment.
- ☑ Preserve evidence before restarting or changing anything.

EVIDENCE SOURCES

3. HYPOTHESIS-DRIVEN INVESTIGATION

OBSERVE What is happening? → **HYPOTHESIZE** What could be causing it? → **TEST** Prove or disprove it. → **CONCLUDE** Identify the root cause.

- Make a hypothesis early, but don't attach to it.
- Test it quickly and safely.
- If it's wrong, adjust and form a new one.
- Repeat until you find the real cause.

☆ A good engineer changes their mind often. A great engineer changes it quickly.

4. AVOIDING ASSUMPTIONS

- **Don't** assume it's the code.
- **Don't** assume it's the server.
- **Don't** assume it's the network.
- **Don't** assume it's the database.
- **Don't** assume it's a one-off issue.
- **Don't** assume the last change is the cause.

REMEMBER
Assumptions create blind spots.
Evidence creates clarity.

5. DOCUMENTATION HABITS

- Document as you go, not after you fix.
- Capture symptoms, timelines, commands, and findings.
- Document what you tried and what happened.
- Write root cause, impact, and resolution.
- Update runbooks and internal knowledge base.
- Great documentation prevents repeat incidents.

WHAT TO DOCUMENT

✓ Date & time	✓ Commands used
✓ Environment	✓ Findings
✓ Symptoms	✓ Root cause
✓ Impact	✓ Resolution
✓ Steps taken	✓ Lessons learned

THE GOAL

Solve the problem once, learn from it forever, and prevent it from happening again.

PRODUCTION NOTE
In production, speed comes from clarity, not guessing. The best engineers are not the ones who know the most tools, but the ones who think the clearest under pressure.

ENTERPRISE TROUBLESHOOTING WORKFLOW

A structured path from alert to learning. Follow the process, not just your gut.

STEP	PURPOSE	KEY ACTIONS / EXAMPLES
1 DETECT	Be aware something is wrong.	• Alerts, monitoring, user reports • Dashboards, logs, SLOs • Example: High CPU alert from Prometheus
2 VERIFY	Confirm the issue is real and ongoing.	• Check dashboards, logs, metrics • Reproduce if possible • Example: top, kubectl get pods, curl endpoint
3 SCOPE	Understand the blast radius and who is affected.	• How many users? Which regions? • Which services / components? • Dependencies impacted? • Example: Check traffic, error rate, affected AZ
4 ISOLATE	Narrow down the problem to the failing component.	• Eliminate healthy parts • Use binary search mindest • Example: Disable new release, check dependencies
5 DIAGNOSE	Find the true cause of the problem.	• Deep dive into logs, metrics, traces • Correlate events and timelines • Example: kubectl logs, journalctl, trace analysis
6 MITIGATE	Reduce impact while working on a fix.	• Workaround, feature flag, scale up • Throttle traffic, failover, rate limit • Example: Scale deployment, enable circuit breaker
7 RECOVER	Restore normal service as quickly and safely as possible.	• Apply the fix • Verify system is stable • Example: Rollout new version, restart service
8 ROOT CAUSE ANALYSIS	Find the underlying cause to prevent recurrence.	• 5 Whys, timeline, logs, metrics • Identify contributing factors • Example: Misconfigured timeout caused retries
9 POST-INCIDENT REVIEW	Learn from the incident and improve the system and team.	• Blameless review • What went well / What didn't • Action items with owners & deadlines • Update runbooks, alerts, docs, tests

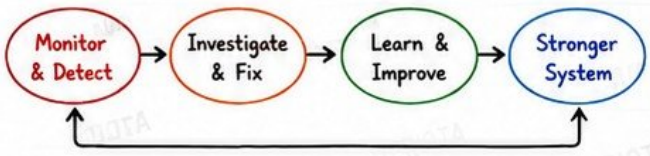
ENGINEER PRINCIPLES

- ☑ Stay calm and think clearly.
- ☑ Follow the process.
- ☑ Gather facts, not opinions.
- ☑ Communicate often and clearly.
- ☑ Document everything.
- ☑ Focus on system recovery first, then RCA.
- ☑ Automate what you learn.

REMEMBER

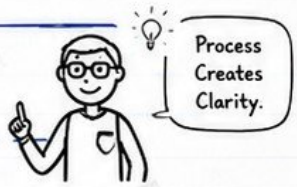
The goal is not just to fix the problem, but to make the system stronger so the same issue doesn't happen again.

TROUBLESHOOTING IS A LOOP



EXAMPLE: HIGH ERROR RATE

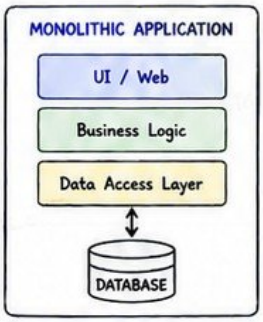
PRO TIP
✓ Good engineers follow the workflow.
✓ Great engineers know when to go deeper and when to step back.



UNDERSTANDING PRODUCTION ARCHITECTURES

Know the pattern before you troubleshoot the problem.

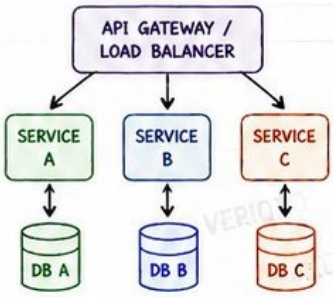
1. MONOLITHS



- Single deployable unit
- Shared codebase & resources
- Tight coupling
- Simple to build, hard to scale

Use when: Small teams, simple domains, early stage.

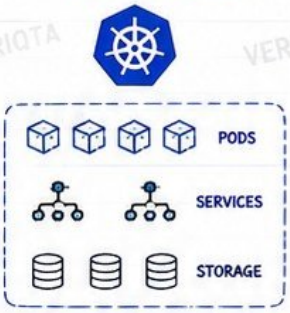
2. MICROSERVICES



- Many small, independent services
- Each service owns its data
- Loose coupling via APIs
- Scales independently

Use when: Complex domains, multiple teams, scale at speed.

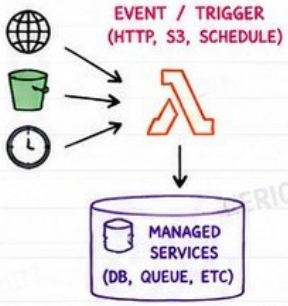
3. KUBERNETES PLATFORMS



- Container orchestration
- Self-healing & scalable
- Declarative configuration
- Great for microservices

Use when: Containerized apps, high scalability, automation.

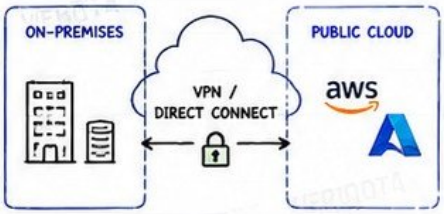
4. SERVERLESS



- No servers to manage
- Auto-scale by design
- Pay per request / duration
- Fast to build & deploy

Use when: Event-driven, spiky workloads, low ops.

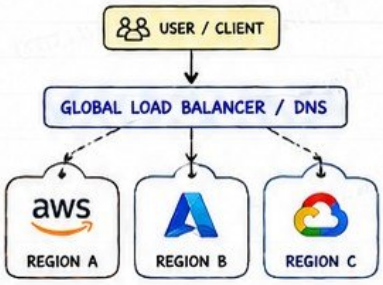
5. HYBRID CLOUD



- Combines on-prem infrastructure with cloud
- Workloads placed where they make sense
- Requires secure connectivity & management

Use when: Compliance, data locality, legacy systems, gradual migration.

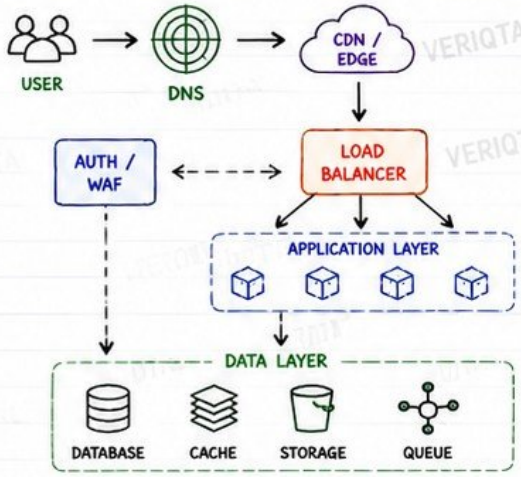
6. MULTI-CLOUD



- Use multiple cloud providers
- Avoid vendor lock-in
- Improve availability & resilience
- More complex networking & governance

Use when: Business critical apps, disaster recovery, global presence.

7. REQUEST FLOW OVERVIEW



ARCHITECTURE COMPARISON

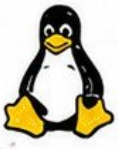
ARCHITECTURE	COMPLEXITY	SCALABILITY	OPERATIONAL OVERHEAD	DEPLOYMENT SPEED	BEST FOR
Monolith	Low	Low	Low	High	Small apps, early teams
Microservices	High	High	High	Medium	Large apps, many teams
Kubernetes	High	High	High	Medium	Containers, microservices
Serverless	Low	Very High	Low	High	Events, APIs, spiky loads
Hybrid Cloud	Medium	High	Medium	Medium	Compliance, legacy, gradual move
Multi-Cloud	High	High	High	Low	Resilience, DR, global scale

PRODUCTION NOTE

There is no "best" architecture. The best architecture is the one your team can operate, secure, and evolve reliably in production.

TROUBLESHOOTING STARTS WITH CONTEXT.
Understand the architecture, the flow, and the dependencies before you touch anything.

- ☑ Know the pattern
- ☑ Map the flow
- ☑ Identify the failure domain
- ☑ Follow the evidence



LINUX SYSTEM HEALTH

A healthy system is a predictable system. Monitor, measure, and understand.

1. CPU

- High CPU = slow system or overloaded processes.
- Check usage, top processes, and load.

CORE COMMANDS

```
$ top                  # Real-time CPU usage
$ htop                 # Interactive process viewer
$ mpstat -P ALL 1 5    # Per-core CPU stats
$ pidstat -u 1 5       # Per-process CPU usage
```

2. MEMORY

- High memory usage can cause swapping and slow performance.
- Watch free memory, buffers, cache, and swap.

CORE COMMANDS

```
$ free -h              # Memory overview
$ vmstat 1 5           # Memory + system stats
$ ps aux --sort=-%mem | head   # Top memory consuming
$ cat /proc/meminfo    # Detailed memory info
```

3. DISK

- Disk full or slow I/O can crash applications.
- Monitor usage, inode, and I/O performance.

CORE COMMANDS

```
$ df -h                # Disk usage
$ du -sh /*            # Folder sizes
$ iostat -x 1 5        # Disk I/O stats
$ df -i                # Inode usage
$ lsblk                # Block devices
$ dmesg | tail         # Disk errors
```

4. LOAD AVERAGE

- Load average shows how busy the system is.
- Rule of thumb:
  Load < CPU cores = Healthy

CORE COMMANDS

```
$ uptime               # Load average (1, 5, 15 min)
$ cat /proc/loadavg    # Raw load average
$ w                    # Users + load
$ top                  # Check running tasks
```

5. UPTIME

- Uptime shows how long the system has been running.
- Frequent reboots may indicate instability.

CORE COMMANDS

```
$ uptime               # Uptime summary
$ who -b               # Last boot time
$ last reboot          # Reboot history
$ systemctl list-unit-files --state=failed
                       # Failed services since boot
```

6. CORE COMMANDS

- Essential quick checks every engineer should know.
- Use these first during any investigation.

CORE COMMANDS

```
$ hostnamectl          # Host information
$ ip a                 # Network interfaces
$ ss -tulnp            # Listening ports
$ dmesg | tail -n 50   # Recent kernel logs
$ journalctl -xe       # System logs
$ sudo dnf updateinfo list
                       # Check updates (RHEL/CentOS)
$ sudo apt update && apt list --upradable
                       # Check updates (Debian/Ubuntu)
```

HEALTH INDICATORS

METRIC	HEALTHY RANGE	WARNING	CRITICAL
CPU Usage	< 70%	70% - 85%	> 85%
Memory Usage	< 70%	70% - 85%	> 85%
Disk Usage	< 75%	75% - 90%	> 90%
Inode Usage	< 75%	75% - 90%	> 90%
Load Average	< CPU Cores	= CPU Cores	> CPU Cores
Uptime	> 7 Days	3 - 7 Days	< 3 Days

QUICK HEALTH CHECK (ONE-LINER)

```
$ echo -e "\n=== SYSTEM HEALTH SUMMARY ==="
$ uptime
$ free -h | awk '/Mem:/ || /Swap:/'
$ df -h | awk 'NR==1 || /\/ {print}'
$ top -bn1 | grep "Cpu(s)"
$ cat /proc/loadavg
$ echo -e "============================\n"
```

REMEMBER

- ☑ Jumitor continuously
- ☑ Set alerts early
- ☑ Investigate trends, not just spikes
- ☑ Document baseline
- ☑ Automate checks

PRO TIP

Always start with the big picture (CPU, Memory, Disk, Load, Uptime), then drill down. Measure → Understand → Act → Verify.

COMMON MISTAKES

- ✗ Jumping to conclusions
- ✗ Not collecting enough data
- ✗ Ignoring trends and patterns
- ✗ Fixing symptoms, not causes
- ✗ Not documenting findings
- ✗ No post-incident review

LINUX PROCESS TROUBLESHOOTING

Processes are the heart of a Linux system. Know how to find, fix and prevent issues.

1. HUNG PROCESSES

- Process is running but not responding.
- Usually stuck in uninterruptible sleep (D state).

HOW TO FIND

```
$ ps -eo pid,ppid,stat,comm | grep " D "
                                  # Look for D state
$ top -H -p <pid>        # Check individual threads
$ echo w > /proc/sysrq-trigger
                                  # Try to get stack trace
```

HOW TO FIX

- ☑ Identify what resource it is waiting for (disk, NFS, etc.)
- ☑ Check dmesg for I/O or storage errors
- ☑ Fix the underlying issue, then kill if needed
- ☑ `kill -9 <pid>` (last resort)

2. ZOMBIE PROCESSES

- Process has finished but parent has not collected its exit status.
- Does not use system resources but can accumulate.

HOW TO FIND

```
$ ps -eo pid,ppid,stat,comm | grep Z
                    # Look for Z state
$ pstree -p | grep "<defunct>"
```

HOW TO FIX

- ☑ Fix the parent process
- ☑ Restart the parent service
- ☑ If needed, kill the parent process
- ☑ Prevent in code: `wait()` or SIGCHLD handling

3. HIGH CPU USAGE

- CPU overloaded by one or more processes.
- Leads to slow system and unresponsive apps.

HOW TO FIND

```
$ top                    # Live view
$ htop                   # Better interactive view
$ ps -eo pid,ppid,pcpu,pmem,comm --sort=-pcpu | head
                                  # Top CPU consumers
```

HOW TO FIX

- ☑ Identify the process causing high CPU
- ☑ Check if it is expected (backup, index, loop, etc.)
- ☑ Optimize the application or query
- ☑ Reduce load or add more resources
- ☑ `kill / nice / ionice` to control impact

4. HIGH MEMORY USAGE

- Excessive memory usage causes swapping, slow system or OOM kill.
- Often caused by memory leaks.

HOW TO FIND

```
$ free -h                # Memory overview
$ top                    # Check %MEM
$ ps -eo pid,ppid,pmem,comm --sort=-pmem | head
                          # Top memory consumers
$ cat /proc/<pid>/status | grep VmRSS
                          # Resident memory
```

HOW TO FIX

- ☑ Identify the memory hungry process
- ☑ Check for memory leaks in application
- ☑ Restart the process/service
- ☑ Tune JVM, buffers, caches if applicable
- ☑ Add swap or more RAM (temporary relleif)

5. SERVICE FAILURES

- Services may fail to start, crash, or keep restarting.
- Check logs, configs, dependencies and resources.

HOW TO FIND

```
$ systemctl status <service>
                          # Detailed status
$ journalctl -u <service> --no-pager -n 50
                          # Recent logs
$ systemctl list-units --state=failed
                          # List failed services
```

HOW TO FIX

- ☑ Check logs for root cause
- ☑ Verify configuration files
- ☑ Check dependencies (DB, network, ports)
- ☑ Check permissions and system resources
- ☑ Restart and enable the service

QUICK CHECKLIST

1. Identify the problem
2. Find the affected process/service
3. Collect evidence (logs, metrics)
4. Determine the root cause
5. Apply the fix
6. Verify the fix
7. Prevent recurrence
8. Document what happened

ESSENTIAL PROCESS COMMANDS

ps aux	top / htop	pstree -p	kill <pid>	kill -9 <pid>	nice -n <n> <cmd>	renice -n <n> -p <pid>	lsof -p <pid>
List all processes	Live process monitoring	Process tree view	Graceful termination	Force kill (ungraceful)	Start process with priority	Change running process priority	Open files by process

TROUBLESHOOTING WORKFLOW

Detect — Find the issue → Investigate — Collect data & evidence → Identify — Find the root cause → Fix — Apply the solution → Verify — Confirm the fix works

Repeat until the system is stable and healthy.

PRO TIPS

- ✓ Always check logs first.
- ✓ Correlate metrics with process behavior.
- ✓ Use small, safe steps when killing processes.
- ✓ Automate monitoring and alerting.
- ✓ Document fixes for future reference.

COMMON MISTAKES

- ✗ Killing processes without finding the cause
- ✗ Ignoring system logs
- ✗ Not checking resource usage
- ✗ Restarting services without root cause
- ✗ No follow-up or documentation

THE GOAL IS NOT JUST TO FIX THE PROBLEM, BUT TO UNDERSTAND WHY IT HAPPENED AND PREVENT IT.

LINUX FILE SYSTEM ISSUES

Filesystems store everything. When they fail, everything fails.

1. DISK FULL

- No space left for new files.
- Applications fail to write.
- Logs grow endlessly.
- Databases stop working.

HOW TO CHECK

```
$ df -h                    # Disk usage
$ df -i                    # Inode usage
$ du -sh /*                # Big directories
$ du -ah /var | sort -rh | head -n 10
$ ncdu /var/log            # Interactive view
```

HOW TO FIX

- ☑ Delete unnecessary files and logs
- ☑ Rotate and compress logs
- ☑ Clear package cache: `apt clean` / `yum clean all`
- ☑ Move large files to another partition
- ☑ Add more disk or extend the filesystem

2. INODE EXHAUSTION

- Disk looks free, but no inodes.
- Too many small files/directories.
- New files cannot be created.

HOW TO CHECK

```
$ df -i                              # Check inode usage
$ find / -xdev -type f | wc -l       # Count files
$ find / -xdev -type d | wc -l       # Count dirs
$ sudo tune2fs -l /dev/sdX | grep "Inode count"
                                     # Filesystem inode details
```

HOW TO FIX

- ☑ Delete unnecessary small files
- ☑ Clean up temp and cache directories
- ☑ Remove old build artifacts, backups
- ☑ Increase inode count (requires reformat)
- ☑ Use filesystems with higher inode ratio (XFS)

3. MOUNT FAILURES

- Filesystem not mounted.
- Invalid fstab entry.
- Device, UUID or label missing.
- Network/remote mount issues.

HOW TO CHECK

```
$ mount                    # Current mounts
$ cat /etc/fstab           # Fstab entries
$ dmesg | tail             # Kernel messages
$ lsblk -f                 # Devices & filesystems
$ blkid                    # UUID/LABEL info
$ systemctl status remote-fs.target
                           # Check remote mounts
```

HOW TO FIX

- ☑ Verify device/UUID in /etc/fstab
- ☑ Check filesystem type and options
- ☑ Fix permissions for mount point
- ☑ Ensure network is up (for NFS/SMB)
- ☑ Remount: `mount -a` or `mount /mount/point`

4. CORRUPTION INDICATORS

- Unexpected crashes or unclean shutdowns.
- Files disappear or get corrupted.
- Applications report I/O errors.
- Filesystem becomes read-only.

SIGNS TO LOOK FOR

- I/O error, Input/output error
- EXT4-fs error, XFS (dm-0): Metadata corruption
- Journal checksum error
- Bad magic number in super-block
- Filesystem mounted read-only

CHECK & REPAIR

```
$ dmesg | grep -i error
$ fsck -n /dev/sdX         # Check only
$ fsck -y /dev/sdX         # Auto repair (offline)
$ e2fsck -f /dev/sdX       # For ext*
$ xfs_repair /dev/sdX      # For XFS
$ btrfs check /dev/sdX     # For Btrfs
```

⚠ Always unmount filesystem before fsck!

5. PERMISSIONS ISSUES

- Permission denied errors.
- Files owned by wrong user/group.
- Services cannot read/write files.

HOW TO CHECK

```
$ ls -l /path/to/file          # Check permissions
$ namei -l /path/to/file       # Path permissions walk
$ stat /path/to/file           # Detailed info
$ getfacl /path/to/file        # ACL permissions
$ sudo -u <user> command       # Test as user
```

HOW TO FIX

- ☑ chown user:group file
- ☑ chmod 644 file
- ☑ chmod -R 755 /path
- ☑ setfacl -m u:user:rwx file
- ☑ Restore correct ownership
- ☑ Fix SELinux context if needed

ESSENTIAL FILESYSTEM COMMANDS

df -h	df -i	du -sh /dir	ncdu /dir	lsblk -f	find / -type f	lsof	mount	fsck
Disk space usage	Inode usage	Directory size	Interactive disk usage	Block devices & filesystems	Find files	List open files	Show mounts	Check & repair filesystem

BEST PRACTICES

- ☑ Monitor disk and inode usage regularly
- ☑ Rotate and compress logs
- ☑ Keep /var and /tmp clean
- ☑ Use separate partitions for critical paths
- ☑ Backup important data
- ☑ Use LVM for flexible storage management

MONITOR FILESYSTEM HEALTH

```
$ sudo apt install sysstat     # or yum install sysstat
$ iostat -x 1                  # Disk IO stats
$ sar -d 1 5                   # Disk performance
$ smartctl -a /dev/sdX         # Disk health (SMART)
$ prometheus node exporter     # Export metrics
```

Set alerts for: Disk usage > 80%, Inode usage > 80%, I/O wait high, Read/Write errors.

QUICK CHECKLIST

- ☑ Is the disk full? (df -h)
- ☑ Are inodes exhausted? (df -i)
- ☑ Can we write to the filesystem?
- ☑ Are mounts working? (mount)
- ☑ Any errors in dmesg?
- ☑ Check permissions and ownership
- ☑ Review logs and recent changes

A HEALTHY FILESYSTEM = STABLE APPLICATIONS = HAPPY USERS
Monitor early, troubleshoot smart, and prevent outages.

LINUX NETWORK TROUBLESHOOTING

A network issue anywhere in the path can break everything.

1. DNS ISSUES

DNS

- Cannot resolve domains.
- Slow DNS responses.
- Wrong DNS records.

HOW TO CHECK

```
$ cat /etc/resolv.conf
$ nslookup example.com
$ dig example.com +short
$ dig @8.8.8.8 example.com
```

HOW TO FIX

- ☑ Check /etc/resolv.conf
- ☑ Verify DNS server reachability
- ☑ Flush nscd or systemd-resolved
- ☑ Check /etc/hosts for overrides

2. ROUTING ISSUES

- Wrong route or gateway.
- No route to host.
- Asymmetric routing.

HOW TO CHECK

```
$ ip route
$ route -n
$ ip rule show
$ traceroute 8.8.8.8
```

HOW TO FIX

- ☑ Verify default gateway
- ☑ Check route metrics
- ☑ Validate policy routing
- ☑ Check return path (asymmetry)

3. FIREWALL ISSUES

- Blocks traffic.
- Wrong rules.
- Default drop policies.

HOW TO CHECK

```
$ sudo firewall-cmd --list-all
$ sudo iptables -L -n -v
$ sudo nft list ruleset
$ sudo ufw status verbose
```

HOW TO FIX

- ☑ Allow required ports/services
- ☑ Check zone / chain policies
- ☑ Add explicit allow rules
- ☑ Reload and test again

4. PORT ISSUES

- Service not listening.
- Wrong port.
- Port blocked.

HOW TO CHECK

```
$ ss -tulnp
$ netstat -tulnp
$ lsof -i :80
$ nc -zv host 80
```

HOW TO FIX

- ☑ Verify service is running
- ☑ Check correct port number
- ☑ Open port in firewall
- ☑ Test with nc or telnet

5. TCP CONNECTIONS

- Connection timeouts.
- Too many connections.
- Half-open connections.

HOW TO CHECK

```
$ ss -s
$ ss -antp | head
$ netstat -s | grep Tcp
$ cat /proc/sys/net/ipv4/tcp
  _max_syn_backlog
```

HOW TO FIX

- ☑ Check target service
- ☑ Tune TCP settings
- ☑ Increase backlog if needed
- ☑ Kill stale or unwanted sessions

6. PACKET FLOW: END-TO-END PATH

CLIENT → **DNS RESOLUTION** → **ROUTING** → **FIREWALL** → **PORT / SERVICE** → **RESPONSE**

CLIENT	DNS RESOLUTION	ROUTING	FIREWALL	PORT / SERVICE	RESPONSE
✓ App generates request	✓ DNS server reachable	✓ Correct route	✓ Rules allow traffic	✓ Service listening	✓ Reply received
✓ Correct URL / IP	✓ Correct record	✓ Gateway reachable	✓ NAT if required	✓ Correct port	✓ No packet loss
✓ Local firewall	✓ Fast response	✓ No routing loops	✓ No drops / rejects	✓ App healthy	✓ Latency OK

ESSENTIAL NETWORK TROUBLESHOOTING COMMANDS

PING	TRACEROUTE	MTR	SS	NMAP	TCPDUMP	DIG	IP	ARP	ETHTOOL
ping -c 4 host	traceroute host	mtr host	ss -tulnp	nmap -sS host	tcpdump -i any	dig domain	ip addr / ip r	arp -a	ethtool eth0
Test basic reachability	Trace the path to destination	Trace with loss & latency	Show sockets & services	Port scan & discovery	Capture packets live	DNS lookup & debugging	IP address & routes	View ARP table	Interface health & stats

NETWORK DIAGNOSTIC WORKFLOW

1. Identify the symptom — What is failing?
2. Check reachability — Ping / DNS / Gateway
3. Validate the path — Routing / Firewall / Ports
4. Verify the service — Is the service healthy?
5. Confirm end-to-end — From client to server

QUICK CHECKLIST

- ☑ Can I reach the host?
- ☑ Can I resolve the name?
- ☑ Is the route correct?
- ☑ Is the firewall blocking?
- ☑ Is the port open?
- ☑ Is the service running?
- ☑ Is the app responding?
- ☑ Check logs and metrics

COMMON NETWORK ISSUES

- ✗ DNS misconfiguration
- ✗ Wrong routes or missing gateway
- ✗ Firewall blocking required ports
- ✗ Service not listening on correct IP/port
- ✗ NAT misconfiguration
- ✗ MTU / Packet size issues
- ✗ High latency or packet loss
- ✗ Too many connections (exhaustion)

PRO TIP

Always validate the complete path:

Client → DNS → Route → Firewall → Port → Service → Response

One small break causes a big outage.

NETWORKS DON'T FAIL RANDOMLY. THEY FAIL QUIETLY AT ONE POINT. FIND THAT POINT.

FOLLOW VERIQTA

YouTube /veriqta · GitHub /veriqta · LinkedIn /veriqta · X @veriqta · Instagram @veriqta · Telegram @veriqta

LINUX PERFORMANCE ANALYSIS

Measure. Analyze. Understand. Optimize.

1. IOSTAT

Analyze CPU, disk I/O and device performance.

COMMON COMMANDS

```
$ iostat -xz 1     # Extended stats
$ iostat -xk 1     # In KB
$ iostat -d 1 5    # Disk only
$ iostat -p ALL 1  # Per CPU
```

KEY METRICS

- %util > 70% = Disk busy
- await > 30ms = Slow response
- svctm High = Device issue
- r/s, w/s High = I/O pressure

2. VMSTAT

Report virtual memory, processes, CPU and more.

COMMON COMMANDS

```
$ vmstat 1      # Summary
$ vmstat 1 5    # 5 samples
$ vmstat -s     # Summary stats
$ vmstat -d     # Disk stats
```

KEY METRICS

- r Run queue (high = CPU busy)
- b Blocked processes
- si/so Swap in/out (should be 0)
- free Free memory (low is bad)
- wa IO wait (high = disk issue)

3. SAR

System Activity Reporter. Historical performance data.

COMMON COMMANDS

```
$ sar -u 1 5      # CPU usage
$ sar -r 1 5      # Memory usage
$ sar -b 1 5      # I/O & paging
$ sar -d 1 5      # Disk usage
$ sar -n DEV 1 5  # Network dev
$ sar -P ALL 1 5  # Per CPU
```

KEY METRICS

- %usr, %sys, %iowait
- pswpin/s, pswpout/s
- kbread/s, kbwrtn/s
- rxpck/s, txpck/s

4. TOP

Real-time overview of processes and resources.

COMMON COMMANDS

```
$ top          # Interactive
Keys:
  P  Sort by CPU
  M  Sort by Memory
  1  Per-CPU view
  h  Help
```

KEY METRICS

- %CPU High process CPU
- %MEM High memory usage
- Load Avg System load
- Tasks Running vs total

5. HTOP

Better interactive process viewer than top.

COMMON COMMANDS

```
$ htop   # Run htop
F5       # Tree view
F6       # Sort options
F9       # Kill process
F10      # Quit
```

KEY METRICS

- CPU% per core
- Memory and swap usage
- Process tree view
- Easy filtering and search

6. PIDSTAT

Monitor per-process CPU, memory, I/O and more.

COMMON COMMANDS

```
$ pidstat -u 1 5     # CPU usage
$ pidstat -r 1 5     # Memory usage
$ pidstat -d 1 5     # Disk I/O
$ pidstat -t 1 5     # Threads
$ pidstat -p <pid> 1 5  # Specific PID
$ pidstat -w 1 5     # Context switch
```

KEY METRICS

- %CPU High = CPU hog
- RSS High = Memory hog
- kB_rd/s, kB_wr/s High I/O
- cswch/s High = Context switching
- nvcswch/s Too high = Overhead

PERFORMANCE ANALYSIS WORKFLOW

1. **Detect** — Obseroe symptoms (Alerts, slow apps, high load)
2. **Measure** — Collect metrics (using tools above)
3. **Analyze** — Identify patterns and anomalies
4. **Identify Bottleneck** — Find the resource causing slowdown
5. **Act** — Optimize, tune or scale
6. **Verify** — Re-measure and confirm improvement

BOTTLENECK IDENTIFICATION GUIDE

SYMPTOM	LIKELY BOTTLENECK	KEY INDICATORS	TOOLS TO USE	WHAT TO CHECK
High Load Average	CPU Saturation	r > cores, top shows high %CPU	top, htop, sar -u	CPU intensive process, optimization, scale
High I/O Wait	Disk I/O Bottleneck	iowait > 20%, await > 30ms, %util > 70%	iostat, sar -d, vmstat	Disk type, queue depth, slow disks
High Memory Usage	Memory Pressure	free low, si/so high, swap usage	vmstat, sar -r, pidstat -r	Memory leaks, high cache, swap usage
High Swap Usage	Insufficient RAM	si/so > 0, swap used, OOM kills	vmstat, sar -r, dmesg	Add RAM, optimize apps, reduce usage
High Disk Activity	Disk Contention	r/s or w/s high, %util high	iostat, pidstat -d	Large I/O process, logs, DB, backups
High Network Usage	Network Bottleneck	rx/tx high, dropped packets	sar -n DEV, iftop, nload	Bandwidth, MTU, network congestion
Too Many Context Switches	CPU Overhead	cswch/s or nvcswch/s very high	pidstat -w, vmstat	Too many threads, poor app design

BEST PRACTICES

- ☑ Always establish a performance baseline.
- ☑ Monitor trends, not just real-time spikes.
- ☑ Correlate metrics from multiple tools.
- ☑ Focus on the biggest bottleneck first.
- ☑ Document findings and changes.

QUICK COMMAND CHEAT SHEET

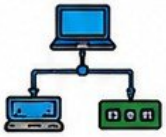
```
$ uptime                    # System load
$ free -h                   # Memory overview
$ df -h                     # Disk usage
$ ps aux --sort=-%cpu | head  # Top CPU processes
$ ps aux --sort=-%mem | head  # Top MEM processes
$ dmesg | tail -50          # Recent kernel messages
$ journalctl -xe            # System logs (systemd)
```

REMEMBER

- ☑ CPU, Memory, Disk, Network are the 4 pillars.
- ☑ Find the bottleneck, not just the symptom.
- ☑ Data-driven decisions beat guesswork.

PERFORMANCE IS NOT JUST ABOUT SPEED. IT IS ABOUT CONSISTENCY, RELIABILITY AND USER EXPERIENCE.

NETWORKING TROUBLESHOOTING



Page 11: OSI Model for Troubleshooting

The OSI model is your map. Troubleshoot layer by layer, isolate the fault, fix with precision.

LAYER-BY-LAYER INVESTIGATION

Start at Layer 1 and move up.
Stop when you find the fault.

LAYER	LAYER NAME	WHAT IT DOES	COMMON SYMPTOMS	HOW TO CHECK (EXAMPLES)
7	APPLICATION End-User Services	Provides network services to applications.	• App not responding • Service unreachable • Authentication failures	• curl, wget • telnet <host> <port> • nc -vz <host> <port> • Check app logs
6	PRESENTATION Data Format & Encryption	Data translation, compression and encryption.	• SSL/TLS errors • Data encoding issues • Certificate problems	• openssl s_client -connect <host>:<port> • Check certificate validity • Review encryption settings
5	SESSION Session Management	Manages sessions, dialogs and connections.	• Session timeouts • Keepalive failures • Connection resets	• ss -tan | grep ESTAB • Check idle timeout settings • Review session logs
4	TRANSPORT End-to-End Delivery	Ensures reliable delivery, segmentation, ports.	• Port closed/filtered • Connection refused • Packet loss	• ss -tulnp • nc -vz <host> <port> • traceroute <host>
3	NETWORK Logical Addressing	Routing, IP addressing and path selection.	• No route to host • Destination unreachable • High latency	• ip route • ping <ip> • traceroute <ip>
2	DATA LINK MAC & Local Delivery	Framing, MAC addressing, error detection.	• ARP failures • MAC flapping • CRC errors	• ip neigh show • arp -a • ethtool -S <iface>
1	PHYSICAL Bits & Signals	Transmits raw bits over physical medium.	• Link down • No lights • Cable/Hardware faults	• ip link show • ethtool <iface> • Check cables, NIC, switch ports

COMMON SYMPTOMS (AT A GLANCE)

- Cannot reach website (L7/L3/L4 issues)
- Connection refused (L4)
- No route to host (L3)
- Network unreachable (L3)
- ARP not resolving (L2)
- Link down / No lights (L1)
- SSL / Certificate error (L6)
- Timeout / Slow response (L4/L7)

ISOLATION STRATEGY

1. Define the problem clearly. What is failing?
2. Reproduce the issue. Can you see it consistently?
3. Start at Layer 1. Verify the foundation.
4. Move up the OSI model one layer at a time.
5. Isolate the failing layer. Prove where it breaks.
6. Fix the root cause, not just the symptom.
7. Verify end-to-end. Confirm full recovery.

TROUBLESHOOTING WORKFLOW (LAYERED APPROACH)

1. Observe — Collect symptoms and user impact.
2. Identify Layer — Map the problem to OSI layer.
3. Test & Verify — Run checks for that layer.
4. Isolate — Narrow down the faulty component.
5. Resolve — Apply the fix or workaround.
6. Validate — Test end-to-end functionality.
7. Document — Record findings, root cause, fix.

PRO TIP

- ✓ Do not guess. Always test.
- ✓ Follow the OSI model.
- ✓ Change one thing at a time.
- ✓ Document everything.
- ✓ Think like a detective, not a technician.

QUICK COMMAND REFERENCE

```
ping <host>                  # Reachability (L3)
traceroute <host>            # Path discovery (L3)
ss -tulnp                    # Ports & connections (L4)
ip addr / ip link show       # Interface status (L1/L2)
ip route                     # Routing table (L3)
arp -a / ip neigh            # ARP & neighbor (L2)
dnslookup / dig <host>       # DNS lookup (L7)
curl -v <url>                # Application test (L7)
openssl s_client -connect <host>:<port> # TLS (L6)
iptables -L -n -v            # Firewall rules (L3/L4)
ethtool <iface>              # NIC & link info (L1)
mii-tool / lspci             # Hardware info (L1)
```

REMEMBER

- ☆ Every problem lives on a layer.
- ☆ Find the layer.
- ☆ Fix the layer.
- ☆ Verify the path.

**SENIOR ENGINEERS DO NOT JUMP TO SOLUTIONS. THEY FOLLOW A PROCESS.
THE OSI MODEL IS THAT PROCESS.**

DNS TROUBLESHOOTING

When names don't resolve, nothing works.
Understand DNS deeply. Troubleshoot with precision.

1. RESOLUTION FAILURES

- Domain does not resolve
- Intermittent resolution
- Works on some networks
- Works for some, not others
- NXDOMAIN / SERVFAIL
- Timeout / No response

HOW TO CHECK

```
$ dig example.com
$ dig @8.8.8.8 example.com
$ nslookup example.com
$ host example.com
$ getent hosts example.com
```

HOW TO FIX

- ☑ Verify domain spelling
- ☑ Check authoritative DNS
- ☑ Verify DNS server health
- ☑ Check network/firewall
- ☑ Review DNS records

2. SPLIT DNS

- Different answers inside vs outside network
- Internal tools not reachable from outside
- VPN works, public fails

HOW TO CHECK

```
$ dig @internal-dns example.com
$ dig @public-dns example.com
    # Compare the answers
$ nslookup example.com 8.8.8.8
$ nslookup example.com 10.0.0.53
```

HOW TO FIX

- ☑ Validate split DNS zones
- ☑ Ensure correct view/zone
- ☑ Check DNS forwarders
- ☑ Review NAT / VPN DNS
- ☑ Document DNS behavior

3. TTL (TIME TO LIVE)

- Changes not reflected
- Old records still cached
- Long TTL delays updates
- Short TTL increases load

HOW TO CHECK

```
$ dig +noall +answer example.com
$ dig example.com | grep TTL
$ dig SOA example.com
    # Check SOA minimum TTL
```

HOW TO FIX

- ☑ Lower TTL BEFORE changes
- ☑ Wait for TTL to expire
- ☑ Purge caches if possible
- ☑ Use smart TTL strategy
- ☑ Balance performance & freshness

4. DNS CACHE ISSUES

- Stale / wrong records
- ISP or resolver cache
- Local OS cache
- Browser/app cache

HOW TO CHECK

```
$ ipconfig /displaydns   # Windows
$ scutil --dns           # macOS
$ systemd-resolve --statistics
$ dig @resolver example.com
```

HOW TO FIX

- ☑ Flush local OS cache
- ☑ Flush resolver cache
- ☑ Clear browser cache
- ☑ Use different DNS server
- ☑ Wait for TTL to expire

5. RECURSIVE LOOKUPS

- Recursion not allowed
- Firewall blocks recursion
- Forwarder misconfiguration
- Upstream DNS failures

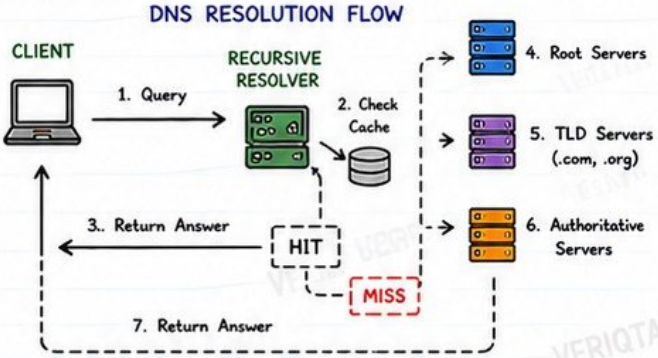
HOW TO CHECK

```
$ dig @dns-server example.com
$ dig +trace example.com
$ dig example.com +norecurse
$ dig example.com +recurse
```

HOW TO FIX

- ☑ Allow recursion (if resolver)
- ☑ Check forwarders
- ☑ Verify upstream DNS
- ☑ Open required ports (53 UDP/TCP)
- ☑ Test from different network

DNS RESOLUTION FLOW



COMMON DNS PROBLEMS & INDICATORS

PROBLEM	INDICATOR	POSSIBLE CAUSE	CHECK THIS
NXDOMAIN	Domain does not exist	Wrong domain	dig example.com
SERVFAIL	Server failure	DNS server issue	dig @dns example.com
TIMEOUT	No response	Network / Firewall	ping, telnet 53
REFUSED	Query refused	Recursion not allowed	dig +recurse
WRONG IP	Resolves to wrong IP	Wrong record / Cache	dig +trace
INTERMITTENT	Sometimes works	TTL / Cache / Network	Check TTL, traceroute

USEFUL DNS COMMANDS CHEAT SHEET

```
dig domain.com                  # Basic query
dig @8.8.8.8 domain.com         # Query specific DNS
dig domain.com A                # A record
dig domain.com AAAA             # IPv6 record
dig domain.com MX               # Mail exchange
dig domain.com NS               # Nameservers
dig domain.com TXT              # TXT record
dig +trace domain.com           # Full resolution path
dig +short domain.com           # Short answer only
nslookup domain.com             # Interactive lookup
host domain.com                 # Simple lookup
getent hosts domain.com         # System resolver test
```

BEST PRACTICES

- ☑ Use reliable and redundant DNS servers
- ☑ Monitor DNS latency and error rates
- ☑ Set appropriate TTL values
- ☑ Implement DNSSEC
- ☑ Document DNS architecture
- ☑ Test from multiple locations
- ☑ Plan for failover and DR
- ☑ Keep DNS simple and consistent

TOOLS YOU SHOULD KNOW

dig, nslookup, host
kdig, drill, delv
tcpdump / Wireshark
resolvectl, systemd-resolve
DNSViz, MXToolbox
Prometheus + Grafana
Route53 / Cloud DNS logs

QUICK CHECKLIST

- ☑ Can I resolve the domain?
- ☑ Which DNS server am I using?
- ☑ Is the DNS server reachable?
- ☑ Is the record correct?
- ☑ Is it cached somewhere?
- ☑ Is TTL delaying my change?
- ☑ Is recursion allowed?
- ☑ Is the network blocking DNS?
- ☑ Test from a different network.

PRO TIP: Always test DNS from the same network as the failing system. DNS behaves differently across networks.

REMEMBER: DNS is the first dependency. If DNS fails, everything fails.

LOAD BALANCER TROUBLESHOOTING

Traffic in, traffic out — but what happens in between matters.
Validate, isolate, fix, and keep users happy.

1. HEALTH CHECKS

- Determine if targets are healthy.
- Unhealthy targets are removed from rotation.
- Misconfigurations cause 5xx errors and downtime.

HOW TO CHECK

```
$ aws elbv2 describe-target-health
$ curl -v http://<target-ip>:<port>/
$ curl -I http://<elb-dns>/health
$ check health check path, port, protocol, matcher
```

HOW TO FIX

- ☑ Correct health check path
- ☑ Verify health check port
- ☑ Increase timeout/interval
- ☑ Fix application response
- ☑ Ensure security group allows health check traffic

2. TARGET GROUPS

- Targets register to target groups.
- Wrong targets, wrong port, or deregistration issues cause failures.

HOW TO CHECK

```
$ aws elbv2 describe-target-groups
$ aws elbv2 describe-target-health
$ check target IPs and status
$ verify target port and protocol
```

HOW TO FIX

- ☑ Register correct targets
- ☑ Verify target port/protocol
- ☑ Ensure instances are running
- ☑ Check security groups
- ☑ Remove unused/unhealthy targets

3. SESSION PERSISTENCE

Sticky Session

- Keeps user on same server.
- Misconfigurations cause login issues, cart loss.
- Important for stateful apps.

HOW TO CHECK

```
$ aws elbv2 describe-load-balancers
$ check stickiness.enabled
$ check cookie duration
$ test from multiple requests
```

HOW TO FIX

- ☑ Enable cookie-based stickiness
- ☑ Set appropriate duration
- ☑ Use application-level sessions (Redis, DB, etc.)
- ☑ Avoid sticky sessions when not required (scale better)

4. TIMEOUTS

- Idle timeout, connection timeout, or deregistration delay too low causes dropped connections.

HOW TO CHECK

```
$ aws elbv2 describe-load-balancers
$ check idle_timeout.timeout_seconds
$ check target response time
$ review application logs
```

HOW TO FIX

- ☑ Increase idle timeout
- ☑ Increase client keep-alive
- ☑ Optimize application response time
- ☑ Adjust deregistration delay

5. SSL TERMINATION

SSL

- Handles SSL/TLS encryption at the load balancer.
- Certificate or protocol mismatch causes errors.

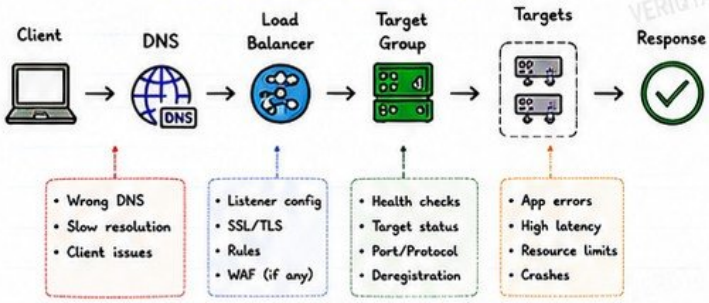
HOW TO CHECK

```
$ openssl s_client -connect <elb-dns>:443
$ check certificate expiry
$ check ssl policies
$ check security policy
```

HOW TO FIX

- ☑ Install valid certificate
- ☑ Renew expired certificates
- ☑ Use strong SSL policies
- ☑ Redirect HTTP to HTTPS
- ☑ Verify backend (if re-encrypting)

LOAD BALANCER REQUEST FLOW



COMMON LAYER 7 SYMPTOMS & CAUSES

SYMPTOM	POSSIBLE CAUSE	CHECK THIS
502 Bad Gateway	Targets unhealthy	Target health, app logs
504 Gateway Timeout	Targets slow/unresponsive	Timeouts, app performance
503 Service Unavailable	No healthy targets	Target group health
Random logouts	Sticky sessions off/short	Stickiness config
SSL errors	Cert expired/mismatch	Certificate, SSL policy
Some users fail	Cross-AZ / Network issue	Security groups, Routing
High 5xx errors	App errors / Overload	App logs, CPU, Memory

TROUBLESHOOTING WORKFLOW

1. **Confirm Issue** — Collect symptoms, time, and user impact.
2. **Check LB Status** — Verify LB state, listeners, rules.
3. **Check Targets** — Verify target group health and status.
4. **Review Logs & Metrics** — Check access logs, CloudWatch metrics.
5. **Validate SSL & Timeouts** — Check certs, policies, and timeout values.
6. **Isolate & Fix** — Apply fix, update configurations.
7. **Verify & Monitor** — Test from multiple locations, monitor.

PRO TIP

Always test from the user's perspective.
Use curl, browser, and different networks.
Check metrics before and after changes, not just logs.

USEFUL AWS CLI COMMANDS

```
$ aws elbv2 describe-load-balancers
$ aws elbv2 describe-listeners --load-balancer-arn <arn>
$ aws elbv2 describe-rules --listener-arn <arn>
$ aws elbv2 describe-target-groups
$ aws elbv2 describe-target-health --target-group-arn <arn>
```

KEY METRICS (CLOUDWATCH)

- HealthyHostCount
- UnHealthyHostCount
- HTTPCode_Target_5XX_Count
- TargetResponseTime
- ActiveConnectionCount
- NewConnectionCount

BEST PRACTICES

- ☑ Keep targets stateless or use shared session store
- ☑ Set health checks to be lightweight and fast
- ☑ Use connection draining (deregistration delay)
- ☑ Monitor metrics and set alarms
- ☑ Document all LB configurations
- ☑ Test failover and scaling regularly

QUICK CHECKLIST

- ☑ DNS resolves
- ☑ LB is healthy
- ☑ Targets are healthy
- ☑ Security groups allow traffic
- ☑ Health check path works
- ☑ Timeouts configured
- ☑ SSL certificate valid

A LOAD BALANCER IS ONLY AS GOOD AS ITS HEALTH CHECKS AND TARGETS.
TRUST, BUT VERIFY CONSTANTLY.

TLS AND CERTIFICATE ISSUES

Encryption is only as strong as your weakest link.
Validate, monitor, and enforce secure connections.

1. EXPIRED CERTIFICATES

- Certificate is past expiry date.
- Browsers block the connection.
- Services fail health checks.
- Users see security warnings.

HOW TO CHECK

```
$ openssl s_client -connect example.com:443 -servername example.com
$ openssl x509 -in cert.pem -noout -dates
$ echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -enddate
```

HOW TO FIX

- ☑ Renew certificate before expiry
- ☑ Automate renewal (ACME/Let's Encrypt)
- ☑ Update certificate on all servers
- ☑ Reload service / load balancer
- ☑ Monitor expiry and set alerts

2. TRUST CHAINS

- Missing intermediate certificates.
- Incomplete chain delivery.
- "Untrusted" or "Unknown CA" errors.
- Works in some clients, fails in others.

HOW TO CHECK

```
$ openssl s_client -connect example.com:443 -showcerts
$ openssl verify -CAfile ca-bundle.crt cert.pem
$ curl -vI https://example.com
```

HOW TO FIX

- ☑ Install full certificate chain
- ☑ Include intermediates on server
- ☑ Use a trusted CA
- ☑ Verify chain with multiple clients
- ☑ Keep CA bundle updated

3. HANDSHAKE FAILURES

- TLS handshake cannot complete.
- Version/cipher incompatibility.
- Protocol errors.
- Connection resets during handshake.
- Common with old clients/servers.

HOW TO CHECK

```
$ openssl s_client -connect example.com:443 -tls1_2
$ openssl s_client -msg -connect example.com:443
$ curl -v https://example.com --tlsv1.2
```

HOW TO FIX

- ☑ Enable required TLS versions
- ☑ Fix protocol/cipher configuration
- ☑ Check server time (NTP sync)
- ☑ Review firewall/IDS interference
- ☑ Update clients/servers if needed

4. MUTUAL TLS (mTLS)

- Client certificate not presented.
- Invalid client certificate.
- Trust issues between services.
- Authorization failures.

HOW TO CHECK

```
$ openssl s_client -connect example.com:443 -cert client.pem -key client.key -CAfile ca.crt
$ curl --cert client.pem --key client.key https://example.com
$ openssl verify -CAfile ca.crt client.crt
```

HOW TO FIX

- ☑ Provide valid client certificate
- ☑ Ensure server trusts client CA
- ☑ Check certificate permissions
- ☑ Verify SAN matches service
- ☑ Sync certs across environments

5. CIPHER MISMATCHES

- No common ciphers.
- Weak/disabled ciphers.
- High security clients fail.
- "Handshake failure" errors.

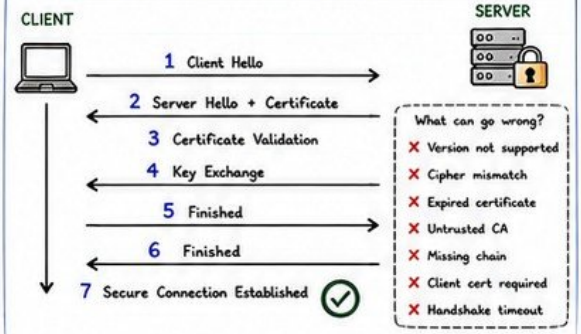
HOW TO CHECK

```
$ nmap --script ssl-enum-ciphers -p 443 example.com
$ openssl ciphers -v
$ openssl s_client -connect example.com:443 -cipher 'ECDHE'
$ testssl.sh example.com
```

HOW TO FIX

- ☑ Enable strong modern ciphers
- ☑ Align cipher suites (client/server)
- ☑ Disable weak/legacy ciphers
- ☑ Update TLS libraries/versions
- ☑ Test with modern compatibility

TLS HANDSHAKE FLOW (SIMPLIFIED)



COMMON TLS ERRORS & WHAT THEY MEAN

ERROR MESSAGE	WHAT IT MEANS	LIKELY CAUSE	CHECK THIS
certificate has expired	Cert is past expiry date	Not renewed	openssl x509 -dates
unable to get local issuer certificate	Missing intermediate CA	Incomplete chain	openssl s_client -showcerts
self signed certificate	Cert not signed by trusted CA	Private/internal CA	Verify CA trust
handshake failure	Handshake could not complete	Cipher/Version mismatch	Check TLS versions/ciphers
no shared cipher	No common ciphers	Strict policies	testssl.sh / nmap
bad certificate	Certificate invalid	Corrupt/Incorrect	openssl verify
unknown ca	CA not trusted	Missing CA bundle	Install CA / Update bundle
alert bad certificate	Client certificate invalid	mTLS issue	Verify client cert

ESSENTIAL TLS DIAGNOSTIC COMMANDS

`$ openssl s_client -connect example.com:443`	Basic TLS connection test
`$ openssl s_client -connect example.com:443 -servername example.com`	Test with SNI (virtual hosts)
`$ openssl s_client -connect example.com:443 -showcerts`	Show certificate chain
`$ openssl s_client -connect example.com:443 -tls1_2`	Test specific TLS version
`$ curl -vI https://example.com`	Verbose HTTP request + TLS
`$ testssl.sh example.com`	Comprehensive TLS test
`$ nmap --script ssl-enum-ciphers -p 443 example.com`	List supported ciphers
`$ openssl x509 -in cert.pem -text -noout`	Inspect certificate details

CERTIFICATE INSPECTION CHECKLIST

- ☑ Domain/SAN matches
- ☑ Validity dates (not expired)
- ☑ Issuer is trusted
- ☑ Full chain presented
- ☑ Key usage is correct
- ☑ Signature algorithm is strong
- ☑ Revocation (OCSP/CRL) OK
- ☑ Certificate not self-signed
- ☑ Not using deprecated algorithms

TOOLS YOU SHOULD KNOW

OpenSSL	- Core diagnostic tool
curl	- Quick connection testing
testssl.sh	- TLS security scanner
nmap	- Cipher enumeration
Wireshark	- Deeper packet analysis
ssllabs.com	- SSL/TLS online test
CFSSL / Smallstep	- Internal PKI tools

PRO TIPS

- ☆ Automate certificate issuance & renewal.
- ☆ Monitor expiry 30+ days in advance.
- ☆ Use strong ciphers and modern protocols.
- ☆ Test from multiple clients and networks.
- ☆ Document your TLS standards.

TLS HEALTH MONITORING

- ☑ Certificate expiry alerts
- ☑ Handshake error rate
- ☑ TLS version distribution
- ☑ Cipher usage reports
- ☑ mTLS authentication failures
- ☑ SSL Labs grade tracking

QUICK CHECKLIST

- ☑ Can the client reach the server?
- ☑ Is the certificate valid and trusted?
- ☑ Is the full chain delivered?
- ☑ Are TLS versions compatible?
- ☑ Are ciphers compatible?
- ☑ Is mTLS configured correctly?
- ☑ Are time and date correct (NTP)?
- ☑ Does the application accept the cert?

REMEMBER

TLS problems usually hide in one of five places: certificate, chain, handshake, trust, or configuration. Test. Verify. Trust, but verify.

SECURE CONNECTIONS ARE BUILT, NOT ASSUMED. VALIDATE EVERYTHING.

NETWORK PERFORMANCE PROBLEMS

Slow networks are expensive. Measure, analyze, and eliminate bottlenecks.

1. LATENCY (DELAY)

- Time taken for packets to reach destination.
- Causes slow apps, timeouts, poor user experience.
- Measured in milliseconds (ms).

HOW TO CHECK

```
$ ping -c 10 <host>
$ mtr <host>
$ traceroute <host>
$ ping -c 10 <host>
```

HOW TO FIX

- ☑ Fix routing issues
- ☑ Use faster network paths
- ☑ Reduce hops
- ☑ Optimize applications
- ☑ Use CDN closer to users

2. PACKET LOSS

- Packets fail to reach destination.
- Causes retries, slow apps, poor quality.
- Measured in percentage (%).

HOW TO CHECK

```
$ ping -c 100 <host>
$ mtr -rwc 100 <host>
$ smokeping (graph)
$ iperf3 -u -b 10M -l 1470
$ tcptraceroute <host>
```

HOW TO FIX

- ☑ Fix bad links or Wi-Fi
- ☑ Replace failing cables
- ☑ Check NIC / drivers
- ☑ Reduce congestion
- ☑ Use error-correcting path

3. MTU (MAX TRANSMISSION UNIT)

1500

- Packet size too big for path.
- Causes fragmentation or drops.
- Common issue with VPN, tunnels, cloud links.

HOW TO CHECK

```
$ ping -M do -s 1472 <host>
$ tracepath <host>
$ ip link show
$ cat /proc/sys/net/ipv4/ip_no_pmtu_disc
```

HOW TO FIX

- ☑ Set correct MTU
- ☑ Adjust MSS (TCP clamping)
- ☑ Enable PMTUD
- ☑ Fix tunnel/VPN MTU
- ☑ Standard MTU = 1500 (or 9000 for jumbo)

4. THROUGHPUT (BANDWIDTH)

- Amount of data successfully transferred.
- Lower than expected speed.
- Measured in Mbps/Gbps.

HOW TO CHECK

```
$ iperf3 -c <server>
$ iperf3 -c <server> -R
$ ethtool <iface>
$ nload / iftop / nethogs
$ vnstat -i <iface>
```

HOW TO FIX

- ☑ Remove bandwidth hogs
- ☑ Upgrade links if needed
- ☑ Tune TCP window size
- ☑ Use compression / caching
- ☑ Balance load

5. CONGESTION

- Too much traffic for available capacity.
- Causes latency, loss, jitter.
- Queue builds up.

HOW TO CHECK

```
$ ss -ti
$ netstat -s
$ sar -n DEV 1
$ iftop -i <iface>
$ tc -s qdisc show dev <iface>
```

HOW TO FIX

- ☑ Shape or limit traffic
- ☑ Prioritize critical traffic (QoS)
- ☑ Add capacity
- ☑ Load balance
- ☑ Use application throttling

6. JITTER (VARIATION)

- Variation in packet arrival time.
- Affects voice, video, real-time apps.
- Measured in ms.

HOW TO CHECK

```
$ mtr -rwc 100 <host>
  (look at "jitter" column)
$ ping -D -i 0.2 <host>
$ smokeping (jitter graph)
```

HOW TO FIX

- ☑ Reduce congestion
- ☑ Fix wireless interference
- ☑ Use QoS for voice/video
- ☑ Optimize routing
- ☑ Use wired where possible

NETWORK PERFORMANCE TROUBLESHOOTING WORKFLOW

1. **Define the Problem** — What's the symptom? Who is affected?
2. **Measure** — Collect baseline metrics (latency, loss, throughput).
3. **Isolate** — Check path hop-by-hop. Find where it breaks.
4. **Analyze** — Identify patterns and root cause.
5. **Fix** — Apply changes to remove bottleneck.
6. **Validate** — Re-test and confirm improvement.
7. **Document** — Record findings and lessons learned.

PRO TIP

Always test from both sides (client and server). The problem is not always where you think it is.

ESSENTIAL COMMAND CHEAT SHEET

PURPOSE	COMMAND
Test connectivity & latency	ping -c 10 <host>
Detailed path, loss, latency, jitter	mtr -rwc 100 <host>
Show route path	traceroute <host>
MTU discovery	tracepath <host>
Test bandwidth (TCP)	iperf3 -c <server>
Test bandwidth (UDP)	iperf3 -u -b 10M -c <server>
Real-time bandwidth usage	iftop -i <iface>
Interface stats	sar -n DEV 1
TCP connection stats	ss -ti
Policy based routing / MTU	ip route show | ip link show
Packet capture	tcpdump -i <iface> -w cap.pcap

KEY METRICS REFERENCE

METRIC	GOOD	WARNING	CRITICAL
Latency (ms)	< 50	50 - 150	> 150
Packet Loss (%)	< 1%	1% - 5%	> 5%
Jitter (ms)	< 20	20 - 50	> 50
Throughput (%)	> 90%	70% - 90%	< 70%
Retransmissions (%)	< 1%	1% - 3%	> 3%
TCP Errors	0	Few	Many
Queue Length	Low	Growing	Full / Dropping

COMMON CAUSES BY AREA

AREA	POSSIBLE CAUSES
Network Path	Bad routing, long path, ISP issues
Link / Physical	Bad cables, weak Wi-Fi, duplex mismatch
Devices	Overloaded routers, old firmware, bugs
Configuration	Wrong MTU, QoS missing, wrong routing
Security	Firewalls dropping, IDS/IPS, NAT issues
Applications	Too many connections, noisy neighbors
DNS / Name Resolution	Slow DNS, retries, wrong servers

ISOLATION STRATEGY

1. Test locally (localhost).
2. Test gateway. If local works but gateway fails, check local network.
3. Test next hop (router).
4. Test upstream providers.
5. Test destination. Compare metrics at each hop.
6. Pinpoint the first bad hop.

USEFUL TOOLS

- **mtr** – Best all-in-one tool.
- **iperf3** – Bandwidth testing.
- **Wireshark / tcpdump** – Deep packet analysis.
- **SmokePing** – Long term latency & loss tracking.
- **nload / iftop** – Real-time bandwidth.
- **vnstat** – Historical usage.

BEST PRACTICES

- ☑ Monitor continuously.
- ☑ Set alerts on key metrics.
- ☑ Keep paths short and efficient.
- ☑ Use modern protocols (TCP BBR, HTTP/2/3).
- ☑ Avoid unnecessary hops and firewalls.
- ☑ Plan capacity for peak usage.
- ☑ Document network baselines.
- ☑ Test after every change.

QUICK CHECKLIST

- ☑ Is the host reachable?
- ☑ Is DNS fast and correct?
- ☑ Is latency acceptable?
- ☑ Is packet loss within limits?
- ☑ Is jitter within limits?
- ☑ Is throughput as expected?
- ☑ Is MTU configured correctly?
- ☑ Is there congestion on the path?
- ☑ Are routes optimal?

MEASURE FIRST. ASSUME LAST. FIX ROOT CAUSE.

GIT TROUBLESHOOTING

Every engineer uses Git. Senior engineers fix Git.
Diagnose, understand, and recover with confidence.

1. MERGE CONFLICTS

- Same lines changed in different commits.
- Git cannot auto-merge.
- Needs manual resolution.

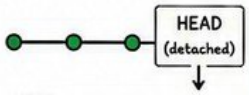
HOW TO CHECK

```
$ git status
  (look for "both modified")
$ git diff --name-only --diff-filter=U
```

HOW TO FIX

- ☑ Open conflicted files
- ☑ Resolve conflicts manually
- ☑ git add <file>
- ☑ git commit

2. DETACHED HEAD



- HEAD points to a commit, not a branch.
- New commits can be lost if not saved.

HOW TO CHECK

```
$ git status
(You are in 'detached HEAD' state...)
$ git log --oneline -5
```

HOW TO FIX

- ☑ Create a branch
  `git switch -c my-branch`
- ☑ Or switch to existing branch
  `git switch main`

3. REBASE FAILURES

- Conflicts during rebase.
- Interrupted rebase.
- Upstream changes.

HOW TO CHECK

```
$ git status
  (rebase in progress...)
$ ls .git/rebase-apply
```

HOW TO FIX

- ☑ Resolve conflicts
- ☑ git add <file>
- ☑ git rebase --continue
- ☑ Or abort: git rebase --abort

4. AUTHENTICATION ISSUES

- Wrong credentials / token.
- Expired token.
- Permission denied.

HOW TO CHECK

```
$ git remote -v
$ git ls-remote origin
$ GIT_TRACE=1 git push
```

HOW TO FIX

- ☑ Update credentials / token
- ☑ Check SSH key / access
- ☑ Re-authenticate
- ☑ Verify repo permissions

5. REPOSITORY CORRUPTION

- Corrupted objects or refs.
- Interrupted operations.
- Disk or network issues.

HOW TO CHECK

```
$ git fsck --full --strict
$ git show-ref
$ git count-objects -v
```

HOW TO FIX

- ☑ Run git fsck and review
- ☑ Remove broken refs
- ☑ git gc --prune=now
- ☑ Restore from backup if needed

COMMON GIT PROBLEMS & QUICK DIAGNOSIS

PROBLEM	SYMPTOMS	CHECK THIS	LIKELY CAUSE
Merge conflicts	Conflict markers in files	git status	Same lines changed
Detached HEAD	Can't find new commits	git status	HEAD not on branch
Rebase failure	Rebase stopped	git status	Conflicts / errors
Auth failure	Permission denied	git remote -v	Bad creds / token
Corrupt repo	fsck errors	git fsck --full	Broken objects / refs
Push rejected	Non-fast-forward	git log --graph	Diverged branches
Lost commits	Missing commits	git reflog	Reset / rebase
Large repo slow	Slow operations	git gc --aggressive	Too many loose objs

GIT TROUBLESHOOTING WORKFLOW

1. **Identify** — What changed? When did it break? Collect symptoms.
2. **Check** — Run status, logs, and diagnostics.
3. **Analyze** — Understand the cause. Isolate the issue.
4. **Fix** — Apply the right fix. Test.
5. **Validate** — Confirm everything works.

GIT COMMAND QUICK REFERENCE

General	Branches	Rebase	Recovery
git status	git branch -a	git rebase <branch>	git reflog
git log --oneline --graph	git switch <branch>	git rebase --continue	git reset --hard <commit>
git diff	git switch -c <new>	git rebase --abort	git stash
git show <commit>	git merge <branch>	git rebase --skip	git fsck --full --strict
git remote -v			

REAL-WORLD SCENARIOS

1. You pulled changes and now you have conflicts.
2. You committed in detached HEAD and lost your work.
3. Rebase stopped in the middle with conflicts.
4. Push rejected because remote has new commits.
5. You cannot push due to authentication failure.
6. Git operations are extremely slow.
7. Repository seems corrupted after a crash.

PRO TIPS

- ✓ Use branches for experiments.
- ✓ Commit small and often.
- ✓ Pull with --rebase to keep history clean.
- ✓ Use git reflog — it saves lives.
- ✓ Always test in a safe branch.
- ✓ Back up important repos.
- ✓ Automate auth with SSH or tokens.

BEST PRACTICES

- ☑ Keep main branch protected.
- ☑ Review before merging.
- ☑ Use meaningful commit messages.
- ☑ Avoid force push on shared branches.
- ☑ Use .gitignore properly.
- ☑ Keep credentials secure.
- ☑ Regularly prune and clean repo.

QUICK CHECKLIST

- ☑ Is the repo healthy?
- ☑ Are you on the right branch?
- ☑ Any conflicts or errors?
- ☑ Credentials valid?
- ☑ Remote reachable?
- ☑ Backup available?
- ☑ Can you reproduce the issue?

GIT IS POWERFUL. BUT WITH GREAT POWER COMES GREAT RESPONSIBILITY.
UNDERSTAND THE HISTORY. CONTROL THE FUTURE.

FOLLOW VERIQTA

YouTube /veriqta · GitHub /veriqta · LinkedIn /veriqta · X @veriqta · Instagram @veriqta · Telegram @veriqta

CI/CD PIPELINE FAILURES

Pipelines don't fail randomly.
They fail for a reason. Find it. Fix it. Improve it.

1. BUILD FAILURES

- Compilation errors
- Dependency issues
- Missing tools or caches
- Misconfigured build steps

HOW TO CHECK

```
$ pipeline logs
$ build output
$ dependency versions
$ cache restore step
$ environment variables
```

HOW TO FIX

- ☑ Read error carefully
- ☑ Fix code or config
- ☑ Update dependencies
- ☑ Clear or restore caches
- ☑ Re-run build step locally

2. TEST FAILURES

- Unit test failures
- Integration test failures
- Flaky tests
- Test environment issues

HOW TO CHECK

```
$ test logs
$ test reports
$ failed test cases
$ environment config
$ test database state
```

HOW TO FIX

- ☑ Re-run failed tests locally
- ☑ Fix code or test data
- ☑ Stabilize flaky tests
- ☑ Check test environment
- ☑ Improve test coverage

3. DEPLOYMENT FAILURES

- Bad configuration
- Service unreachable
- Health check failures
- Permission or secret issues

HOW TO CHECK

```
$ deploy logs
$ kubectl get pods
$ kubectl describe pod
$ service / target health
$ environment & secrets
```

HOW TO FIX

- ☑ Check deployment config
- ☑ Verify environment variables
- ☑ Fix broken manifests
- ☑ Check service health
- ☑ Validate secrets/permissions

4. ROLLBACK STRATEGY

- Fast rollback saves users
- Keep deployments reversible
- Automate where possible
- Always test rollback process

HOW TO CHECK

```
$ deployment history
$ previous stable version
$ rollback script
$ backup / artifacts
$ database migration state
```

HOW TO FIX

- ☑ Rollback to last known good
- ☑ Verify system stability
- ☑ Check database compatibility
- ☑ Communicate the incident
- ☑ Run post-incident review

TYPICAL PIPELINE STAGES	COMMON FAILURE CAUSES
CODE COMMIT	• Bad merge • Broken config • Missing files
BUILD	• Compile errors • Bad dependencies • Tool version
TEST	• Broken tests • Env mismatch • Flaky tests
PACKAGE	• Artifact errors • Versioning • Missing files
DEPLOY	• Config errors • Infra issues • Permission
VERIFY	• Health check fail • Monitor alerts • User errors

CI/CD PIPELINE TROUBLESHOOTING WORKFLOW

1. **Detect** — Alert or failure is triggered.
2. **Identify** — Find the failed stage quickly.
3. **Investigate** — Check logs, artifacts, and environment.
4. **Fix** — Resolve the root cause and test locally.
5. **Validate** — Re-run pipeline and confirm success. Monitor.

LOGS ARE YOUR BEST FRIEND

```
$ pipeline logs --follow
$ Jenkins: Check Console Output
$ GitLab: Check Job Logs
$ GitHub Actions: View Workflow Logs
$ kubectl logs -f <pod-name>
$ docker logs <container-id>
$ tail -f /var/log/<app>.log
```

PRO TIP

Always understand what the pipeline is doing at each step. Add clear logging, meaningful alerts, and automated notifications. Observability turns failures into fast fixes.

COMMON SYMPTOMS, CHECKS & ROOT CAUSES

SYMPTOM	POSSIBLE CAUSES	CHECK THIS	ROOT CAUSE EXAMPLES
Build failed	Code error, dependency, tool	Build logs, compiler output	Syntax error, wrong version, missing lib
Tests failed	Code bug, flaky test, data	Test logs, reports, env	Assertion failed, bad mock, timing issue
Deploy failed	Config, permission, infra	Deploy logs, k8s events	Bad manifest, missing secret, quota
Service unhealthy	App crash, config, DB	Pod logs, metrics, health	App error, wrong port, DB down
Timeout / stuck	Slow build, network, resource	Step duration, network logs	Large artifact, bandwidth, CPU limit
Rollback triggered	Health check failed	Alerts, deploy history	New version unstable, bad release

AUTOMATED ROLLBACK STEPS

1. Detect failure (health check / alert)
2. Stop traffic to failed version
3. Rollback to last stable version
4. Run DB migration rollback (if needed)
5. Verify system is healthy
6. Notify team and stakeholders
7. Create incident report

USEFUL COMMANDS

```
$ git log --oneline --graph
$ git diff
$ kubectl get pods -o wide
$ kubectl describe pod <pod>
$ docker ps -a
$ docker logs <container-id>
$ aws s3 ls s3://bucket --recursive
```

BEST PRACTICES

- ☑ Keep builds fast & repeatable
- ☑ Fail early, fail loud
- ☑ Use caches wisely
- ☑ Automate tests & quality gates
- ☑ Version everything
- ☑ Monitor and alert everything
- ☑ Document runbooks & rollback

TOOLS YOU CAN USE

Jenkins, GitLab CI, GitHub Actions
Argo CD, Spinnaker
SonarQube, Trivy
Prometheus, Grafana, Alertmanager
Datadog, New Relic
ELK / OpenSeerch

QUICK CHECKLIST

- ☑ Did the code change cause it?
- ☑ Are dependencies correct?
- ☑ Are tests stable and reliable?
- ☑ Is the environment healthy?
- ☑ Are configs and secrets valid?
- ☑ Is the target reachable?
- ☑ Can we rollback safely?

A STRONG PIPELINE BUILDS FAST. A RELIABLE PIPELINE RECOVERS FASTER.
AUTOMATE EVERYTHING. OBSERVE EVERYTHING. TRUST NOTHING.

Jenkins

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

JENKINS TROUBLESHOOTING

Every pipeline failure has a signal.
Find it. Understand it. Fix it. Automate it.

1. AGENTS OFFLINE

- Agent disconnected
- Network issues
- Agent crashed
- Wrong labels / offline

HOW TO CHECK

```
$ jenkins-cli list-jnlp
$ jenkins-cli list-nodes
$ java -jar agent.jar -jnlpUrl ...
$ Check node status in UI
```

HOW TO FIX

- ☑ Restart agent service
- ☑ Check network / firewall
- ☑ Verify JNLP URL & secret
- ☑ Check disk space & logs
- ☑ Reconnect agent

2. QUEUE PROBLEMS

- No available executors
- All agents busy
- Wrong labels
- Long-running jobs

HOW TO CHECK

```
$ Check queue in UI
$ jenkins-cli who-ami
$ jenkins-cli list-executors
$ Review long running builds
```

HOW TO FIX

- ☑ Add more executors/agents
- ☑ Optimize long running jobs
- ☑ Use proper labels
- ☑ Enable throttling / limits
- ☑ Cancel blocked builds

3. PLUGIN FAILURES

- Plugin incompatible
- Plugin not loaded
- ClassNotFound errors
- Plugin upgrade issues

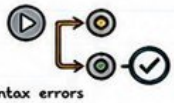
HOW TO CHECK

```
$ Manage Jenkins > Plugins
$ Check system log
$ Check plugin versions
$ Look for stack traces
```

HOW TO FIX

- ☑ Update / downgrade plugin
- ☑ Clear plugin cache
- ☑ Restart Jenkins
- ☑ Check plugin compatibility
- ☑ Reinstall broken plugin

4. PIPELINE DEBUGGING



- Syntax errors
- Script logic issues
- Environment problems
- Step / stage failures

HOW TO CHECK

```
$ Check console output
$ Enable pipeline logging
$ Use 'echo' for debugging
$ Replay / dry-run pipeline
```

HOW TO FIX

- ☑ Fix syntax / script errors
- ☑ Add logging (echo / sh -x)
- ☑ Validate environment vars
- ☑ Use try-catch / post blocks
- ☑ Test stages individually

5. CONTROLLER HEALTH

- High CPU / memory
- Disk full
- Slow response
- Service / JVM issues

HOW TO CHECK

```
$ Check system log
$ Monitor CPU & memory
$ Check disk usage
$ jenkins-cli check-health
```

HOW TO FIX

- ☑ Restart Jenkins service
- ☑ Increase memory (JVM)
- ☑ Clean up old builds/logs
- ☑ Check disk space
- ☑ Optimize system resources

JENKINS TROUBLESHOOTING WORKFLOW

1. **Detect** — Identify the issue from alerts, users or pipeline logs.
2. **Investigate** — Check logs, nodes, queue, plugins, and system metrics.
3. **Analyze** — Find the root cause and understand the impact.
4. **Fix** — Apply the right fix and test the solution.
5. **Validate** — Confirm pipeline success and system stability.

USEFUL JENKINS CLI COMMANDS

```
$ jenkins-cli -s http://<jenkins> who-ami
$ jenkins-cli -s http://<jenkins> list-jobs
$ jenkins-cli -s http://<jenkins> list-nodes
$ jenkins-cli -s http://<jenkins> list-executors
$ jenkins-cli -s http://<jenkins> safe-restart
$ jenkins-cli -s http://<jenkins> get-job <job>
$ jenkins-cli -s http://<jenkins> clear-queue
```

PRO TIP

Most Jenkins issues are visible in logs. Check system log, pipeline log, and agent log first. Logs don't lie. Symptoms do.

COMMON SYMPTOMS, CAUSES & FIXES

SYMPTOM	POSSIBLE CAUSE	CHECK THIS	FIX
Agent offline	Agent crashed / network issue	Check node in UI & logs	Restart agent, check connectivity
Build stuck in queue	No executors / wrong label	Check queue & executors	Add executors, correct label
Plugin failed to load	Incompatible / broken plugin	Check system log	Update / reinstall plugin
Pipeline failed early	Syntax / configuration error	Check console output	Fix script / pipeline syntax
Build environment issue	Missing tools / variables	Check node & env vars	Install tools / set variables
Slow Jenkins	High load / low resources	Check CPU, memory, disk	Increase resources, cleanup
Job not triggered	Webhook / SCM issue	Check webhook & SCM log	Fix SCM / webhook config

PIPELINE LOGGING & DEBUGGING TIPS

```
# Enable detailed logging in pipeline
pipeline {
  options {
    timestamps()
    buildDiscarder(logRotator(daysToKeepStr: '30'))
  }
  stages {
    stage('Debug Example') {
      steps {
        script {
          echo "DEBUG: Starting build"
          sh 'set -x'    // Debug shell steps
          echo "DEBUG: Environment = ${env.BUILD_ENV}"
        }
      }
    }
  }
} Tip: Use 'echo', 'sh -x', and 'post' blocks
    for better visibility.
```

CONTROLLER HEALTH CHECKLIST

- ☑ Jenkins service is running
- ☑ System log has no critical errors
- ☑ CPU usage is normal
- ☑ Memory usage is normal
- ☑ Disk space is sufficient
- ☑ Backup is working
- ☑ Plugins are compatible

AGENT HEALTH CHECKLIST

- ☑ Agent service is running
- ☑ Agent connected to controller
- ☑ Correct labels assigned
- ☑ Sufficient disk space
- ☑ Required tools installed
- ☑ Network connectivity OK
- ☑ Logs are clean

BEST PRACTICES

- ☑ Keep Jenkins & plugins updated
- ☑ Use agents for builds, not controller
- ☑ Monitor system & job health
- ☑ Clean old builds & workspace
- ☑ Backup regularly
- ☑ Use credentials securely
- ☑ Test changes before production

QUICK CHECKLIST

- ☑ Is the agent online?
- ☑ Is the queue full?
- ☑ Are plugins compatible?
- ☑ Are there errors in console log?
- ☑ Is the controller healthy?
- ☑ Is the pipeline script correct?
- ☑ Is SCM / webhook working?

A HEALTHY JENKINS = RELIABLE PIPELINES = HAPPY TEAMS

FOLLOW VERIQTA

YouTube /veriqta · GitHub /veriqta · LinkedIn /veriqta · X @veriqta · Instagram @veriqta · Telegram @veriqta

GitLab CI/CD

GITLAB CI/CD TROUBLESHOOTING

Every failed pipeline is a clue.
Diagnose fast. Fix smart. Ship with confidence.

1. RUNNER FAILURES

- Runner offline or not registered
- Runner not picking jobs
- Job stuck in preparing
- Permission / config issues

HOW TO CHECK

```
$ gitlab-runner status
$ gitlab-runner verify
Check runner in GitLab UI
Review runner logs
```

HOW TO FIX

- ☑ Restart runner service
- ☑ Re-register runner
- ☑ Check runner token
- ☑ Fix permissions
- ☑ Update runner version

2. CACHE ISSUES

- Cache not found
- Cache not restoring
- Cache key mismatch
- Cache upload failures

HOW TO CHECK

```
$ Check cache key in job log
$ Verify paths in cache
$ Review cache policy
$ Check runner disk space
```

HOW TO FIX

- ☑ Use consistent cache keys
- ☑ Correct cache paths
- ☑ Use pull-push policy
- ☑ Increase cache TTL
- ☑ Ensure runner has space

3. VARIABLES ISSUES

- Variable not found
- Protected variable blocked
- Wrong environment scope
- Masked variable problems

HOW TO CHECK

```
$ echo $VAR_NAME
Review variables in UI
Check variable scope
Check protected settings
```

HOW TO FIX

- ☑ Define variable in correct scope
- ☑ Mark variable as protected
- ☑ Check environment settings
- ☑ Use masked variables
- ☑ Restart pipeline

4. PIPELINE DEBUGGING

- Job fails unexpectedly
- Script errors
- Exit code failures
- Logic / condition issues

HOW TO CHECK

```
$ cat .gitlab-ci.yml
Enable debug mode
Check job logs
Re-run failed job
```

HOW TO FIX

- ☑ Add set -x for shell debug
- ☑ Validate .gitlab-ci.yml
- ☑ Check dependencies
- ☑ Add echo for variables
- ☑ Use allow_failure wisely

5. ARTIFACT FAILURES

- Artifact not uploaded
- Artifact expired
- Artifact path incorrect
- Download failures

HOW TO CHECK

```
$ Check job artifacts
Review artifact paths
Check expire_in setting
Check storage quota
```

HOW TO FIX

- ☑ Correct artifact paths
- ☑ Increase expire_in
- ☑ Ensure artifact is generated
- ☑ Check runner permissions
- ☑ Clean old artifacts

GITLAB CI/CD TROUBLESHOOTING WORKFLOW

1. **Detect** — Identify failure from pipeline status or logs.
2. **Investigate** — Check job logs, runner logs, variables, cache, and artifacts.
3. **Analyze** — Find the root cause. Reproduce if possible.
4. **Fix** — Apply the correct fix. Update config or scripts.
5. **Validate** — Re-run pipeline or job. Confirm success.

USEFUL GITLAB CLI COMMANDS

```
$ glab auth login
$ glab pipeline list
$ glab pipeline view <id>
$ glab job list --pipeline <id>
$ glab job view <job-id> --log
$ glab project variable list
$ glab runner list
$ glab runner verify
$ glab cache ls
$ glab pipeline retry <id>
$ glab pipeline cancel <id>
```

PRO TIP

Always check the pipeline log first. 95% of GitLab CI/CD issues have clear clues in the logs. Logs don't lie. They tell the story.

COMMON SYMPTOMS, CAUSES & FIXES

SYMPTOM	POSSIBLE CAUSE	CHECK THIS	FIX
Runner offline	Runner service stopped	Runner status in UI / logs	Restart runner, check token
Job stuck in preparing	Runner not available	Runner queue in UI	Add runners, check tags
Pipeline fails early	Script error / bad command	Job log	Fix script, add debug
Cache not found	Wrong key / expired cache	Cache key in job log	Use correct key, extend TTL
Variable not found	Wrong scope / not defined	Print variable / check UI	Define variable in right scope
Artifact missing	Path incorrect / not created	Job log / artifact path	Fix path, ensure file exists
Pipeline not triggered	Rules / only/except issue	.gitlab-ci.yml rules	Fix rules and conditions
Permission denied	Protected variable / env	Variable settings	Adjust protection settings
Job timeout	Timeout too low	Job log / timeout setting	Increase timeout

ENABLE DEBUG MODE

```
# Add to your job script
set -euo pipefail
set -x
# Print environment
env | sort
# Example
echo "Debug mode enabled"
pwd
ls -la
whoami
```

ARTIFACT BEST PRACTICES

- ☑ Store only necessary artifacts
- ☑ Use clear artifact paths
- ☑ Set reasonable expire_in
- ☑ Use artifacts: reports for reports
- ☑ Clean up old artifacts regularly

CACHE BEST PRACTICES

- ☑ Use meaningful cache keys
- ☑ Do not cache dependencies blindly
- ☑ Use paths specific to the project
- ☑ Use pull-push policy
- ☑ Do not cache large files

RUNNER HEALTH CHECKLIST

- ☑ Runner is online
- ☑ Correct GitLab URL
- ☑ Valid runner token
- ☑ Tags are correct
- ☑ Enough disk space
- ☑ Runner version updated
- ☑ Jobs picked up successfully

PIPELINE DEBUG CHECKLIST

- ☑ Check job logs
- ☑ Validate .gitlab-ci.yml
- ☑ Print variables
- ☑ Add set -x
- ☑ Re-run job
- ☑ Check dependencies
- ☑ Review exit codes

BEST PRACTICES

- ☑ Keep pipelines simple
- ☑ Fail fast and clearly
- ☑ Use stages logically
- ☑ Use templates and includes
- ☑ Monitor runners
- ☑ Automate cleanup
- ☑ Document everything

QUICK CHECKLIST

- ☑ Is runner online?
- ☑ Are variables set?
- ☑ Is cache working?
- ☑ Do artifacts exist?
- ☑ Do logs show the error?
- ☑ Can the job be reproduced?
- ☑ Did you try a retry?

REMEMBER

A reliable pipeline is a product.
Treat it like code.
Test it. Monitor it.
Improve it.

OBSERVE THE PIPELINE. UNDERSTAND THE LOGS. FIX THE CAUSE. DELIVER WITH CONFIDENCE.

GitHub Actions

GITHUB ACTIONS TROUBLESHOOTING

Every failed workflow leaves a clue.
Diagnose faster. Fix smarter. Automate with confidence.

1. WORKFLOW FAILURES

- Job or step failed
- Exit code non-zero
- Script or command error
- Conditional logic issues

HOW TO CHECK

$ Check the failed job
$ Review step logs
$ Check exit code
$ Re-run with debug logs

HOW TO FIX

- [x] Read error message carefully
- [x] Fix command / script
- [x] Validate inputs & env
- [x] Re-run workflow
- [x] Add better error handling

2. SECRETS ISSUES

- Secret not found
- Empty secret value
- Invalid secret name
- Secrets not available in forked PRs

HOW TO CHECK

$ echo ${{ secrets.MY_SECRET }}
$ Verify secret name
$ Check repo / org secrets
$ Check environment secrets

HOW TO FIX

- [x] Add or update secret
- [x] Use correct secret name
- [x] Check environment access
- [x] Avoid secrets in forks
- [x] Use OIDC when possible

3. PERMISSIONS ISSUES

- GITHUB_TOKEN lacks scope
- Permission denied
- Cannot push / access API
- Checkout or artifact blocked

HOW TO CHECK

$ cat $GITHUB_TOKEN_PERMISSIONS
$ Check workflow permissions
$ Review job permissions
$ Check repo settings

HOW TO FIX

- [x] Set correct permissions
- [x] Use least privilege
- [x] Update workflow permissions
- [x] Grant write if pushing
- [x] Use fine-grained PAT if needed

4. RUNNER PROBLEMS

- Runner offline / busy
- Runner not found
- Self-hosted runner issues
- Runner label mismatch

HOW TO CHECK

$ Check runner status
$ Review runner logs
$ Verify labels / groups
$ Check capacity / queue

HOW TO FIX

- [x] Start or register runner
- [x] Check runner labels
- [x] Update runner version
- [x] Increase runner capacity
- [x] Fix network / connectivity

5. DEBUG LOGGING

- Need more details
- Intermittent failures
- Hard to reproduce
- Unknown root cause

HOW TO CHECK

$ Enable step debug
$ Enable runner debug
$ Check environment debug
$ Review full logs

HOW TO FIX

- [x] Enable debug logs
- [x] Add more logging
- [x] Mask sensitive data
- [x] Re-run and analyze
- [x] Document findings

GITHUB ACTIONS TROUBLESHOOTING WORKFLOW

1. **Detect** — Alert or failure is triggered.
2. **Investigate** — Check workflow run, jobs, and step logs.
3. **Analyze** — Find the root cause: script, config, env, permissions, or runner.
4. **Fix** — Apply the right fix and update workflow or settings.
5. **Validate** — Re-run workflow and confirm success.

USEFUL GITHUB CLI COMMANDS

```
$ gh run list --limit 10
$ gh run view <run-id> --log
$ gh run rerun <run-id>
$ gh run cancel <run-id>
$ gh run view --json conclusion,headBranch,event
$ gh workflow list
$ gh workflow view <workflow-file>
$ gh api repos/:owner/:repo/actions/runners
$ gh api repos/:owner/:repo/actions/secrets
$ gh api repos/:owner/:repo/actions/permissions/workflow
```

PRO TIP

Always read the logs from bottom to top. The root cause is usually near the first error, not the last one.

COMMON SYMPTOMS, CAUSES & FIXES

SYMPTOM	POSSIBLE CAUSE	CHECK THIS	FIX
Workflow failed	Script error, bad config	Check failed step logs	Fix script / syntax
Step skipped	Condition / if failure	Check step conditions	Fix condition logic
Permission denied	GITHUB_TOKEN scope	Check job permissions	Update permissions
Cannot push	Read-only token	Check permissions	Grant write permission
Secret not found	Wrong secret name	Check secrets list	Add or correct secret
Runner offline	Runner stopped	Check runner status	Start / register runner
Job stuck in queue	No available runner	Check runner capacity	Add more runners
Artifact missing	Upload failed / path	Check upload step	Fix path / permissions
Action failed	Action error / version	Check action logs	Update or fix action
Timeout	Job took too long	Check timeout setting	Increase timeout

ENABLE DEBUG LOGGING

1. Enable Step Debug

```
- name: Enable debug logging
  run: echo "ACTIONS_STEP_DEBUG=true" >> $GITHUB_ENV
```

2. Enable Runner Debug (self-hosted)

```
# On self-hosted runner machine
sudo ./config.sh --debug
```

3. Enable Verbose Output in Actions

```
- name: Run with debug
  run: |
    set -x
    your-command-here
```

ARTIFACT BEST PRACTICES

- [x] Upload artifacts only when needed
- [x] Use unique names per run
- [x] Set retention-days wisely
- [x] Verify paths before upload
- [x] Check artifact size limits
- [x] Download artifacts for debugging

CACHE BEST PRACTICES

- [x] Use key with OS + deps hash
- [x] Restore-keys for fallback
- [x] Avoid caching large files
- [x] Clear cache when deps change
- [x] Monitor cache hit rate

WORKFLOW CHECKLIST

- [x] YAML syntax valid?
- [x] Triggers correct?
- [x] Jobs have runners?
- [x] Steps ordered correctly?
- [x] Environment variables set?
- [x] All secrets available?
- [x] Permissions configured?
- [x] Timeout values appropriate?

SECRETS CHECKLIST

- [x] Secret name correct?
- [x] Secret value set?
- [x] Environment access OK?
- [x] Not used in forks?
- [x] Masked in logs?
- [x] Rotated regularly?
- [x] Scope is minimal?

RUNNER CHECKLIST

- [x] Runner online?
- [x] Labels match?
- [x] Enough capacity?
- [x] Version up to date?
- [x] Network access OK?
- [x] Disk space enough?
- [x] Runner logs clean?

PERMISSIONS CHECKLIST

- [x] Default GITHUB_TOKEN OK?
- [x] Write access where needed?
- [x] Actions: read/write set?
- [x] Contents: read/write set?
- [x] Packages: access set?
- [x] Id-token: write if using OIDC?
- [x] Least privilege followed?

QUICK CHECKLIST

- [x] Read failure logs
- [x] Identify first error
- [x] Check config & YAML
- [x] Verify secrets
- [x] Check permissions
- [x] Check runner status
- [x] Re-run & validate

AUTOMATE WITH INTELLIGENCE. OBSERVE EVERYTHING. FIX THE CAUSE. DELIVER RELIABLY.

DOCKER CONTAINER FAILURES

Containers are simple to run.
But in production, they fail. Senior engineers know how to find why.

1. EXIT CODES

EXIT 137

- Non-zero exit codes mean failure
- Each code has a meaning
- OOMKilled returns 137
- App errors return custom codes

HOW TO CHECK

```
$ docker inspect <container>
$ echo $?
$ docker ps -a --format
  "{{.Names}}\t{{.Status}}"
```

HOW TO FIX

- ☑ Check container logs
- ☑ Understand the exit code
- ☑ Fix app error / config
- ☑ Add health checks

2. RESTART LOOPS

- Container starts and stops
- Restart policy keeps restarting
- Root cause may be app, config, dependency, or resource

HOW TO CHECK

```
$ docker ps -a
$ docker inspect <container>
$ docker events --filter
  "event=die"
```

HOW TO FIX

- ☑ Check logs and exit code
- ☑ Fix underlying issue
- ☑ Adjust restart policy
- ☑ Add health check / delay

3. CRASH ANALYSIS

- App crashes inside container
- Could be code bug, OOM, or dependency failure
- Investigate deeply

HOW TO CHECK

```
$ docker logs <container>
$ docker inspect <container>
$ dmesg | grep -i kill
$ docker stats <container>
```

HOW TO FIX

- ☑ Reproduce the crash
- ☑ Check resource limits
- ☑ Fix code / dependency
- ☑ Add monitoring & alerts

4. LOGS

LOG

- Logs tell the real story
- Always check logs first
- Stdout/stderr are captured
- Use structured logging

HOW TO CHECK

```
$ docker logs <container>
$ docker logs -f <container>
$ docker logs --tail 200
  <container>
```

HOW TO FIX

- ☑ Read logs from the start
- ☑ Follow in real-time
- ☑ Add proper log levels
- ☑ Ship logs to central system

COMMON EXIT CODES

EXIT CODE	MEANING	COMMON CAUSES	WHAT TO CHECK
0	Success	Normal exit	No action needed
1	General error	App error, script failure	Logs, app code
2	Misuse of shell	Bad command or arguments	Command syntax
126	Cannot execute	Permission issue	File permission
127	Command not found	Missing binary	Image content
137	OOMKilled (SIGKILL)	Out of memory	Memory limits, usage
139	Segmentation fault	App crash	Logs, core dump
143	SIGTERM	Graceful stop	Stop signal handled
255	Unknown error	Unhandled exception	Logs, app code

DOCKER TROUBLESHOOTING WORKFLOW

1. **Detect** — Alert or user reports issue.
2. **Investigate** — Check status, logs, and events.
3. **Analyze** — Find root cause: app, config, resource, or dependency.
4. **Fix** — Apply the correct fix and restart or rebuild.
5. **Validate** — Confirm container is healthy and stable.

USEFUL DOCKER COMMANDS

```
$ docker ps -a
List all containers
$ docker logs <container>
View logs
$ docker logs -f <container>
Follow logs
$ docker inspect <container>
Detailed info
$ docker stats <container>
Resource usage
$ docker top <container>
Processes in container
$ docker events --filter container=<id>
Events for container
$ docker exec -it <container> sh
Access container shell
$ docker system df
Disk usage
$ docker image prune -a
Clean unused images
```

PRO TIP

Always check logs first.
80% of container issues show up in the logs.
Logs don't lie.
They explain.

COMMON ROOT CAUSES & HOW TO FIX

ROOT CAUSE	SYMPTOMS	HOW TO CHECK	HOW TO FIX
Application crash	Container exits with code 1	docker logs <container>	Fix code / config
Out of memory	Exit 137, OOMKilled	docker inspect, dmesg	Increase memory limit
Permission denied	Exit 126 or 13	docker logs, ls -l	Fix permissions
Missing dependency	App fails on start	docker logs	Update image / install deps
Bad configuration	App fails with error	docker logs	Fix config / env vars
Port already in use	Container fails to start	docker ps, netstat	Change port / stop process

CRASH ANALYSIS CHECKLIST

- ☑ Check container status
- ☑ Check exit code
- ☑ Read full logs
- ☑ Inspect container details
- ☑ Check resource usage
- ☑ Check recent events
- ☑ Check host logs (dmesg/journalctl)
- ☑ Reproduce the issue
- ☑ Apply fix and verify

BEST PRACTICES

- ☑ Use health checks
- ☑ Set resource limits
- ☑ Use restart policies wisely
- ☑ Log to stdout/stderr
- ☑ Monitor containers
- ☑ Keep images updated
- ☑ Test before deployment

RESTART POLICIES

- no = Never restart
- on-failure = Restart on failure
- always = Always restart
- unless-stopped = Always restart, even after reboot

Choose the policy based on application behavior.

LOGGING TIPS

- ☑ Use structured logs
- ☑ Add timestamps
- ☑ Use log levels
- ☑ Avoid sensitive data
- ☑ Centralize logs
- ☑ Monitor and alert

QUICK CHECKLIST

- ☑ Is container running?
- ☑ Any restart loops?
- ☑ What is the exit code?
- ☑ What do the logs show?
- ☑ Any resource issues?
- ☑ Any config errros?
- ☑ Any_dependency failures?

A HEALTHY CONTAINER IS OBSERVED, UNDERSTOOD, AND PROTECTED.
CHECK LOGS. CHECK METRICS. FIND ROOT CAUSE. FIX PERMANENTLY.

DOCKER NETWORKING

Containers don't talk by magic.
Networks make it work. Know your paths.

1. BRIDGE NETWORKS

- Default network type
- Containers on same bridge can talk to each other
- Isolated from other bridges
- Good for single-host apps

HOW TO CHECK

```
$ docker network ls
$ docker network inspect bridge
$ docker network inspect <net>
```

HOW TO FIX

- ☑ Connect container to correct network
- ☑ Recreate network if broken
- ☑ Check IP address conflicts
- ☑ Restart containers

2. OVERLAY NETWORKS

- Multi-host networking
- Built for Docker Swarm
- Encrypted traffic between nodes
- Scales across servers

HOW TO CHECK

```
$ docker network ls
$ docker network inspect <ovl>
$ docker node ls
```

HOW TO FIX

- ☑ Ensure nodes are up
- ☑ Verify swarm status
- ☑ Check network encryption
- ☑ Recreate overlay network

3. DNS IN DOCKER

- Containers resolve by name
- Docker embedded DNS at 127.0.0.11
- Works on user-defined networks

HOW TO CHECK

```
$ cat /etc/resolv.conf
$ docker exec <c> nslookup <name>
$ docker exec <c> dig <name>
```

HOW TO FIX

- ☑ Use service name, not IP
- ☑ Connect to same network
- ☑ Check DNS settings
- ☑ Restart DNS if needed

4. PORT MAPPING

- Publish container ports to host
- Format: hostPort:containerPort
- Required for external access

HOW TO CHECK

```
$ docker ps --format "{{.Ports}}"
$ ss -tulnp | grep <port>
$ netstat -tulnp | grep <port>
```

HOW TO FIX

- ☑ Map correct ports
- ☑ Check host firewall
- ☑ Ensure app listening on 0.0.0.0
- ☑ Restart container

5. CONNECTIVITY

- Containers must share the same network
- Check routes, firewall, and port exposure
- MTU and latency matter

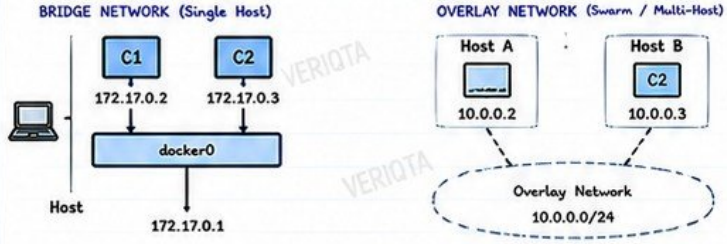
HOW TO CHECK

```
$ docker exec <c1> ping <c2>
$ docker exec <c> ip a
$ docker exec <c> traceroute <c2>
```

HOW TO FIX

- ☑ Attach both to same network
- ☑ Check NetworkPolicies
- ☑ Validate routes
- ☑ Check MTU / firewall rules

DOCKER NETWORK ARCHITECTURE OVERVIEW



COMMON NETWORK DRIVERS

DRIVER	SCOPE	USE CASE	NOTES
bridge	Single Host	Default, standalone apps	Isolated per host
host	Single Host	High performance apps	No isolation
none	Single Host	Advanced custom setups	No networking
overlay	Multi-Host	Docker Swarm services	Encrypted
macvlan	Single Host	Assign MAC to container	L2 networks
ipvlan	Single Host	More IPs from parent	Efficient
custom	Single/Multi	Plugin based	Advanced

NETWORK CONNECTIVITY FLOW

1. Container sends traffic
2. Traffic goes through network
3. Route is selected
4. Port mapping applies (if needed)
5. Reaches host or external destination

USEFUL DOCKER NETWORK COMMANDS

```
$ docker network ls                                  # List all networks
$ docker network inspect <network>                   # Inspect details
$ docker network create <name> --driver bridge       # Create bridge network
$ docker network create -d overlay <name>            # Create overlay network
$ docker network connect <network> <container>       # Connect container
$ docker network disconnect <network> <container>    # Disconnect container
$ docker exec <container> ip a                       # Show container IP
$ docker exec <container> ping <target>              # Test connectivity
$ docker exec <container> nslookuy <name>            # DNS lookup
$ docker exec <container> traceroute <target>        # Trace route
```

COMMON NETWORKING ISSUES

ISSUE	POSSIBLE CAUSE	CHECK THIS	FIX IT
Container can't reach another	Different networks	docker inspect <c> | grep Networks	Connect to same network
DNS resolution fails	Not on user-defined network	cat /etc/resolv.conf	Use user-defined network
Port not accessible	Port not published / firewall	docker ps | grep <port>	Map port / open firewall
Connection refused	App not listening / wrong port	ss -tulnp inside container	Fix app / port mapping
Intermittent connection	MTU / network congestion	ping -M do -s 1472 <target>	Adjust MTU / routes
Overlay containers can't talk	Swarm / node issue	docker node ls	Fix node / recreate network
High latency	Network congestion	ping / traceroute	Optimize routes / MTU

DNS INSIDE DOCKER

- Docker uses embedded DNS at 127.0.0.11
- Resolve by container name or service name
- Works only on same network or swarm
- Example:

```
$ docker exec web nslookup db
Server:    127.0.0.11
Address:   127.0.0.11#53
Name:      db
Address:   10.0.0.5
```

PRO TIP

- Always prefer user-defined networks. They give you better DNS, isolation, and control.
- Avoid the default bridge unless you must.

BRIDGE CHECKLIST

- ☑ Containers on same bridge
- ☑ No IP conflicts
- ☑ DNS working
- ☑ Correct port mapping
- ☑ Firewall rules verified

OVERLAY CHECKLIST

- ☑ Swarm is active
- ☑ All nodes healthy
- ☑ Overlay network exists
- ☑ Encryption enabled
- ☑ Services on same overlay

SECURITY CHECKLIST

- ☑ Don't expose unnecessary ports
- ☑ Use host firewall
- ☑ Limit inter-container traffic
- ☑ Use NetworkPolicies (K8s)
- ☑ Regular network audits

BEST PRACTICES

- ☑ Use user-defined networks
- ☑ Use service names, not IPs
- ☑ Document port mappings
- ☑ Monitor network metrics
- ☑ Test connectivity after deploy

QUICK CHECKLIST

- ☑ Can containers ping each other?
- ☑ DNS resolving names?
- ☑ Ports accessible from outside?
- ☑ No errors in logs?
- ☑ Latency acceptable?

A STRONG NETWORK = RELIABLE CONTAINERS. DESIGN IT RIGHT. TEST IT. MONITOR IT.

DOCKER STORAGE

Data is persistent by design.
Understand how Docker stores, mounts and manages data.

1. VOLUMES

- Managed by Docker
- Stored in /var/lib/docker/volumes
- Portable and easy to backup
- Best for databases & apps

HOW TO CHECK

```
$ docker volume ls
$ docker volume inspect <vol>
$ docker system df --volumes
```

HOW TO FIX

- ☑ Create named volumes
- ☑ Use volumes for critical data
- ☑ Backup volume data
- ☑ Set proper permissions

2. BIND MOUNTS

- Map host paths into container
- Full control from host
- Good for configs & code
- Host path dependency

HOW TO CHECK

```
$ docker inspect <container>
$ mount | grep <path>
$ ls -ld <host-path>
```

HOW TO FIX

- ☑ Validate host path exists
- ☑ Set correct permissions
- ☑ Use absolute paths
- ☑ Avoid sensitive host dirs

3. OVERLAYFS

- Default storage driver
- Union filesystem
- Copy-on-write (CoW)
- Efficient & lightweight

HOW TO CHECK

```
$ docker info | grep Storage
$ docker info | grep Driver
$ ls /var/lib/docker/overlay2
```

HOW TO FIX

- ☑ Monitor disk usage
- ☑ Prune unused layers
- ☑ Keep Docker updated
- ☑ Ensure enough disk space

4. DISK USAGE

- Images, containers, volumes
- Build cache, layers
- Logs and temp files
- Can fill disk quickly

HOW TO CHECK

```
$ docker system df
$ docker system df -v
$ du -sh /var/lib/docker/*
```

HOW TO FIX

- ☑ Remove unused data
- ☑ Limit log sizes
- ☑ Clean build cache
- ☑ Monitor regularly

5. CLEANUP

- Remove unused images
- Remove stopped containers
- Prune networks & volumes
- Keep system clean

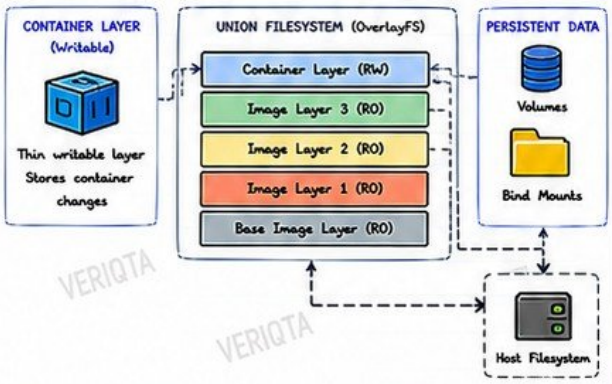
HOW TO CHECK

```
$ docker system df
$ docker ps -a
$ docker volume ls
```

HOW TO FIX

- ☑ docker system prune -a
- ☑ docker volume prune
- ☑ docker image prune -a
- ☑ Automate cleanup tasks

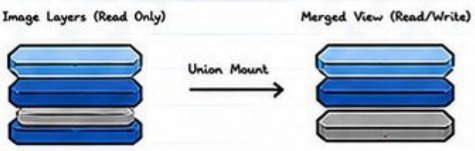
DOCKER STORAGE ARCHITECTURE



STORAGE TYPES COMPARED

TYPE	MANAGED BY	LOCATION	PORTABLE	PERFORMANCE	USE CASE
Volumes	Docker	/var/lib/docker/volumes	High	Good	Databases, apps
Bind Mounts	Host	Host filesystem path	Low	Native	Configs, code
tmpfs Mounts	RAM	Memory	N/A	Very Fast	Temp data, secrets
OverlayFS	Docker	/var/lib/docker/overlay2	N/A	Good	Image & container layers

STORAGE DRIVER OVERLAYFS EXPLAINED



PRO TIP

- Use volumes for persistent data.
- Use bind mounts for configs & code.
- Keep disk clean.
- Monitor disk usage.
- Automate cleanup.

USEFUL DOCKER STORAGE COMMANDS

```
$ docker volume ls                      # List all volumes
$ docker volume inspect <name>          # Inspect volume details
$ docker run -v myvol:/data image       # Use named volume
$ docker run -v /host/path:/data image  # Use bind mount
$ docker inspect <container>            # View mounts and storage
$ docker system df                      # Docker disk usage summary
$ docker system df -v                   # Verbose disk usage
$ docker image ls                       # List images
$ docker container ls -a                # List containers
$ docker system prune -a                # Remove unused everything
$ docker volume prune                   # Remove unused volumes
$ docker builder prune                  # Clean build cache
```

EXAMPLE: INSPECT VOLUME

```
$ docker volume inspect myvol
```

```
[{
  "CreatedAt": "2024-06-01T12:00:00Z",
  "Driver": "local",
  "Mountpoint": "/var/lib/docker/volumes/myvol/_data",
  "Labels": null,
  "Scope": "local"
}]
```

EXAMPLE: CONTAINER MOUNTS

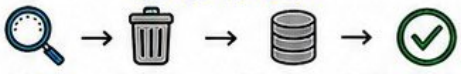
```
$ docker inspect mycontainer
```

```
"Mounts": [
  {
    "Type": "volume",
    "Name": "myvol",
    "Source": "/var/lib/docker/volumes/myvol/_data",
    "Destination": "/data",
    "Mode": "rw",
    "RW": true
  }
]
```

COMMON STORAGE ISSUES

ISSUE	CAUSE	CHECK THIS	FIX
No space left on device	Disk full	docker system df	Prune unused data
Container data not persisting	Using container layer	Inspect mounts	Use volumes/bind mounts
Permission denied	Incorrect permissions	ls -l host path	Fix ownership/permissions
Slow I/O	Wrong storage driver	docker info	Use faster storage
Bind mount not working	Path doesn't exist	ls -ld host path	Create path & restart
Deleted data in volume	Volume pruned	docker volume ls	Restore from backup

CLEANUP WORKFLOW



⚠ Always backup important data before performing cleanup.

CLEANUP COMMANDS

```
$ docker system prune -a --volumes  # Full cleanup
$ docker image prune -a
$ docker container prune
$ docker volume prune
$ docker network prune
$ docker builder prune
$ docker system df                  # Check again
```

VOLUMES CHECKLIST

- ☑ Use named volumes
- ☑ Backup important data
- ☑ Check volume health
- ☑ Set proper permissions

BIND MOUNTS CHECKLIST

- ☑ Use absolute paths
- ☑ Validate path exists
- ☑ Secure sensitive files
- ☑ Avoid root mount

STORAGE HEALTH CHECKLIST

- ☑ Monitor disk usage
- ☑ Check inodes
- ☑ Review logs
- ☑ Plan capacity

BEST PRACTICES

- ☑ Use volumes for data
- ☑ Bind mounts for configs
- ☑ Keep Docker updated
- ☑ Automate cleanup

QUICK CHECKLIST

- ☑ Disk space OK?
- ☑ Volumes healthy?
- ☑ Unused data removed?
- ☑ Backups in place?

REMEMBER

Clean storage means reliable containers.
Plan. Monitor. Maintain.

GOOD STORAGE PRACTICES = RELIABLE CONTAINERS = HAPPY USERS.

DOCKER IMAGE PROBLEMS

Images are built once, run everywhere.
But when images break, nothing works.

1. BUILD FAILURES

- Syntax errors in Dockerfile
- Missing files in build context
- Package install failures
- Network issues during build

HOW TO CHECK

```
$ docker build -t myimg .
$ docker build --no-cache .
$ docker build --progress=plain .
$ docker build -f Dockerfile .
```

HOW TO FIX

- ☑ Check Dockerfile syntax
- ☑ Verify base image availability
- ☑ Check file paths & COPY cmds
- ☑ Fix package or network issues
- ☑ Use --no-cache and rebuild

2. LAYER CACHING

- Stale cache causes old layers
- COPY order breaks caching
- Unnecessary cache invalidation
- Large layers increase build time

HOW TO CHECK

```
$ docker build --progress=plain .
$ docker history myimg
$ docker images
$ docker build --no-cache .
```

HOW TO FIX

- ☑ Order Dockerfile from stable to volatile (deps first, code last)
- ☑ Use .dockerignore properly
- ☑ Leverage build cache correctly
- ☑ Clean old images & prune

3. REGISTRY AUTHENTICATION

- Login expired or missing
- Wrong credentials or token
- Insufficient permissions
- 2FA / Token issues

HOW TO CHECK

```
$ docker login
$ docker logout
$ docker whoami
$ docker info | grep -i registry
```

HOW TO FIX

- ☑ Login with correct credentials
- ☑ Use access token (not password)
- ☑ Check account permissions
- ☑ Refresh token / re-authenticate
- ☑ Configure credential helper

4. IMAGE PULL ERRORS

- Image not found
- Network / DNS failures
- Rate limit exceeded
- Registry unavailable

HOW TO CHECK

```
$ docker pull nginx:latest
$ docker pull --disable-content-trust nginx
$ docker images
$ curl -I https://registry-1.docker.io/v2/
```

HOW TO FIX

- ☑ Verify image name & tag
- ☑ Check network & DNS
- ☑ Login to avoid rate limits
- ☑ Try alternate registry / mirror
- ☑ Check registry service status

DOCKER IMAGE TROUBLESHOOTING WORKFLOW

1. **Detect** — Notice build or pull failure.
2. **Investigate** — Check logs, error message, and context.
3. **Analyze** — Identify root cause: config, network, auth, or cache.
4. **Fix** — Apply the correct fix and rebuild or repull.
5. **Validate** — Confirm image build or pull successful.

USEFUL DOCKER IMAGE COMMANDS

```
$ docker build -t myimg .
$ docker build --no-cache -t myimg .
$ docker build --progress=plain .
$ docker history myimg
$ docker images
$ docker rmi <image_id|name>
$ docker rmi $(docker images -q)
$ docker login
$ docker logout
$ docker whoami
$ docker pull <image>:<tag>
$ docker push <image>:<tag>
$ docker inspect <image>
$ docker image prune -a
```

PRO TIP

Build small, cache smart, and pull purposefully. A clean image process saves time, storage, and production headaches.

COMMON BUILD FAILURE CAUSES

CAUSE	SYMPTOM	CHECK THIS	FIX
Syntax error	Build fails immediately	Dockerfile syntax	Fix Dockerfile instructions
Missing file	COPY failed: no such file	.dockerignore / path	Check paths & .dockerignore
Package failure	apt/yum/pip install fails	Network / repo	Check network / repo / proxy
Network issue	Timeout / download error	Ping / DNS / firewall	Fix network / DNS / proxy
Permission issue	Permission denied	File ownership	Fix perms / use USER correctly
Base image issue	pull access denied	Registry / auth	Login / check permissions
Disk full	No space left on device	df -h	Free disk space
Timeout	Build stops / hangs	Logs / network	Increase timeout / check logs

EXAMPLE: BUILD FAILURE LOG

```
Step 5/12 : RUN apt-get update && apt-get install -y curl
 ---> Running in 3a2b1c9d8e7f
Err:1 http://archive.ubuntu.com/ubuntu focal InRelease
Temporary failure resolving 'archive.ubuntu.com'
E: Failed to fetch http://archive.ubuntu.com/ubuntu/dists/
E: Some index files failed to download.

ERROR: The command "/bin/sh -c apt-get update &&
apt-get install -y curl" returned a non-zero code: 100
```

EXAMPLE: PULL ERROR

```
$ docker pull myapp:latest
Error response from daemon: manifest
for myapp:latest not found: manifest
unknown: manifest unknown

$ docker pull myapp:latest
Error response from daemon: pull access
denied for myapp, repository does not
exist or may require 'docker login'
```

UNDERSTANDING IMAGE LAYERS & CACHE

Each instruction in Dockerfile creates a layer. Changes in earlier layers invalidate subsequent layers.

GOOD PRACTICE:

- ☑ Put stable instructions first
- ☑ Copy dependency files early
- ☑ Copy source code last
- ☑ Use .dockerignore

EFFICIENT DOCKERFILE ORDER

```
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --production
COPY . .
CMD ["node", "server.js"]
```

Stable Layers

Volatile Layer

REGISTRY AUTHENTICATION METHODS

METHOD	USE CASE	HOW TO USE
Username/Password	Personal access	docker login
Access Token	CI/CD, automation	docker login -u <user>
Credential Helper	Secure storage	docker-credential-helpers
Registry Token	Service accounts	Pass token via --password-stdin

BUILD CHECKLIST

- ☑ Dockerfile syntax correct?
- ☑ All files in context?
- ☑ Base image accessible?
- ☑ Dependencies install ok?
- ☑ No unnecessary cache busting?

CACHE CHECKLIST

- ☑ .dockerignore optimized?
- ☑ Order of instructions right?
- ☑ Large layers minimized?
- ☑ Using cache effectively?
- ☑ Old images cleaned?

PULL CHECKLIST

- ☑ Image name & tag correct?
- ☑ Logged in to registry?
- ☑ Network & DNS working?
- ☑ Rate limits considered?
- ☑ Registry service up?

BEST PRACTICES

- ☑ Keep images small
- ☑ Use multi-stage builds
- ☑ Scan images for vulnerabilities
- ☑ Tag images explicitly
- ☑ Push to private registry

QUICK CHECKLIST

- ☑ Build fails?
- ☑ Try --no-cache
- ☑ Check logs first
- ☑ Verify network/auth
- ☑ Fix and rebuild

CLEAN IMAGES. FAST BUILDS. RELIABLE PULLS. STRONGER CONTAINERS.

CONTAINER PERFORMANCE

Performance issues don't always look like errors.
Know how to measure, understand, and fix them.

1. RESOURCE LIMITS

- Limits prevent noisy neighbor issues
- Set CPU, memory, and I/O limits
- Ensure fair resource allocation

HOW TO CHECK

```
$ docker inspect <container>
$ docker stats <container>
$ cat /sys/fs/cgroup/*/*
```

HOW TO FIX

- [x] Set --cpus and --memory
- [x] Configure --memory-reservation
- [x] Set --memory-swap
- [x] Use limits in compose/K8s

2. CPU THROTTLING

- CPU quota limits can throttle
- Causes high latency, slow apps
- Check throttling and usage

HOW TO CHECK

```
$ docker stats --no-stream
$ cat /sys/fs/cgroup/cpu.stat
$ cat /sys/fs/cgroup/cpu.max
```

HOW TO FIX

- [x] Increase --cpus or cpu quota
- [x] Optimize CPU-intensive code
- [x] Use multi-threading wisely
- [x] Distribute load across replicas

3. MEMORY PRESSURE

- High memory usage slows system
- Leads to swapping and OOMKills
- Monitor and right-size memory

HOW TO CHECK

```
$ docker stats --no-stream
$ cat /sys/fs/cgroup/memory.max
$ free -h   (on host)
$ vmstat  1 5
```

HOW TO FIX

- [x] Increase memory limit
- [x] Optimize memory usage
- [x] Enable memory swap (carefully)
- [x] Use memory-reservation

4. OOMKILLED

- Out Of Memory kills container
- Causes sudden container exits
- Check dmesg and events

HOW TO CHECK

```
$ docker inspect <container>
$ dmesg | grep -i killed
$ kubectl describe pod <pod>
```

HOW TO FIX

- [x] Increase memory limit
- [x] Fix memory leaks
- [x] Reduce cache/buffer sizes
- [x] Optimize application memory

5. I/O BOTTLENECKS

- Disk I/O can limit performance
- High latency = slow apps
- Check disk and filesystem

HOW TO CHECK

```
$ iostat -xz 1 5
$ df -hT
$ docker stats --no-stream
```

HOW TO FIX

- [x] Use faster storage (SSD/NVMe)
- [x] Optimize logging & writes
- [x] Use tmpfs for temp data
- [x] Reduce sync writes (tune app)

REAL-TIME MONITORING COMMANDS

```
$ docker stats                          # Live container metrics
$ docker stats --no-stream              # One-time snapshot
$ docker top <container>                # Running processes
$ docker inspect <container>            # Detailed config & limits
$ cat /sys/fs/cgroup/cpu.stat           # CPU throttling stats
$ cat /sys/fs/cgroup/memory.stat        # Memory usage details
$ iostat -xz 1 5                        # Disk I/O stats
$ dmesg | grep -i killed                # OOM kill events
$ pidstat -u 1 5                        # CPU usage per process
$ free -h                               # Host memory usage
```

DOCKER STATS METRICS EXPLAINED

METRIC	WHAT IT MEANS	PROBLEM INDICATOR	ACTION
CPU %	CPU usage of container	Sustained > 80-90%	Increase CPU or optimize app
MEM USAGE / LIMIT	Memory used vs limit	> 85% of limit	Increase memory or optimize
MEM %	Memory usage percentage	> 85% for long time	Tune memory usage
NET I/O	Network in/out	Very high or errors	Check network & app
BLOCK I/O	Disk read/write	High latency or util%	Use faster disk / optimize I/O
PIDS	Number of processes	Near limit	Increase pids-limit

CPU THROTTLING: WHAT HAPPENS?

Container wants more CPU → Hits CPU Limit (Quota) → Throttled (Slower response)

CHECK THROTTLING

```
$ cat /sys/fs/cgroup/cpu.stat
```

```
nr_periods      12345
nr_throttled    678
throttled_time  234567890
```

High throttled_time means CPU throttling

OOMKILLED: WHAT HAPPENS?

Memory Usage Increases → Exceeds Limit → OOM Killer Terminates

CHECK OOM KILLS

```
$ dmesg | grep -i killed
```

```
[12345.678901] Out of memory: Kill process 1234 (node)
score 987 or sacrifice child
[12345.678905] Killed process 1234 (node) total-vm:2048000kB,
anon-rss:1024000kB, file-rss:102400kB
```

EXAMPLE: SET RESOURCE LIMITS

```
# Run container with limits
$ docker run -d --name webapp \
    --cpus=1.5 \
    --memory=512m \
    --memory-reservation=256m \
    --memory-swap=512m \
    --pids-limit=200 \
    myapp:latest

# In docker-compose.yml
services:
  webapp:
    image: myapp:latest
    deploy:
      resources:
        limits:
          cpus: '1.5'
          memory: 512M
        reservations:
          memory: 256M
```

PRO TIP

- Set limits in production.
- Monitor continuously.
- Performance issues are warnings before failures.
- Measure, don't guess.

COMMON CONTAINER PERFORMANCE ISSUES

ISSUE	SYMPTOMS	CHECK THIS	FIX
CPU Throttling	High latency, slow requests	cpu.stat throttled_time	Increase CPU limit or optimize
High Memory Usage	Slow app, swapping	docker stats, free -h	Increase memory, optimize app
OOMKilled	Container exits suddenly	dmesg, inspect, events	Increase memory, fix leaks
I/O Bottleneck	High response time	iostat, docker stats	Use faster disk, optimize I/O
Too Many Processes	Fork failures, errors	docker top, pids limit	Increase pids-limit, fix app
Disk Full	Writes fail, app errors	df -h, docker system df	Cleanup, increase disk
Network Saturation	High latency, drops	net I/O, iftop/nload	Optimize network, scale out

PERFORMANCE TROUBLESHOOTING WORKFLOW

1. Detect — Alert or user complaint
2. Measure — Collect metrics (CPU, Mem, I/O)
3. Analyze — Find bottleneck & root cause
4. Fix — Apply tuning or scaling
5. Validate — Confirm metrics improved

⚠ Always change one thing at a time and measure the impact.

CPU CHECKLIST

- [x] Check CPU usage
- [x] Check throttling
- [x] Review CPU limits
- [x] Optimize hot code

MEMORY CHECKLIST

- [x] Check memory usage
- [x] Check memory limit
- [x] Check for leaks
- [x] Tune app / cache

DISK CHECKLIST

- [x] Check disk usage
- [x] Check disk I/O
- [x] Check log growth
- [x] Use faster storage

NETWORK CHECKLIST

- [x] Check network I/O
- [x] Check latency
- [x] Check packet drops
- [x] Optimize bandwidth

BEST PRACTICES

- [x] Right-size resources
- [x] Monitor everything
- [x] Set alerts early
- [x] Test under load

QUICK CHECKLIST

- [x] docker stats
- [x] df -h
- [x] iostat -xz 1 5
- [x] dmesg | grep killed

WELL-TUNED CONTAINERS DELIVER FAST APPS, HAPPY USERS, AND LOWER COSTS.

kubernetes

KUBERNETES TROUBLESHOOTING WORKFLOW

Good engineers fix symptoms.
Great engineers follow a workflow.

1. CLUSTER-FIRST THINKING

- Always assume the problem could be anywhere in the stack
- Validate the cluster is healthy
- Check control plane, nodes, network and core add-ons first
- Don't jump straight to the app

HOW TO CHECK

```
$ kubectl cluster-info
$ kubectl get nodes -o wide
$ kubectl get --raw='/healthz'
$ kubectl get pods -A
```

HOW TO FIX

- ☑ Restore control plane health
- ☑ Fix node NotReady state
- ☑ Restart core add-ons
- ☑ Check network and DNS

2. NAMESPACE VALIDATION

- Problems often start with the wrong namespace or missing objects
- Validate the namespace exists
- Validate quotas and limits
- Validate all required objects

HOW TO CHECK

```
$ kubectl get ns
$ kubectl config view --minify
$ kubectl get resourcequota -n <ns>
$ kubectl get limitrange -n <ns>
```

HOW TO FIX

- ☑ Use the correct namespace
- ☑ Create missing namespace
- ☑ Adjust resource quotas
- ☑ Fix limit ranges

3. OBJECT RELATIONSHIPS

- Kubernetes is built on relationships
- Pods depend on Deployments
- Deployments depend on ReplicaSets
- Services depend on Endpoints

HOW TO CHECK

```
$ kubectl get deploy -n <ns>
$ kubectl get rs -n <ns>
$ kubectl get pods -n <ns> -o wide
$ kubectl get svc,ep -n <ns>
```

HOW TO FIX

- ☑ Fix broken Deployments
- ☑ Ensure Pods match selectors
- ☑ Recreate broken ReplicaSets
- ☑ Fix Service/Endpoint issues

4. EVENT ANALYSIS

- Events tell the real story
- Kubernetes won't always fail fast
- Read events before deep debugging
- Look for patterns and timestamps

HOW TO CHECK

```
$ kubectl get events -n <ns>
  --sort-by=.lastTimestamp
$ kubectl describe <resource> -n <ns>
$ kubectl get events -A
  --sort-by=.metadata.creationTimestamp
```

HOW TO FIX

- ☑ Resolve root cause from event
- ☑ Fix dependencies
- ☑ Correct configuration
- ☑ Recreate resources if needed

KUBERNETES TROUBLESHOOTING WORKFLOW (THE 7 STEP PATH)

1	2	3	4	5	6	7
Validate Cluster Health	Validate Namespace & Quotas	Check Object Relationships	Analyze Events Deeply	Identify Root Cause	Fix the Root Cause	Validate & Monitor Recovery

☆ Follow the path. Don't guess. Validate each layer before moving forward.

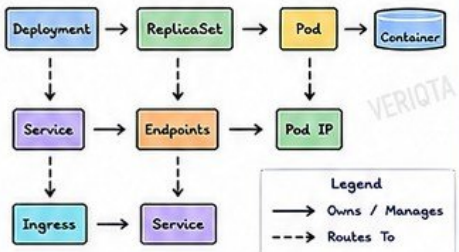
ESSENTIAL TROUBLESHOOTING COMMANDS

Cluster Health	kubectl cluster-info, kubectl get nodes -o wide
Namespace	kubectl get ns, kubectl get all -n <ns>
Pods	kubectl get pods -n <ns> -o wide
Describe	kubectl describe pod <pod> -n <ns>
Logs	kubectl logs <pod> -n <ns> -c <container> --tail=200
Events	kubectl get events -n <ns> --sort-by=.lastTimestamp
Resources	kubectl top pods -n <ns>, kubectl top nodes
YAML	kubectl get <resource> -n <ns> -o yaml
Exec	kubectl exec -it <pod> -n <ns> -- /bin/sh
All Resources	kubectl get all -n <ns>

CLUSTER HEALTH CHECKLIST

- ☑ Control plane is responding
- ☑ All nodes are Ready
- ☑ CoreDNS is Running
- ☑ kube-proxy is Running
- ☑ CNI plugin is healthy
- ☑ No disk pressure on nodes
- ☑ No memory pressure on nodes

KUBERNETES OBJECT RELATIONSHIP MAP



EVENT ANALYSIS EXAMPLE

```
$ kubectl get events -n app --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m	Warning	FailedScheduling	pod/web-7d8f9	0/3 nodes available: 2 Insufficient CPU.
1m	Normal	Scheduled	pod/web-7d8f9	Successfully assigned to node-ip-10-0-1-12
45s	Warning	FailedPullBackOff	pod/web-7d8f9	Failed to pull image "myapp:latest"
30s	Warning	BackOff	pod/web-7d8f9	Back-off pulling image "myapp:latest"

Events show what Kubernetes is trying to tell you.
Always read events before assuming.

COMMON ISSUES AND WHAT TO CHECK FIRST

Pod Pending
- Check events
- Check node resources
- Check taints/tolerations
- Check resource limits
- Check PVC binding

CrashLoopBackOff
- Check logs
- Check probes
- Check environment variables
- Check configmaps/secrets
- Check application code

ImagePullBackOff
- Check image name
- Check registry access
- Check imagePullSecrets
- Check network connectivity
- Check registry authentication

Service Not Working
- Check endpoints
- Check selector labels
- Check pod readiness
- Check network policies
- Check DNS

Ingress 404/502
- Check ingress rules
- Check backend service
- Check endpoints
- Check TLS certificates
- Check ingress controller logs

PRO TIP

- Observe before you act.
- Collect facts, not guesses.
- One change at a time.
- Validate the fix.
- Document what you learn.
- Think in layers, not silos.

YAML VALIDATION
- ☑ Use kubectl explain
- ☑ Validate apiVersion
- ☑ Validate kind
- ☑ Validate indentation
- ☑ Dry-run before apply

LOG ANALYSIS
- ☑ Check application logs
- ☑ Check sidecar logs
- ☑ Check init container logs
- ☑ Increase log verbosity
- ☑ Correlate with events

NETWORK CHECK
- ☑ Check DNS resolution
- ☑ Test service reachability
- ☑ Check network policies
- ☑ Check ingress controller
- ☑ Test from inside pod

RESOURCE CHECK
- ☑ Check CPU usage
- ☑ Check memory usage
- ☑ Check disk usage
- ☑ Check limits/requests
- ☑ Use kubectl top

BEST PRACTICES
- ☑ Cluster-first thinking
- ☑ Follow the 7-step path
- ☑ Read events early
- ☑ Validate relationships
- ☑ Monitor after fixing

QUICK CHECKLIST
- ☑ Cluster healthy?
- ☑ Correct namespace?
- ☑ Objects healthy?
- ☑ Events checked?
- ☑ Root cause found?
- ☑ Fix validated?

TROUBLESHOOTING IS A SKILL. WORKFLOW IS YOUR SUPERPOWER.
OBSERVE. ANALYZE. UNDERSTAND. FIX. VALIDATE. IMPROVE.

kubernetes

KUBERNETES POD FAILURES

Different status. Different cause.
Same goal: Restore service and learn.

1. PENDING

Pod is accepted by the cluster but one or more conditions are not met.

HOW TO CHECK

```
$ kubectl get pods -A
$ kubectl describe pod <pod>
$ kubectl get events --sort-by=.lastTimestamp
```

HOW TO FIX

- ☑ Check scheduling events
- ☑ Verify node resources
- ☑ Check node taints
- ☑ Verify node selectors
- ☑ Check PVC / storage
- ☑ Check imagePullSecrets

2. CRASHLOOPBACKOFF

Container starts, crashes, and kubernetes restarts it repeatedly.
Back-off increases over time.

HOW TO CHECK

```
$ kubectl logs <pod> -c <c>
$ kubectl describe pod <pod>
$ kubectl get pods -w
```

HOW TO FIX

- ☑ Check application logs
- ☑ Fix app error / config
- ☑ Verify environment variables
- ☑ Check liveness/readiness probes
- ☑ Ensure dependencies are up
- ☑ Redeploy after fix

3. IMAGEPULLBACKOFF

Kubelet cannot pull the image from the registry.
Often auth, image name, or network related.

HOW TO CHECK

```
$ kubectl describe pod <pod>
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl get pods -A
```

HOW TO FIX

- ☑ Verify image name & tag
- ☑ Check registry credentials
- ☑ Check imagePullSecrets
- ☑ Test registry connectivity
- ☑ Pull image manually on node
- ☑ Correct image path / repo

4. COMPLETED

Pod ran to completion and container exited successfully.
Not an error.

HOW TO CHECK

```
$ kubectl get pods
$ kubectl describe pod <pod>
$ kubectl logs <pod>
```

HOW TO FIX

- ☑ Verify if Job/CronJob
- ☑ Check exit code
- ☑ Review logs for success
- ☑ If unexpected, check restartPolicy
- ☑ Rerun job if needed

5. EVICTED

Pod was evicted by the node due to resource pressure, storage pressure, or kubelet eviction rules.

HOW TO CHECK

```
$ kubectl describe pod <pod>
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl top nodes
```

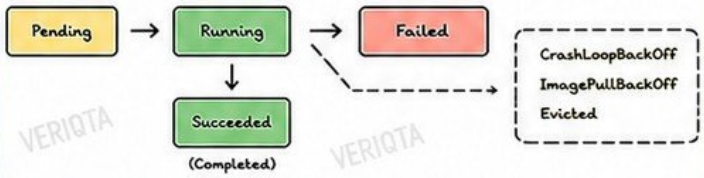
HOW TO FIX

- ☑ Check node resource usage
- ☑ Increase resource limits/requests
- ☑ Optimize application usage
- ☑ Add more nodes / capacity
- ☑ Configure eviction thresholds

QUICK STATUS REFERENCE

STATUS	MEANING	COMMON CAUSES	IS IT BAD?
Pending	Waiting to be scheduled	No resources, taints, PVC, limits	Yes (not running)
CrashLoopBackOff	Container crashes repeatedly	App error, bad config, missing deps	Yes
ImagePullBackOff	Cannot pull container image	Auth, wrong image, network, repo	Yes
Completed	Pod finished successfully	Job done, command exited 0	No
Evicted	Removed by kubelet	Resource pressure, disk pressure	Yes

POD LIFECYCLE (SIMPLIFIED)



TROUBLESHOOTING WORKFLOW (FOLLOW THIS ORDER)

1. **IDENTIFY** — What is the Pod status?
   `$ kubectl get pods`
2. **DESCRIBE** — Get the full story.
   `$ kubectl describe pod <pod>`
3. **CHECK EVENTS** — Events show the real reason.
   `$ kubectl get events --sort-by=.lastTimestamp`
4. **GET LOGS** — If container started, check logs.
   `$ kubectl logs <pod> -c <container>`
5. **ANALYZE** — Understand the root cause. Look for errors, OOM, auth, etc.
6. **FIX IT** — Fix configuration, resources, code, or infrastructure.
7. **VALIDATE** — Confirm the Pod is running and ready.
   `$ kubectl get pods`

PRO TIP

- Events first, logs second.
- Most issues tell you the root cause.
- Check node resources before blaming the app.
- Understand the pattern, then automate the fix.

COMMON COMMANDS CHEAT SHEET

```
$ kubectl get pods -A                                  # List all pods in all namespaces
$ kubectl describe pod <pod> -n <ns>                   # Detailed pod information
$ kubectl logs <pod> -c <container>                    # Logs from container
$ kubectl logs <pod> -c <container> --previous         # Logs from previous container instance
$ kubectl get events -n <ns> --sort-by=.lastTimestamp  # Latest events first
$ kubectl top nodes                                    # Node resource usage
$ kubectl top pod <pod> -n <ns>                        # Pod resource usage
$ kubectl exec -it <pod> -n <ns> -- /bin/sh            # Exec into running container
$ kubectl get pods -o wide                             # See node, IP, and more
$ kubectl get pods -w                                  # Watch pod status in real time
```

EVENT REASON QUICK GUIDE

REASON	WHAT IT MEANS
FailedScheduling	No resources or constraints
Pulling / Pulled	Image is being pulled / pulled
Failed	Failed to start or create
BackOff	Back-off restarting container
Killing	Container is being killed
Evicted	Pod evicted by kubelet
Unhealthy	Liveness/Readiness probe failed

NODE PRESSURE SIGNS

- High CPU usage
- High Memory usage
- Disk Pressure
- PID Pressure
- Network Pressure

Can cause pod eviction, slow apps, or failed scheduling.

PENDING CHECKLIST

- ☑ Check events
- ☑ Node capacity
- ☑ Taints / tolerations
- ☑ Node selectors
- ☑ PVC binding
- ☑ Limit ranges

CRASHLOOPBACKOFF CHECKLIST

- ☑ Check logs
- ☑ Check probes
- ☑ Check env variables
- ☑ Check config / secrets
- ☑ Check dependencies
- ☑ Fix & redeploy

IMAGEPULLBACKOFF CHECKLIST

- ☑ Verify image name/tag
- ☑ Check registry access
- ☑ Check imagePullSecrets
- ☑ Check network / firewall
- ☑ Pull image manually
- ☑ Fix & redeploy

EVICITED CHECKLIST

- ☑ Check node pressure
- ☑ CPU / Memory usage
- ☑ Disk usage
- ☑ Eviction thresholds
- ☑ Right-size resources
- ☑ Add capacity if needed

COMPLETED CHECKLIST

- ☑ Verify Job status
- ☑ Check exit code
- ☑ Review logs
- ☑ If unexpected,
- ☑ check restartPolicy
- ☑ Rerun if required

REMEMBER

Fix the root cause, not the symptom. Document what you learn. Prevent it from happening again.

OBSERVE. UNDERSTAND. FIX. AUTOMATE. IMPROVE. THAT IS KUBERNETES ENGINEERING.

kubernetes

DEPLOYMENT FAILURES

deployment

Deployments simplify updates.
But failures happen. Senior engineers know how to fix them fast.

1. REPLICASETS

- Wrong number of ReplicaSets
- Old ReplicaSets not cleaned
- Selector mismatch
- Manual changes break linkage

HOW TO CHECK

```
$ kubectl get rs
$ kubectl get rs -o wide
$ kubectl describe rs <rs>
$ kubectl get pods -l app=<app>
```

HOW TO FIX

- [x] Ensure correct selector
- [x] Check template changes
- [x] Delete old ReplicaSets
- [x] Avoid manual pod changes
- [x] Use kubectl rollout

2. ROLLOUTS

- Rollout stuck
- Image pull errors
- Probe failures
- MaxUnavailable blocking

HOW TO CHECK

```
$ kubectl rollout status deploy/<app>
$ kubectl rollout history deploy/<app>
$ kubectl describe deploy/<app>
$ kubectl get events --sort-by=.lastTimestamp
```

HOW TO FIX

- [x] Fix underlying issue
- [x] Use correct probes
- [x] Verify resource limits
- [x] Check node resources
- [x] Rerun rollout

3. ROLLBACKS

- Bad release deployed
- App crashes after update
- Config or DB incompatibility
- Traffic not healthy

HOW TO CHECK

```
$ kubectl rollout history deploy/<app>
$ kubectl rollout undo deploy/<app>
$ kubectl get rs
$ kubectl get pods
```

HOW TO FIX

- [x] Rollback to last stable
- [x] Fix root cause
- [x] Test in staging first
- [x] Use canary / blue-green
- [x] Validate after rollback

4. READINESS

- Readiness probe failing
- Pods not ready
- Slow startup
- Wrong probe path/port

HOW TO CHECK

```
$ kubectl get pods
$ kubectl describe pod <pod>
$ kubectl logs <pod>
$ kubectl exec -it <pod> --
  curl -v http://localhost:8080/health
```

HOW TO FIX

- [x] Fix application health
- [x] Correct probe path/port
- [x] Increase initialDelaySeconds
- [x] Tune timeout and failureThreshold
- [x] Ensure dependencies are up

5. AVAILABILITY

- Insufficient replicas
- Node failures
- PDB blocking
- Resource shortages

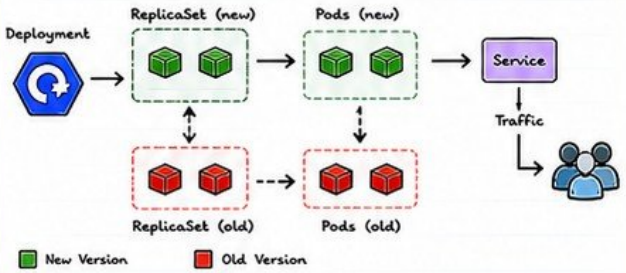
HOW TO CHECK

```
$ kubectl get deploy/<app>
$ kubectl get pods -o wide
$ kubectl get pdb
$ kubectl describe pdb <pdb>
```

HOW TO FIX

- [x] Increase replicas
- [x] Fix node capacity
- [x] Adjust PDB if needed
- [x] Check resource requests/limits
- [x] Ensure anti-affinity rules

DEPLOYMENT ARCHITECTURE OVERVIEW



ROLLING UPDATE PROCESS

1. Update Deployment
2. Create New ReplicaSet
3. Start new Pods
4. New Pods become Ready
5. Traffic shifts to new Pods
6. Old Pods terminated

ROLLING UPDATE CHECKLIST

- [x] Image exists and accessible
- [x] Resources are sufficient
- [x] Readiness/Liveness probes are correct
- [x] Dependencies (DB, APIs) are healthy
- [x] ConfigMaps/Secrets are correct
- [x] PDB allows update
- [x] Node capacity is sufficient
- [x] Observe rollout status

PRO TIP

- Always watch rollout status in another terminal.
- Fix the cause, not just the symptom.
- Automate rollbacks on failure.

ROLLBACK WORKFLOW

1. Detect Problem
2. Analyze Impact
3. Rollback to Last Good
4. Verify Health
5. Investigate Root Cause
6. Improve & Document

ROLLBACK COMMANDS

```
# See history
$ kubectl rollout history deploy/<app>
# Rollback to previous revision
$ kubectl rollout undo deploy/<app>
# Rollback to specific revision
$ kubectl rollout undo deploy/<app>
--to-revision=<rev>
# Check status
$ kubectl rollout status deploy/<app>
```

VIEW ROLLOUT HISTORY EXAMPLE

```
$ kubectl rollout history deploy/myapp
deployment.apps/myapp
REVISION  CHANGE-CAUSE
1         kubectl apply --filename=myapp.yaml
2         kubectl set image deploy/myapp app=v2
3         kubectl set image deploy/myapp app=v3 (current)
```

COMMON DEPLOYMENT FAILURE SIGNS

SIGN	WHAT IT MEANS	WHAT TO CHECK FIRST
Rollout stuck	New Pods not becoming Ready	Events, Logs, Probes
0/desired Pods	Pods not scheduled	Events, Resources, Nodes
ImagePullBackOff	Image cannot be pulled	Image name, Registry, Secrets
CrashLoopBackOff	App crashes after start	Logs, Config, Resources
Old Pods still running	Rollout not progressing	Readiness, PDB, MaxUnavailable

USEFUL DIAGNOSTIC COMMANDS

```
$ kubectl get deploy,pods,rs -l app=<app> -o wide   # Overview
$ kubectl describe deploy/<app>                     # Deployment details
$ kubectl describe rs <rs-name>                     # ReplicaSet details
$ kubectl describe pod <pod-name>                   # Pod details
$ kubectl logs <pod-name> -c <container>            # Container logs
$ kubectl get events --sort-by=.lastTimestamp       # Recent events
$ kubectl top pods                                  # Resource usage
$ kubectl rollout status deploy/<app>               # Watch rollout
```

REPLICASETS CHECKLIST

- [x] Correct selector
- [x] Matching labels
- [x] No manual changes
- [x] Old RS cleaned up

ROLLOUT CHECKLIST

- [x] Rollout status green
- [x] Events clean
- [x] Pods Ready
- [x] No CrashLoopBackOff

ROLLBACK CHECKLIST

- [x] History reviewed
- [x] Rolled back successfully
- [x] Service healthy
- [x] Root cause captured

READINESS CHECKLIST

- [x] Probe path correct
- [x] Dependencies up
- [x] Startup time tuned
- [x] Pods Ready

AVAILABILITY CHECKLIST

- [x] Desired replicas running
- [x] Nodes healthy
- [x] PDB allows updates
- [x] Resources sufficient

BEST PRACTICES

- [x] Canary or Blue-Green
- [x] Test in Staging
- [x] Monitor Everything
- [x] Automate Rollbacks

QUICK CHACTICES

- [x] Rollout failing?
- [x] Check events first
- [x] Check pods next
- [x] Fix root cause

GREAT DEPLOYMENTS ARE PLANNED. SMART ENGINEERS OBSERVE, ADAPT, AND RECOVER FAST.

kubernetes

SERVICE & NETWORKING ISSUES

networking

Services connect people to applications.
Networking connects everything in the cluster.

1. CLUSTERIP

IP

- Default service type
- Internal to cluster only
- Stable virtual IP
- DNS name: <svc>.<ns>.svc.cluster.local

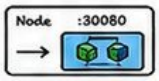
HOW TO CHECK

```
$ kubectl get svc -n <ns>
$ kubectl describe svc <svc>
$ kubectl get endpoints <svc>
```

HOW TO FIX

- ☑ Verify selector matches pods
- ☑ Check endpoints exist
- ☑ Validate service port
- ☑ Check pod readiness
- ☑ Verify NetworkPolicies

2. NODEPORT



- Exposes service on each node
- External access via <NodeIP>:<Port>
- Port range: 30000-32767
- Used for testing or bare metal

HOW TO CHECK

```
$ kubectl get svc -o wide
$ ss -tulnp | grep <port>
$ curl http://<NodeIP>:<Port>
```

HOW TO FIX

- ☑ Verify NodePort is open
- ☑ Check security groups / firewall
- ☑ Ensure kube-proxy is running
- ☑ Confirm service endpoints
- ☑ Test from another node

3. LOADBALANCER

- Provisions cloud load balancer
- External IP from cloud provider
- Health checks built-in
- Best for production

HOW TO CHECK

```
$ kubectl get svc -o wide
$ kubectl describe svc <svc>
$ kubectl get events
  --sort-by=.lastTimestamp
```

HOW TO FIX

- ☑ Check cloud controller manager
- ☑ Verify subnet & security group
- ☑ Check health check path/port
- ☑ Ensure targets are healthy
- ☑ Review cloud provider logs

4. ENDPOINTS

- List of pod IPs behind a service
- Created automatically
- Empty endpoints = no traffic
- Affects all service types

HOW TO CHECK

```
$ kubectl get endpoints <svc> -
  wide
$ kubectl describe endpoints <svc>
```

HOW TO FIX

- ☑ Pods must be Ready
- ☑ Selector must match labels
- ☑ Check pod container ports
- ☑ Verify readiness probes
- ☑ Check NetworkPolicies

5. KUBE-PROXY

proxy

- Maintains network rules
- Handles service routing
- Modes: iptables, IPVS
- Runs on every node

HOW TO CHECK

```
$ kubectl get pods -n kube-system
  -l k8s-app=kube-proxy
$ kubectl logs -n kube-system
  kube-proxy-xxxx
```

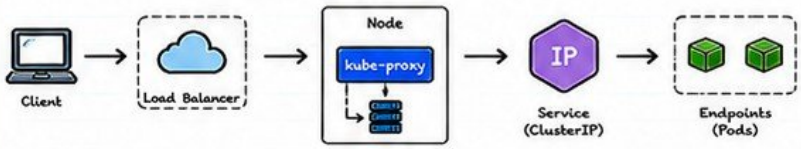
HOW TO FIX

- ☑ Restart kube-proxy pod
- ☑ Check node networking
- ☑ Verify iptables / IPVS rules
- ☑ Ensure CNI is working
- ☑ Check kube-proxy config

SERVICE TYPES OVERVIEW

TYPE	SCOPE	ACCESS	USE CASE	EXAMPLE
ClusterIP	Internal	Cluster only	Microservices	Backend API
NodePort	External	NodeIP:NodePort	Testing / Dev	NodeIP:30080
LoadBalancer	External	Cloud LB IP	Production	ALB / NLB
ExternalName	Internal	CNAME to external	External services	DNS mapping

SERVICE TRAFFIC FLOW (EXAMPLE)

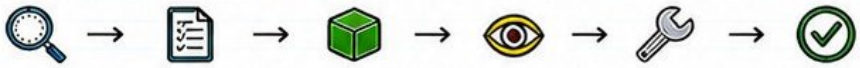


1. Request comes in
2. LB routes to a node
3. kube-proxy applies rules
4. Traffic sent to service IP
5. Traffic forwarded to pod IPs

COMMON NETWORKING ISSUES & SYMPTOMS

ISSUE	SYMPTOM
Service has no endpoints	kubectl get endpoints shows <none>
Pod not reachable	Connection refused / timeout
External access fails	NodePort / LB not reachable
Intermittent connectivity	Some pods reachable, some not
DNS resolution fails	Service name not resolving
Cross-namespace blocked	NetworkPolicy blocking traffic
kube-proxy down	All services unreachable

TROUBLESHOOTING WORKFLOW



1. **IDENTIFY** — What service is affected?
2. **CHECK SERVICE** — kubectl get svc describe svc
3. **CHECK ENDPOINTS** — Do endpoints exist?
4. **CHECK PODS** — Are pods Ready and running?
5. **CHECK NETWORK** — Policies, CNI, DNS, proxy
6. **VALIDATE** — Test from client inside/outside

ESSENTIAL COMMANDS CHEAT SHEET

```
$ kubectl get svc -A
$ kubectl get svc -o wide
$ kubectl describe svc <svc>
$ kubectl get endpoints <svc> -o wide
$ kubectl get pods -l app=<app> -o wide
$ kubectl exec -it <pod> -- curl <service>:<port>
$ kubectl run netshoot --rm -it --image=nicolaka/netshoot
$ ss -tulnp | grep <port>
$ iptables -t nat -L -n -v
$ ipvsadm -Ln
```

PRO TIP

Always test from inside the cluster first.
If internal works but external fails, focus on Node, LB, or firewall.

KUBE-PROXY MODES

IPTABLES MODE

- Default in most clusters
- Uses iptables NAT rules
- Good compatibility
- Slower at scale (large ruleset)

IPVS MODE

- High performance
- Better for large clusters
- Uses IPVS kernel modules
- Supports session persistence

CHECK KUBE-PROXY HEALTH

```
$ kubectl get pods -n
  kube-system -l
  k8s-app=kube-proxy
$ kubectl logs -n
  kube-system
  kube-proxy-xxxx
$ kubectl describe node
  <node> | grep -AS
  kube-proxy
$ iptables -t nat -L -n -v
$ ipvsadm -Ln
```

NETWORKING CHECKLIST

- ☑ CNI plugin is healthy
- ☑ Node network is stable
- ☑ Pod IPs are reachable
- ☑ DNS is resolving
- ☑ No blocking NetworkPolicies

SERVICE CHECKLIST

- ☑ Service selector matches pods
- ☑ Endpoints are populated
- ☑ Correct port and targetPort
- ☑ Health checks configured
- ☑ Service type as expected

LOAD BALANCER CHECKLIST

- ☑ LB created successfully
- ☑ Target group healthy
- ☑ Health check path works
- ☑ Security group allows traffic
- ☑ DNS record resolves

NODEPORT CHECKLIST

- ☑ Port within 30000-32767
- ☑ Port open on all nodes
- ☑ kube-proxy is running
- ☑ No firewall blocking
- ☑ Service endpoints exist

BEST PRACTICES

- ☑ Use ClusterIP for internal
- ☑ Use LB for production
- ☑ Monitor endpoints
- ☑ Use readiness probes
- ☑ Document network paths

REMEMBER

If users cannot reach the app, the problem is always in the network path. Prove each hop.

GREAT NETWORKING = HAPPY USERS. BAD NETWORKING = SILENT FAILURES. CHECK. TEST. VERIFY.

kubernetes

INGRESS TROUBLESHOOTING

ingress

Ingress is the gateway.
Troubleshoot layer by layer. Validate each hop.

1. CONTROLLERS

- Ingress needs a controller
- Controller watches Ingress
- Misconfigurations break routing
- Check controller health first

HOW TO CHECK

```
$ kubectl get pods -n ingress-nginx
$ kubectl logs deploy/ingress-nginx-controller
$ kubectl describe deploy/ingress-nginx-controller
```

HOW TO FIX

- [x] Restart controller pods
- [x] Fix configuration errors
- [x] Update controller image
- [x] Ensure Service is reachable

2. RULES

- Rules map host/path -> service
- Wrong host or path = 404
- Regex / path issues
- Rule order matters

HOW TO CHECK

```
$ kubectl get ingress -A
$ kubectl describe ingress <name>
$ kubectl get ingress <name> -o yaml
```

HOW TO FIX

- [x] Verify host and paths
- [x] Check pathType (Prefix/Exact)
- [x] Fix typos in service name
- [x] Validate rule order

3. TLS

- TLS cert issues cause failures
- Expired or wrong cert
- Secret missing or wrong name
- Handshake errors

HOW TO CHECK

```
$ openssl s_client -connect <host>:443 -servername <host>
$ kubectl get secrets -A
$ kubectl describe secret <tls-secret>
```

HOW TO FIX

- [x] Verify certificate validity
- [x] Check secret exists
- [x] Ensure cert matches host
- [x] Renew / replace certificate

4. BACKEND SERVICES

- Service must exist and work
- Endpoints must be ready
- Port or selector mismatch
- Pod readiness impacts routing

HOW TO CHECK

```
$ kubectl get svc <svc> -o wide
$ kubectl get endpoints <svc>
$ kubectl describe svc <svc>
```

HOW TO FIX

- [x] Fix selector / labels
- [x] Check port / targetPort
- [x] Ensure pods are Ready
- [x] Fix readiness probes

5. ROUTING FAILURES

- 404 = rule not matched
- 502/503 = bad backend
- 504 = timeout
- Check each layer in order

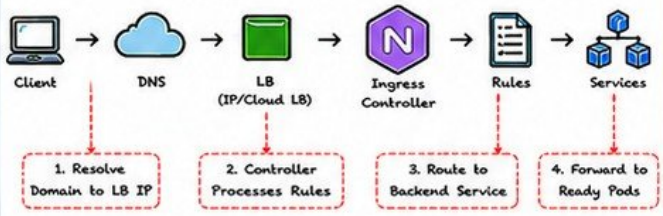
HOW TO CHECK

```
$ curl -I -H "Host: <host>" http://<ingress-ip>
$ kubectl logs ingress-nginx-controller-controller
$ kubectl get events --sort-by=.lastTimestamp
```

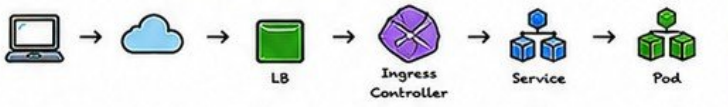
HOW TO FIX

- [x] Fix rule or host mismatch
- [x] Fix backend service
- [x] Increase timeouts
- [x] Check controller logs

INGRESS ARCHITECTURE OVERVIEW



REQUEST FLOW (EXAMPLE)



LAYERED TROUBLESHOOTING APPROACH

COMMON FAILURE CODES & MEANING

CODE	MEANING	COMMON CAUSE	CHECK FIRST
404	Not Found	Rule/Host/Path mismatch	Ingress rules
502	Bad Gateway	Service has no endpoints	Endpoints / Pods
503	Service Unavailable	No ready endpoints	Pod readiness
504	Gateway Timeout	Backend timeout	Timeout / Network

ESSENTIAL INGRESS COMMANDS

```
$ kubectl get ingress -A                                  # List all ingress
$ kubectl describe ingress <name>                         # Details & status
$ kubectl get ingress <name> -o yaml                      # Full configuration
$ kubectl logs deploy/ingress-nginx-controller -n ingress-nginx # Controller logs
$ kubectl get svc -n ingress-nginx                        # Controller Service
$ kubectl get endpoints <svc> -A                          # Service endpoints
$ kubectl get events --sort-by=.lastTimestamp             # Recent events
```

DEBUG WITH CURL (EXAMPLES)

```
# Test by IP (bypass DNS)
$ curl -I http://<ingress-ip>
$ curl -I -H "Host: app.veriqta.com" http://<ingress-ip>
# Test HTTPS (TLS handshake)
$ curl -vI https://app.veriqta.com
# Test path
$ curl -I -H "Host: app.veriqta.com" \\
  http://<ingress-ip>/api/health
```

INGRESS CONTROLLER HEALTH CHECK

- [x] Controller pods are Running
- [x] No CrashLoopBackOff
- [x] Service has correct ports
- [x] IngressClass matches controller
- [x] No critical errors in logs
- [x] Leader election is stable

SAMPLE CONTROLLER LOGS (WHAT TO LOOK FOR)

I	Configuration loaded successfully	✓ Good
I	NGINX ingress controller started	✓ Good
W	Error getting SSL certificate "app.veriqta.com"	⚠ Check TLS secret
E	no endpoints available for service "api-service"	✗ Check backend service
W	upstream timed out (110: Connection timed out)	⚠ Backend slow/timeout
I	Backend successfully reloaded	✓ Recovered

PRO TIP

- Always test from outside the cluster first.
- Use --resolve to test DNS quickly.
- Check one layer at a time.
- Read controller logs - they tell the truth.
- Most Ingress issues are rule or backend related.

BEST PRACTICES

- [x] Use specific hostnames
- [x] Keep rules simple and explicit
- [x] Use TLS everywhere
- [x] Monitor controller & endpoints
- [x] Set appropriate timeouts
- [x] Document your Ingress rules

CONTROLLER CHECKLIST
- [x] Pods running
- [x] No errors in logs
- [x] Service reachable
- [x] Correct IngressClass

RULES CHECKLIST
- [x] Hosts are correct
- [x] Paths are valid
- [x] pathType set
- [x] No typos

TLS CHECKLIST
- [x] Cert not expired
- [x] Secret exists
- [x] Host matches cert
- [x] Full chain valid

BACKEND CHECKLIST
- [x] Service exists
- [x] Endpoints exist
- [x] Pods are Ready
- [x] Ports are correct

NETWORK CHECKLIST
- [x] LB reachable
- [x] Firewall allows 80/443
- [x] DNS resolves
- [x] No network policies blocking

QUICK REMINDER
Validate.
Observe.
Fix.
Verify again.

A GOOD INGRESS = HAPPY USERS. A BROKEN INGRESS = NO TRAFFIC.

kubernetes

PERSISTENT STORAGE PROBLEMS

storage

Data is critical. Storage must be reliable.
Understand the layers. Follow the path. Fix the root cause.

1. PV (PERSISTENT VOLUME)

PV

- Cluster resource
- Provisioned by admin or dynamically
- Backing storage in the cluster
- Independent of claims and pods

HOW TO CHECK

```
$ kubectl get pv
$ kubectl describe pv <pv-name>
$ kubectl get pv -o wide
$ kubectl get events --sort-by=.lastTimestamp
```

HOW TO FIX

- ☑ Verify storage exists
- ☑ Check capacity and accessModes
- ☑ Fix node affinity / labels
- ☑ Recreate or reprovision PV
- ☑ Check underlying storage health

2. PVC (PERSISTENT VOLUME CLAIM)

PVC

- Request storage from PV
- Used by Pods to mount storage
- Bound to a matching PV
- Namespace scoped

HOW TO CHECK

```
$ kubectl get pvc
$ kubectl describe pvc <pvc-name>
$ kubectl get pvc -n <ns> -o wide
$ kubectl get events -n <ns>
```

HOW TO FIX

- ☑ Check status (Bound/Pending)
- ☑ Verify StorageClass & size
- ☑ Ensure PV exists or provision
- ☑ Fix accessModes mismatch
- ☑ Delete & recreate if stuck

3. STORAGECLASS

- Defines provisioner and parameters
- Used for dynamic provisioning
- Tells Kubernetes how to create PVs
- Default SC is used if none set

HOW TO CHECK

```
$ kubectl get sc
$ kubectl describe sc <sc-name>
$ kubectl get sc -o wide
$ kubectl get events --all-namespaces
```

HOW TO FIX

- ☑ Verify provisioner is healthy
- ☑ Check parameters and zones
- ☑ Set default StorageClass
- ☑ Fix typo in SC name
- ☑ Recreate StorageClass if broken

4. CSI (CONTAINER STORAGE INTERFACE)

- Standard for storage plugins
- CSI driver connects K8s to storage
- Handles attach, mount, unmount
- Managed by driver pods

HOW TO CHECK

```
$ kubectl get csidrivers
$ kubectl get pods -n kube-system \
  -l app.kubernetes.io/name in (csi-*)
$ kubectl logs -n kube-system <csi-pod>
```

HOW TO FIX

- ☑ Check CSI driver pods running
- ☑ Check logs for errors
- ☑ Verify node plugin DaemonSet
- ☑ Restart CSI driver pods
- ☑ Reinstall / upgrade driver

5. MOUNT FAILURES

- Pods cannot mount the volume
- Common cause of app failures
- Check path, permissions, quotas
- Node or driver issues

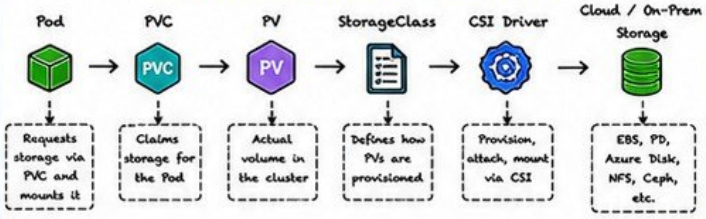
HOW TO CHECK

```
$ kubectl describe pod <pod-name>
$ kubectl get events -n <ns> \
  --sort-by=.lastTimestamp
$ kubectl logs <csi-pod> -n kube-system
```

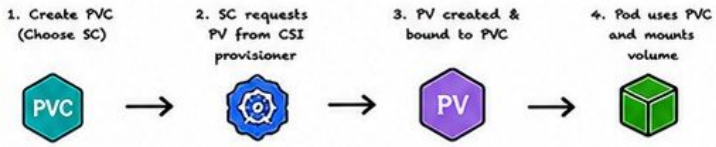
HOW TO FIX

- ☑ Verify PVC is Bound
- ☑ Check Pod events & logs
- ☑ Check node disk & Kubelet
- ☑ Verify permissions & fsGroup
- ☑ Reattach or remount volume

KUBERNETES STORAGE ARCHITECTURE



STORAGE FLOW (DYNAMIC PROVISIONING)



COMMON STORAGE PROBLEMS, CAUSES & SOLUTIONS

PROBLEM	COMMON CAUSE	SOLUTION
PVC Pending	No matching PV or SC misconfigured	Check SC, PV availability, events
PVC Bound but Pod Pending	Node affinity / accessModes mismatch	Check PV nodeAffinity & accessModes
MountVolume.SetUp failed	CSI driver issue / wrong permissions	Check CSI logs, permissions, fsGroup
Multi-Attach error	RWO volume mounted on multiple nodes	Use RWX or fix deployment
Filesystem not found	Volume formatted incorrectly	Format volume or recreate PV
ReadOnly mount	PV mounted readOnly	Check PV spec & mount options
Quota exceeded	Storage quota exceeded	Increase quota or clean up data

EVENTS EXAMPLES (WHAT TO LOOK FOR)

```
Warning   FailedMount            MountVolume.SetUp failed for volume "pvc-123" :
                                 rpc error: code = Internal desc = mount failed
Warning   ProvisioningFailed     failed to provision volume with StorageClass "gp2-sc":
                                 no available zones
Warning   FailedAttachVolume     AttachVolume.Attach failed for volume "vol-abc":
                                 Multi-Attach error
Normal    ProvisioningSucceeded  Successfully provisioned volume pvc-123
Warning   FailedScheduling       0/3 nodes are available: 3 node(s) had volume node affinity conflict.
Warning   VolumeInUse           Volume is already exclusively attached to one node
```

Always sort events by time to see the full story.

```
$ kubectl get events -A --sort-by=.lastTimestamp
```

STORAGE TROUBLESHOOTING WORKFLOW

1. **IDENTIFY** — Check Pod & PVC status and events
2. **CHECK PVC/PV** — Verify Bound and capacity
3. **CHECK SC & CSI** — Verify StorageClass and CSI driver
4. **CHECK NODE** — Disk space, Kubelet and permissions
5. **ANALYZE EVENTS** — Find root cause from events/logs
6. **FIX & VALIDATE** — Apply fix and verify Pod is Running

USEFUL COMMANDS CHEAT SHEET

```
$ kubectl get pv                                          # List PVs
$ kubectl get pvc -A                                      # List PVCs
$ kubectl describe pv <pvc-name>                          # PV details
$ kubectl describe pvc <pvc-name> -n <ns>                 # PVC details
$ kubectl get sc                                          # StorageClasses
$ kubectl get csidrivers                                  # CSI Drivers
$ kubectl get pods -n kube-system -l app in (csi-*)       # CSI Driver Pods
$ kubectl logs -n kube-system <csi-pod>                   # CSI Driver Logs
$ df -h    # on the node                                  # Check disk space
$ journalctl -u kubelet -f    # on the node               # Check kubelet logs
```

PV CHECKLIST

- ☑ Status is Available or Bound
- ☑ Capacity is correct
- ☑ AccessModes match
- ☑ ReclaimPolicy is correct
- ☑ NodeAffinity (if set)

PVC CHECKLIST

- ☑ Status is Bound
- ☑ StorageClass is correct
- ☑ Requested size is valid
- ☑ AccessModes match
- ☑ Namespace is correct

STORAGECLASS CHECKLIST

- ☑ Provisioner is correct
- ☑ Parameters are valid
- ☑ defaultClass set (if needed)
- ☑ AllowVolumeExpansion set
- ☑ ReclaimPolicy understood

CSI CHECKLIST

- ☑ CSI pods are Running
- ☑ No errors in CSI logs
- ☑ Node plugin running
- ☑ Attacher/Provisioner OK
- ☑ Version compatibility

MOUNT FAILURE CHECKLIST

- ☑ PVC is Bound
- ☑ Pod events show mount error
- ☑ Permissions & fsGroup set
- ☑ Node disk has space
- ☑ App path exists

PRO TIP

Storage issues are layered. Check from the application down to the disk.
The path never lies.

GOOD STORAGE = RELIABLE APPLICATIONS. BAD STORAGE = DATA LOSS & OUTAGES.

kubernetes

NODE TROUBLESHOOTING

node

A healthy node is the foundation of a healthy cluster.
Think: Network → Kubelet → Runtime → Resources → Scheduling.

1. NODE NOTREADY

- Node status is NotReady
- Pods cannot run or are evicted
- Kubernetes can't reach kubelet
- Cluster capacity is reduced

HOW TO CHECK

```
$ kubectl get nodes
$ kubectl describe node <node>
$ kubectl get events
  --sort-by=.lastTimestamp
```

HOW TO FIX

- ☑ Check network connectivity
- ☑ Restart kubelet service
- ☑ Check kubelet logs
- ☑ Verify node resource pressure
- ☑ Fix underlying root cause

2. KUBELET ISSUES

- Kubelet not running or failing
- Cannot register node status
- Pod lifecycle failures
- Image pull or mount issues

HOW TO CHECK

```
$ systemctl status kubelet
$ journalctl -u kubelet -xe
$ ps -ef | grep kubelet
$ kubectl describe node <node>
```

HOW TO FIX

- ☑ Restart kubelet service
- ☑ Check kubelet config
- ☑ Fix invalid flags / certs
- ☑ Verify node IP / hostname
- ☑ Ensure kubelet can reach API

3. CONTAINER RUNTIME

- Container runtime down
- Cannot start containers
- Image pull failures
- Pod sandbox creation fails

HOW TO CHECK

```
$ systemctl status containerd
$ crictl info
$ crictl ps -a
$ journalctl -u containerd -xe
```

HOW TO FIX

- ☑ Restart container runtime
- ☑ Check runtime logs
- ☑ Verify cgroup driver match
- ☑ Clean up corrupted images
- ☑ Check disk and inode usage

4. RESOURCE PRESSURE

- High CPU / Memory usage
- Disk pressure or inode full
- Evictions and throttling
- Node becomes unstable

HOW TO CHECK

```
$ kubectl describe node <node>
$ kubectl top node
$ df -h
$ free -h
```

HOW TO FIX

- ☑ Free up disk space
- ☑ Remove unused images
- ☑ Tune pod resource limits
- ☑ Increase node capacity
- ☑ Investigate noisy workloads

5. SCHEDULING FAILURES

- Pods stay in Pending
- No nodes available
- Taints / tolerations mismatch
- Insufficient resources

HOW TO CHECK

```
$ kubectl get pods -A
$ kubectl describe pod <pod>
$ kubectl get nodes
$ kubectl describe node <node>
```

HOW TO FIX

- ☑ Check taints and tolerations
- ☑ Verify node selectors/affinity
- ☑ Increase cluster capacity
- ☑ Fix resource requests/limits
- ☑ Remove unschedulable taints

NODE STATUS QUICK REFERENCE

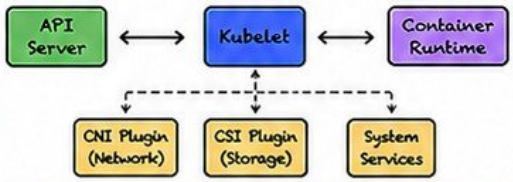
STATUS	MEANING	WHAT TO DO
Ready	Node is healthy	Monitor
NotReady	Kubelet not reporting	Investigate immediately
Unknown	Controller can't reach node	Check network / kubelet

- Ready = True
- MemoryPressure = False
- DiskPressure = False
- PIDPressure = False
- NetworkUnavailable = False

RESOURCE PRESSURE INDICATORS

TYPE	SYMPTOMS	CHECK COMMANDS
CPU	High CPU usage, throttling	kubectl top node
Memory	OOMKills, evictions	free -h
Disk	Low disk space	df -h
Inodes	Disk full (inodes)	df -i
PID	Cannot create processes	cat /proc/sys/kernel/pid_max

NODE COMPONENT ARCHITECTURE



NODE TROUBLESHOOTING WORKFLOW

1. IDENTIFY — Node NotReady or issue detected
2. CHECK — Network connectivity
3. CHECK — Kubelet health
4. CHECK — Container runtime
5. CHECK — Resources (CPU/Memory/Disk)
6. CHECK — Scheduling constraints
7. RESOLVE — Fix root cause

ESSENTIAL COMMANDS CHEAT SHEET

```
$ kubectl get nodes -o wide                          # List nodes with IPs
$ kubectl describe node <node>                       # Node details and conditions
$ kubectl get events --sort-by=.lastTimestamp        # Recent events
$ kubectl top node                                   # Resource usage
$ kubectl get pods -A --field-selector spec.nodeName=<node>  # Pods on a node
$ journalctl -u kubelet -xe                          # Kubelet logs
$ journalctl -u containerd -xe                       # Container runtime logs
$ crictl ps -a                                       # Containers via CRI
$ df -h && df -i                                     # Disk space and inodes
$ free -h                                            # Memory usage
```

COMMON NODE ISSUES & CAUSES

ISSUE	COMMON CAUSE
Node NotReady	Kubelet down, network, cert, runtime
Kubelet not running	Service stopped, config error
Container runtime down	Service stopped, crash, config
Image pull failures	Registry auth, network, DNS
Disk pressure	Disk full, logs, images
Memory pressure	High usage, leaks, limits
Scheduling failures	Taints, affinity, lack of resources

POD SCHEDULING CHECKLIST

- ☑ Enough CPU/Memory on nodes
- ☑ Correct node selectors
- ☑ Taints and tolerations match
- ☑ Node affinity / anti-affinity
- ☑ Pod resource requests are set
- ☑ Not hitting pod or node limits
- ☑ Storage (PVC) can be mounted
- ☑ No confficting constraints

NODE NOTREADY CHECKLIST

- ☑ Check network (ping API server)
- ☑ Check kubelet service
- ☑ Check kubelet logs
- ☑ Check node conditions
- ☑ Check disk and memory
- ☑ Check container runtime
- ☑ Fix and verify

KUBELET HEALTH CHECKLIST

- ☑ Service running
- ☑ Correct config file
- ☑ Valid certificates
- ☑ Able to reach API server
- ☑ No error in logs
- ☑ Cgroup driver matches
- ☑ System time is correct

RUNTIME HEALTH CHECKLIST

- ☑ Runtime service running
- ☑ crictl info works
- ☑ No failed sandboxes
- ☑ Images can be pulled
- ☑ Storage driver healthy
- ☑ Enough disk space
- ☑ Logs show no errors

RESOURCE HEALTH CHECKLIST

- ☑ CPU usage normal
- ☑ Memory usage normal
- ☑ Disk space sufficient
- ☑ Inodes available
- ☑ No frequent evictions
- ☑ System load normal
- ☑ No OOMKills

PRO TIP

- Start from the bottom (node) and move up.
- Fix one layer at a time.
- Events tell the story.
- Data > Guesswork.

REMEMBER

A broken node affects many workloads.
Detect early.
Investigate deep.
Resolve completely.
Then monitor.

HEALTHY NODES = STABLE CLUSTER. STABLE CLUSTER = HAPPY USERS.

kubernetes

CONTROL PLANE TROUBLESHOOTING

control plane

The control plane is the brain of Kubernetes.
When it struggles, the whole cluster suffers.

1. API SERVER

- Entry point for all cluster operations
- Handles auth, authz, admission & request routing
- If API is down, cluster is unusable

HOW TO CHECK

```
$ kubectl get --raw='/healthz'
$ kubectl get --raw='/readyz'
$ kubectl get apiservices
$ kubectl get --raw='/metrics'
```

HOW TO FIX

- ☑ Check API server pod logs
- ☑ Check certs / expiring certs
- ☑ Check network connectivity
- ☑ Verify kubeconfig / RBAC
- ☑ Review admission plugins
- ☑ Restart static pod if needed

2. ETCD

- Key-value store for all cluster data
- Slowness or data loss breaks the cluster
- Quorum is critical

HOW TO CHECK

```
$ kubectl -n kube-system exec -it etcd-<pod> -- etcdctl endpoint health
$ kubectl -n kube-system exec -it etcd-<pod> -- etcdctl member list
$ etcdctl endpoint status --write-out=table
```

HOW TO FIX

- ☑ Check disk latency & IOPS
- ☑ Ensure quorum (odd number)
- ☑ Defrag if database is large
- ☑ Restore from backup if needed
- ☑ Replace unhealthy member
- ☑ Check etcd logs

3. SCHEDULER

- Decides which node runs new Pods
- Scheduling issues cause Pending Pods
- Relies on API & etcd

HOW TO CHECK

```
$ kubectl -n kube-system logs kube-scheduler-<node>
$ kubectl get pods -A --field-selector spec.nodeName=''
$ kubectl describe pod <pod>
```

HOW TO FIX

- ☑ Check scheduler logs
- ☑ Verify node resources
- ☑ Check taints / tolerations
- ☑ Review scheduling profiles
- ☑ Ensure scheduler running
- ☑ Restart scheduler if needed

4. CONTROLLER MANAGER

- Runs core controllers (Deployment, Node, Endpoints, Service, etc.)
- If down, cluster state drifts
- Reconciles desired vs actual

HOW TO CHECK

```
$ kubectl -n kube-system logs kube-controller-manager-<node>
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl get nodes -o wide
```

HOW TO FIX

- ☑ Check controller-manager logs
- ☑ Verify leader election
- ☑ Ensure etcd connectivity
- ☑ Check flags / config
- ☑ Restart controller manager

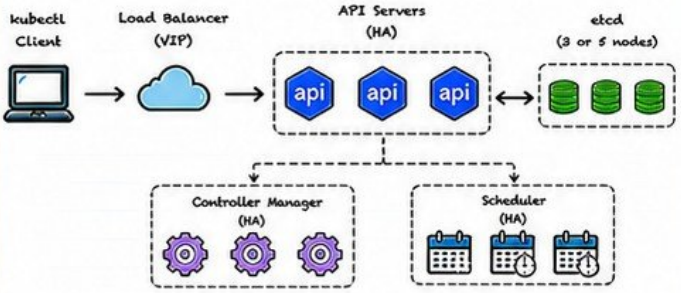
5. HA CONSIDERATIONS

- Run control plane on multiple nodes
- Use load balancer for API Server
- Maintain etcd quorum
- Plan for failure, test failover

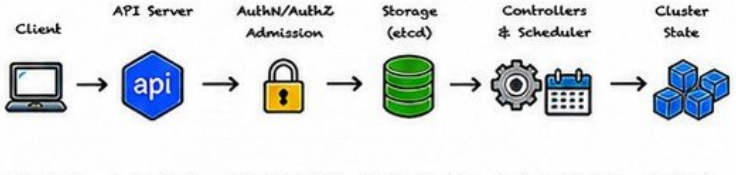
BEST PRACTICES

- ☆ 3 or 5 control plane nodes
- ☆ Odd number for etcd
- ☆ Use external LB for API
- ☆ Monitor all components
- ☆ Back up etcd regularly
- ☆ Test disaster recovery

CONTROL PLANE ARCHITECTURE (HIGH AVAILABILITY)



CONTROL PLANE REQUEST FLOW



1. Request sent
2. Accepted by API
3. Validated & Authorized
4. Persisted in etcd
5. Reconciled to actual state
6. Result returned

COMMON CONTROL PLANE FAILURE SYMPTOMS

SYMPTOM	POSSIBLE CAUSE	WHAT TO CHECK
kubectl timeout	API Server down/unreachable	API Server pod, network, LB
503 Service Unavailable	API overloaded	API metrics, CPU, etcd latency
Pods stuck in Pending	Scheduler down / misconfigured	Scheduler logs, node resources
Resources not updating	Controller Manager down	Controller Manager logs, events
Intermittent errors	etcd slowness	etcd health, disk latency, quorum
Cluster split brain	etcd quorum lost	etcd members, network partitions

ESSENTIAL CHECK COMMANDS

```
$ kubectl cluster-info                                  # Basic cluster info
$ kubectl get componentstatuses                         # Component health
$ kubectl get pods -n kube-system -o wide               # All control plane pods
$ kubectl describe pod <pod> -n kube-system             # Inspect a control plane pod
$ kubectl logs <pod> -n kube-system                     # View logs
$ kubectl get --raw='/readyz?verbose'                   # API readiness checks
$ kubectl get --raw='/healthz'-                         # API health checks
$ kubectl get events --all-namespaces --sort-by=.lastTimestamp  # Recent events
$ kubectl top node                                      # Node resource usage
$ kubectl -n kube-system exec -it <etcd-pod> -- etcdctl endpoint health  # Etcd endpoint health
```

ETCD HEALTH CHECK LIST

- ☑ Quorum is healthy (majority up)
- ☑ Disk space > 20%
- ☑ DB size stable (defrag if > 2GB)
- ☑ No high commit latency
- ☑ Backups are recent & tested
- ☑ Network between members stable

PRO TIP

If API Server is down, you cannot fix the cluster from inside it. Use static pod access, node SSH, and etcdctl to investigate.

HA FAILURE SCENARIOS

SCENARIO	IMPACT	ACTION
1 API Server down	Partial/No access	LB routes to healthy API
etcd member down	No impact if quorum	Monitor, replace failed member
etcd quorum lost	Cluster read-only / down	Restore from backup
Scheduler down	New pods not scheduled	Restart scheduler
Controller Manager down	State drift / outages	Restart controller manager
All control plane down	Cluster completely down	Restore, follow DR plan

KEY METRICS (WHAT TO MONITOR)

- API Server: request rate, latency, errors
- etcd: commit latency, has_leader, db_size
- Scheduler: scheduling attempts, latency
- Controller Manager: reconciliations, errors
- Cluster: events rate, node count, pod count
- Resource usage: CPU, Memory, Disk, Network

A STABLE CONTROL PLANE = A STABLE CLUSTER. FAIL FAST, MONITOR EVERYTHING, TEST RECOVERY.

kubernetes

KUBERNETES NETWORKING DEEP DIVE

networking

Network is the foundation.
Understand the layers. Troubleshoot with clarity. Scale with confidence.

1. CNI (CONTAINER NETWORK INTERFACE)

- Connects Pods to network
- Provides IPAM, routing, bandwidth, connectivity
- Example: Calico, Flannel, Weave, Cilium

HOW TO CHECK

```
$ kubectl get pods -A -o wide
$ kubectl describe pod <pod>
$ ls /etc/cni/net.d/
```

HOW TO FIX

- ☑ Check CNI daemon
- ☑ Verify CNI configuration
- ☑ Restart CNI pods
- ☑ Check node networking

2. COREDNS

- Provides DNS for the cluster
- Resolves Services to IPs
- Handles internal and external lookups

HOW TO CHECK

```
$ kubectl -n kube-system get pods | grep coredns
$ kubectl -n kube-system logs deploy/coredns
$ kubectl get svc -n kube-system kube-dns
```

HOW TO FIX

- ☑ Check CoreDNS pods
- ☑ Verify ConfigMap (coredns)
- ☑ Check DNS resolution
- ☑ Restart CoreDNS if needed

3. NETWORKPOLICIES

- Control pod-to-pod and pod-to-service traffic
- Define allow/deny rules
- Enforced by network plugin

HOW TO CHECK

```
$ kubectl get networkpolicies -A
$ kubectl describe networkpolicy <np>
$ kubectl get pods -o wide
```

HOW TO FIX

- ☑ Verify policy syntax
- ☑ Check labels and selectors
- ☑ Ensure CNI supports policies
- ☑ Test connectivity after changes

4. SERVICE DISCOVERY

- Services discovered via DNS
- Pods use DNS to reach Services
- Dynamic and automatic updates

HOW TO CHECK

```
$ kubectl get svc -A
$ kubectl run test --image=busybox
  --rm -it -- nslookup kubernetes
$ dig @kube-dns kubernetes.default
```

HOW TO FIX

- ☑ Check CoreDNS
- ☑ Verify Service existence
- ☑ Check DNS configuration
- ☑ Restart CoreDNS if needed

5. POD COMMUNICATION

- Pods communicate via IP or DNS
- Depends on CNI and policies
- Intra-node and inter-node traffic

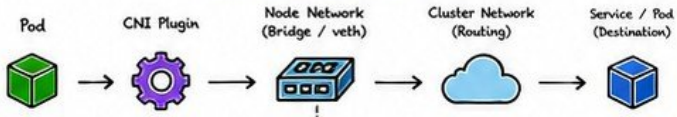
HOW TO CHECK

```
$ kubectl exec -it pod-a -- ping pod-b
$ kubectl get endpoints -A
$ kubectl get networkpolicies -A
```

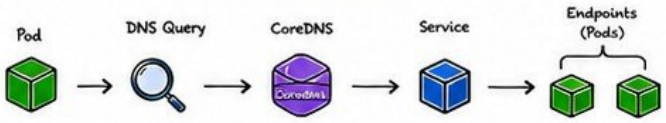
HOW TO FIX

- ☑ Check pod IPs and DNS
- ☑ Verify NetworkPolicies
- ☑ Verify CNI connectivity
- ☑ Check firewall / routing

KUBERNETES NETWORKING FLOW



SERVICE DISCOVERY FLOW



Pods use DNS to find Services. CoreDNS resolves to ClusterIP of Service.

NETWORKPOLICY EXAMPLE

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: frontend
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: backend
    ports:
    - protocol: TCP
      port: 80
```

CORE DNS CONFIG (SNIPPET)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
      errors
      health
      kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
      }
      forward . /etc/resolv.conf
      cache 30
      loop
      reload
    }
```

COMMON NETWORK COMMANDS

```
$ kubectl get pods -A
$ kubectl get svc -A
$ kubectl get endpoints -A
$ kubectl get networkpolicies -A
$ kubectl describe networkpolicy <np>
$ kubectl exec -it <pod> -- sh
$ kubectl logs <pod> -n <ns>
$ nslookup <service>.<ns>.svc.cluster.local
$ dig <service>.<ns>.svc.cluster.local
```

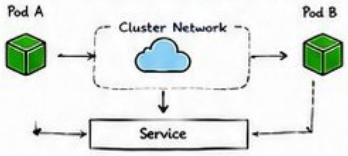
TROUBLESHOOTING CHECKLIST

- ☑ Verify CNI plugin running
- ☑ Check Pod IPs and connectivity
- ☑ Verify CoreDNS and Services
- ☑ Validate NetworkPolicies
- ☑ Test Service discovery (DNS)
- ☑ Check Pod to Pod communication
- ☑ Review logs and events

COMMON ISSUES & SOLUTIONS

ISSUE	CHECK	SOLUTION
Pods cannot communicate	ip addr, ping, NetworkPolicy	Check CNI and policies
DNS resolution fails	nslookup, CoreDNS logs	Check CoreDNS config
Service not reachable	kubectl get svc, endpoints	Check selector & endpoints
Traffic blocked	NetworkPolicy, CNI logs	Adjust NetworkPolicy
High latency	Network path, CNI metrics	Optimize CNI and routes

NETWORK MODEL (OVERVIEW)



- Pods communicate via IP
- Services provide abstraction
- Network Policies control traffic
- CNI connects everything

BEST PRACTICES

- ☑ Use NetworkPolicies
- ☑ Monitor CNI and DNS
- ☑ Use Services for discovery
- ☑ Test connectivity regularly
- ☑ Document network architecture

NETWORK SMART. TROUBLESHOOT FASTER. BUILD BETTER.

kubernetes

PRODUCTION KUBERNETES INCIDENT RESPONSE

Be Ready. Respond Fast.

Incidents will happen. Senior engineers are measured by how fast they investigate, decide, and restore.

1. INVESTIGATION WORKFLOW

- Define the blast radius
- Check user impact
- Validate affected components
- Collect signals & telemetry
- Form hypotheses
- Drill down and confirm root cause

KEY ACTIONS

- ☑ Check alerts & dashboards
- ☑ Validate symptoms
- ☑ Use cluster-first approach
- ☑ Follow the traffic path
- ☑ Identify recent changes

2. RECOVERY PRIORITIES

- Stop the bleeding
- Restore critical services
- Maintain data safety
- Communicate updates
- Restore full functionality
- Prevent recurrence

KEY ACTIONS

- ☑ Prioritize user impact
- ☑ Restore availability first
- ☑ Maintain data integrity
- ☑ One change at a time
- ☑ Validate after each step

3. EVIDENCE COLLECTION

- Capture state before changes
- Collect logs, events, configs
- Take screenshots
- Export diagnostics
- Preserve timestamps
- Store securely

KEY ACTIONS

- ☑ Don't restart yet
- ☑ Capture logs & events
- ☑ Save YAML of objects
- ☑ Take screenshots
- ☑ Document timeline

4. ESCALATION

- Escalate based on severity
- Involve domain experts
- Engage platform / SRE team
- Notify stakeholders
- Use clear communication
- Provide impact & ETA

KEY ACTIONS

- ☑ Follow on-call process
- ☑ Provide clear summary
- ☑ Share evidence collected
- ☑ Update regularly
- ☑ Escalate early, not late

5. COMMUNICATION

- Communicate early
- Use simple language
- Share impact & actions
- Provide ETAs
- Update until resolved
- Post-incident summary

KEY ACTIONS

- ☑ Declare incident
- ☑ Update stakeholders
- ☑ Use status page / Slack
- ☑ Clear & honest updates
- ☑ Share resolution details

KUBERNETES INCIDENT INVESTIGATION WORKFLOW

Alert	Triage	Validate Impact	Collect Evidence	Analyze	Identify Root Cause	Recover	Validate	Post Incident Review
Alert from monitoring or user report	Confirm severity & acknowledge incident	Confirm what is affected & who is impacted	Capture logs, events, metrics, configs, YAML	Correlate signals & look for patterns	Confirm the root cause with evidence	Implement fix & restore service	Verify service is healthy and stable	Document what happened & improve

CLUSTER-FIRST INVESTIGATION CHECKLIST

- ☑ Are the Nodes healthy?
- ☑ Are the Pods running?
- ☑ Are the Services having Endpoints?
- ☑ Are the Ingress rules correct?
- ☑ Are the NetworkPolicies blocking traffic?
- ☑ Are there recent changes?
- ☑ Are there resource issues?
- ☑ Are there application errors?
- ☑ What do the Events say?

SENIOR TIP

Always start at the Cluster level. Don't debug the application until the platform is proven healthy.

EVIDENCE COLLECTION COMMANDS

```
$ kubectl get nodes -o wide                          # Node status
$ kubectl get pods -A -o wide                        # Pod status
$ kubectl get events -A --sort-by=.lastTimestamp     # Recent events
$ kubectl describe pod <pod> -n <ns>                 # Pod details
$ kubectl logs <pod> -n <ns> --all-containers --tail=200   # Pod logs
$ kubectl get svc,endpoints -A -o wide               # Service & endpoints
$ kubectl get ingress -A -o wide                     # Ingress status
$ kubectl get networkpolicies -A                     # Network Policies
$ kubectl top nodes                                  # Node resources
$ kubectl top pods -A                                # Pod resources
```

RECOVERY PRIORITY MATRIX

PRIORITY	FOCUS	EXAMPLES
P1 (Critical)	Restore availability	API down, Payments down
P2 (High)	Restore core services	Web down, Queue down
P3 (Medium)	Restore secondary services	Reports down, Jobs failing
P4 (Low)	Optimize & cleanup	Non-critical features

SEVERITY LEVELS (EXAMPLE)

SEVERITY	IMPACT	RESPONSE TIME
P1 - Critical	Major outage, users blocked	0 - 15 minutes
P2 - High	Degraded service, limited impact	15 - 60 minutes
P3 - Medium	Minor issues, workaround available	1 - 4 hours
P4 - Low	No user impact, cosmetic issues	4+ hours

ESCALATION PATH (EXAMPLE)

On-Call Engineer
↓
SRE / Platform Team
↓
Engineering / Dev Team
↓
Management / Leadership

REMEMBER

Fast investigation
+ correct action
+ clear communication
= Happy users
and stable systems.

COMMON INCIDENT TYPES

- Pod Crash / CrashLoopBackOff
- Pods in Pending
- Service has no Endpoints
- Ingress returning 5xx
- Node NotReady
- High CPU / Memory
- Database connection failures

QUICK TRIAGE COMMANDS

```
$ kubectl get nodes
$ kubectl get pods -A
$ kubectl get svc,endpoints -A
$ kubectl get ingress -A
$ kubectl get events -A
$ kubectl top nodes
$ kubectl top pods -A
```

POST INCIDENT REVIEW

- ☑ What happened?
- ☑ What was the root cause?
- ☑ What was the impact?
- ☑ What went well?
- ☑ What can we improve?
- ☑ Action items & owners
- ☑ Update runbooks & alerts

USEFUL TOOLS

- Prometheus / Grafana
- ELK / OpenSearch
- Loki
- Datadog / New Relic
- Kubectl
- K9s / Lens
- Slack / PagerDuty

PRO TIP

Practice incident response like a fire drill. Run game days. The best teams prepare before production burns.

PREPARE BEFORE. INVESTIGATE FAST. COMMUNICATE CLEAR. RECOVER SAFELY. LEARN ALWAYS.

AWS TROUBLESHOOTING

Great engineers don't guess.
They follow a process, collect evidence, and eliminate possibilities.

Fix Fast.
Stay Calm.

1. EC2 INSTANCES

- Instance not reachable
- High CPU / Memory
- Boot / OS issues
- EBS volumes not attached
- Status check failures

HOW TO CHECK

```
$ aws ec2 describe-instances
$ aws ec2 get-console-output
$ aws ec2 describe-status-checks
$ aws ec2 describe-volumes
```

HOW TO FIX

- ☑ Verify instance state
- ☑ Check security groups
- ☑ Check system logs (OS)
- ☑ Attach / reattach volumes
- ☑ Replace or stop problematic instance

2. IAM (IDENTITY & ACCESS)

- Access denied errors
- Invalid credentials
- Permissions not working
- MFA / AssumeRole issues
- Service not authorized

HOW TO CHECK

```
$ aws sts get-caller-identity
$ aws iam simulate-principal-policy
$ aws iam list-attached-user-policies
$ aws cloudtrail lookup-events
```

HOW TO FIX

- ☑ Verify user / role permissions
- ☑ Attach correct policies
- ☑ Check trust relationship
- ☑ Validate MFA setup
- ☑ Review CloudTrail for denials

3. VPC (NETWORKING)

- Cannot connect to resources
- No internet access
- Subnets not reachable
- NACL / SG blocking traffic
- VPC peering / Endpoint issues

HOW TO CHECK

```
$ aws ec2 describe-vpcs
$ aws ec2 describe-subnets
$ aws ec2 describe-network-acls
$ aws ec2 describe-vpc-endpoints
```

HOW TO FIX

- ☑ Validate CIDR & subnets
- ☑ Check route tables
- ☑ Verify NACL rules
- ☑ Check Internet / NAT Gateway
- ☑ Test connectivity (ping / telnet)

4. SECURITY GROUPS

- Ports blocked
- Inbound / Outbound issues
- Rule conflicts
- SSH / RDP connection failures
- Application not reachable

HOW TO CHECK

```
$ aws ec2 describe-security-groups
$ aws ec2 describe-instances
$ aws ec2 describe-network-interfaces
$ nc -vz <host> <port>
```

HOW TO FIX

- ☑ Add / correct inbound rules
- ☑ Allow ephemeral ports outbound
- ☑ Remove conflicting rules
- ☑ Restrict 0.0.0.0/0 if not needed
- ☑ Attach correct SG to resources

5. ROUTE TABLES

- Wrong route / blackhole
- No route to internet
- Traffic not reaching target
- Peering / TGW route issues
- Propagation not working

HOW TO CHECK

```
$ aws ec2 describe-route-tables
$ aws ec2 describe-routes
$ aws ec2 get-route-table-associations
$ aws ec2 describe-vpc-peering-connections
```

HOW TO FIX

- ☑ Add correct route
- ☑ Assoc. route table to subnet
- ☑ Verify target (IGW/NAT/TGW)
- ☑ Check propagation settings
- ☑ Remove conflicting routes

AWS TROUBLESHOOTING WORKFLOW

1. IDENTIFY	2. ISOLATE	3. COLLECT EVIDENCE	4. ANALYZE	5. REMEDIATE	6. VALIDATE	7. DOCUMENT
Understand the symptoms & impact	Determine the affected service or component	Use AWS tools & logs to gather facts	Find the root cause using patterns	Apply the fix & verify the solution	Confirm service is healthy & stable	Document what happened & what changed

USEFUL AWS TOOLS

CloudWatch	Metrics, Logs, Alarms
CloudTrail	API activity & Audit logs
VPC Flow Logs	Network traffic visibility
AWS Config	Resource configuration history
Systems Manager	Run commands & collect logs
Trusted Advisor	Best practices & health checks

PRO TIP

Check the simple things first.
90% of outages are caused by misconfigurations.

SERVICE CHECKLIST & COMMON ROOT CAUSES

SERVICE	COMMON SYMPTOMS	TOP ROOT CAUSES	WHAT TO CHECK FIRST
EC2	Instance unreachable	SG rules, NACL, Route, Instance state	SG, Route table, Status checks
IAM	Access denied	Missing permissions, Wrong role	Policies, Trust relationship, MFA
VPC	No connectivity	Route tables, NACL, Endpoint	Routes, NACLs, Endpoints
SG	Connection timeout	Inbound rule missing	Inbound rules & source IPs
Route Table	Blackhole / No route	Missing route, Wrong target	Routes, Associations, Targets

COMMON ERROR MESSAGES

AuthFailure	Invalid credentials or permission denied
UnauthorizedOperation	User does not have required permissions
InvalidGroup.NotFound	Security Group ID does not exist
InvalidRoute.NotFound	Route table or route not found
InstanceLimitExceeded	Service quota or limit reached
RequestLimitExceeded	Too many requests (throttling)
DependencyViolation	Resource in use by dependent service
IncorrectState	Resource is in an invalid state for action

QUICK COMMAND CHEATSHEET

EC2

```
$ aws ec2 describe-instances
$ aws ec2 start-instances --instance-ids i-123
$ aws ec2 stop-instances --instance-ids i-123
$ aws ec2 get-console-output --instance-id i-123
```

IAM

```
$ aws sts get-caller-identity
$ aws iam list-users
$ aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123:user/NAME \
  --action-names s3:ListBucket ec2:DescribeInstances
```

VPC

```
$ aws ec2 describe-vpcs
$ aws ec2 describe-subnets
$ aws ec2 describe-network-acls
$ aws ec2 describe-internet-gateways
```

SECURITY GROUPS

```
$ aws ec2 describe-security-groups
$ aws ec2 authorize-security-group-ingress \
  --group-id sg-123 --protocol tcp --port 80 \
  --cidr 0.0.0.0/0
```

ROUTE TABLES

```
$ aws ec2 describe-route-tables
$ aws ec2 create-route \
  --route-table-id rtb-123 \
  --destination-cidr-block 0.0.0.0/0 \
  --gateway-id igw-123

$ aws ec2 associate-route-table \
  --subnet-id subnet-123 --route-table-id rtb-123
```

TROUBLESHOOTING REMINDERS

- ☑ Verify assumptions
- ☑ Check documentation
- ☑ Reproduce the issue
- ☑ Use logs and metrics
- ☑ Change one thing at a time
- ☑ Validate after every change
- ☑ Document everything

OBSERVE. ANALYZE. ACT. VERIFY. DOCUMENT. THAT'S HOW SENIOR ENGINEERS SOLVE PROBLEMS.

AWS MANAGED SERVICES

Use managed services. Focus on building, not maintaining.

Managed services remove undifferentiated operational burden.
Know how they work, how they fail, and how to fix them.

1. ALB (APPLICATION LOAD BALANCER)

- Layer 7 load balancer (HTTP/HTTPS)
- Routes to targets in multiple AZs
- Health checks ensure availability

COMMON ISSUES	HOW TO CHECK
☑ 5xx errors from targets	$ aws elbv2 describe-load-balancers
☑ Targets unhealthy	$ aws elbv2 describe-target-health
☑ Listener or rule misconfiguration	$ aws elbv2 describe-listeners
☑ SSL/TLS certificate issues	$ aws elbv2 describe-rules

HOW TO FIX

- ☑ Verify target health & SG rules
- ☑ Check listener rules & priorities
- ☑ Validate certificates (ACM)
- ☑ Check target response & logs

2. RDS (RELATIONAL DATABASE SERVICE)

- Managed MySQL, PostgreSQL, MariaDB, etc.
- Automated backups, patching, HA
- Multi-AZ for high availability

COMMON ISSUES	HOW TO CHECK
☑ Cannot connect / timeouts	$ aws rds describe-db-instances
☑ High CPU / memory	$ aws rds describe-db-parameters
☑ Storage full	$ aws rds describe-events
☑ Failover / replication issues	$ aws rds describe-db-log-files

HOW TO FIX

- ☑ Verify SG, subnet & endpoint
- ☑ Tune performance / scale
- ☑ Increase storage / clean up
- ☑ Review events & logs

3. ECS (ELASTIC CONTAINER SERVICE)

- Run containers without managing servers
- Supports Fargate and EC2 launch types
- Built-in service discovery & IAM integration

COMMON ISSUES	HOW TO CHECK
☑ Tasks not running / stopped	$ aws ecs list-clusters
☑ Cannot pull image	$ aws ecs list-services --cluster <c>
☑ Health check failing	$ aws ecs describe-services --cluster <c> --services <s>
☑ Service not stable	$ aws logs tail /aws/ecs/<task> --follow

HOW TO FIX

- ☑ Check task definition & events
- ☑ Verify IAM role & permissions
- ☑ Check SG, network, logs
- ☑ Increase desired count if needed

4. EKS (ELASTIC KUBERNETES SERVICE)

- Managed Kubernetes control plane
- Automatic version upgrades
- Integrated IAM, VPC, and add-ons

COMMON ISSUES	HOW TO CHECK
☑ Cluster / node not ready	$ aws eks describe-cluster --name <c>
☑ Pod scheduling failures	$ kubectl get nodes -o wide
☑ Network / CNI issues	$ kubectl get pods -A
☑ Auth / RBAC problems	$ kubectl describe node <node>

HOW TO FIX

- ☑ Check node group health
- ☑ Verify IAM roles (IRSA)
- ☑ Check CNI (VPC CNI) & SGs
- ☑ Review events & control plane logs

5. LAMBDA (SERVERLESS COMPUTE)

- Run code without provisioning servers
- Pay per request / per duration
- Integrates with 200+ AWS services

COMMON ISSUES	HOW TO CHECK
☑ Function errors / timeouts	$ aws lambda list-functions
☑ Permissions (IAM) denied	$ aws lambda get-function-configuration --function-name <fn>
☑ Cold starts / high latency	$ aws logs tail /aws/lambda/<fn> --follow
☑ Incorrect environment variables	$ aws lambda get-function-logs --function-name <fn>

HOW TO FIX

- ☑ Review CloudWatch logs
- ☑ Fix IAM role & policies
- ☑ Adjust memory / timeout
- ☑ Validate code & env vars

6. S3 (SIMPLE STORAGE SERVICE)

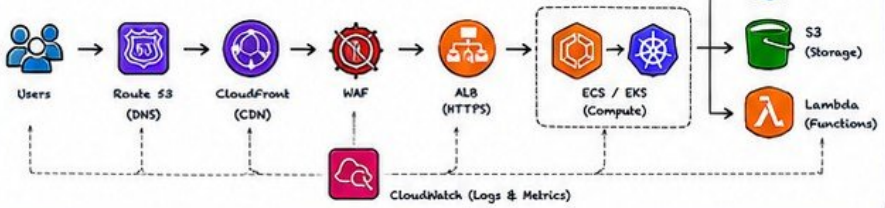
- Durable object storage (11 9s durability)
- Highly available & scalable
- Many storage classes & lifecycle rules

COMMON ISSUES	HOW TO CHECK
☑ Access denied	$ aws s3 ls s3://<bucket>
☑ Objects not visible	$ aws s3api head-bucket --bucket <b>
☑ Wrong region / endpoint	$ aws s3api get-bucket-policy --bucket <b>
☑ Slow uploads / downloads	$ aws s3api get-bucket-location --bucket <b>

HOW TO FIX

- ☑ Check bucket policy / IAM
- ☑ Verify region & endpoint
- ☑ Check ACL / Block Public Access
- ☑ Use proper storage class & lifecycle

TYPICAL MODERN APPLICATION ARCHITECTURE (USING MANAGED SERVICES)



MANAGED SERVICES HIGHLIGHTS

ALB	Smart routing, health checks, multi-AZ, SSL termination
RDS	Automated backups, failover, patching, monitoring
ECS	Container orchestration, auto scaling, rollouts
EKS	Managed control plane, secure, highly available
Lambda	Pay-per-use, auto scaling, event-driven compute
S3	99.999999999% durability, versioning, lifecycle rules

TROUBLESHOOTING WORKFLOW

1. **IDENTIFY** — Understand the symptoms & impact
2. **ISOLATE** — Narrow down the affected service or component
3. **INVESTIGATE** — Collect evidence using tools & logs
4. **ANALYZE** — Find the root cause
6. **VALIDATE** — 7. Ensure service is healthy

USEFUL AWS CLI SHORTCUTS

```
aws configure                              # Configure credentials
aws sts get-caller-identity                # Verify identity
aws ec2 describe-instances                 # List EC2 instances
aws rds describe-db-instances              # List RDS instances
aws ecs list-services --cluster <c>        # List ECS services
aws eks describe-cluster --name <c>        # Describe EKS cluster
aws s3 ls s3://<bucket>                    # List S3 bucket
aws cloudwatch get-metric-data             # Get metrics
aws logs tail /aws/lambda/<fn> --follow    # Tail Lambda logs
```

COMMON ROOT CAUSES

- ⚠ Misconfigured IAM permissions
- ⚠ Wrong network configuration (VPC/SG/Route)
- ⚠ Resource limits (CPU, Memory, Storage)
- ⚠ Health checks failing
- ⚠ Throttling / Service quotas
- ⚠ Expired certificates / credentials
- ⚠ Incorrect region or endpoint

PRO TIP

Read the service health first, then your logs, then metrics.

OPERATIONAL EXCELLENCE TIPS

- ☑ Use least-privilege IAM roles
- ☑ Enable CloudWatch alarms
- ☑ Centralize logs (CloudWatch Logs)
- ☑ Automate backups & snapshots
- ☑ Use auto scaling & multi-AZ
- ☑ Test disaster recovery
- ☑ Monitor costs & quotas
- ☑ Document runbooks

SENIOR INSIGHT

Managed does not mean hands-off. Design, monitor, and test always.

Microsoft Azure

AZURE TROUBLESHOOTING

Identify. Investigate. Resolve. Improve.

Great engineers don't guess. They investigate.
Follow the process, collect evidence, and fix the right thing.

1. VIRTUAL MACHINES (COMPUTE)

COMMON ISSUES
- VM not reachable
- High CPU / Memory
- Disk full
- Extensions failing
- Boot / OS issues

HOW TO CHECK
- ☑ Check VM status
- ☑ Check NSG rules
- ☑ Check extension status
- ☑ Check disks & availability
- ☑ Check boot diagnostics

HOW TO FIX
- ⌘ Restart or redeploy VM
- ⌘ Fix NSG / firewall rules
- ⌘ Extend disk / cleanup
- ⌘ Reinstall / update extensions
- ⌘ Use Serial Console if needed

2. VIRTUAL NETWORKS (NETWORKING)

COMMON ISSUES
- Cannot reach resources
- Subnet misconfiguration
- NSG / UDR blocking traffic
- DNS resolution failure
- VPN / Peering issues

HOW TO CHECK
- ☑ Check effective routes
- ☑ Check NSG flow logs
- ☑ Check DNS settings
- ☑ Check peering status
- ☑ Check connectivity

HOW TO FIX
- ⌘ Update routes / UDR
- ⌘ Fix NSG rules
- ⌘ Correct DNS servers
- ⌘ Re-establish peering / VPN
- ⌘ Test with Network Watcher

3. AKS (KUBERNETES SERVICE)

COMMON ISSUES
- Cluster unreachable
- Nodes NotReady
- Pods in CrashLoopBackOff
- Image pull / registry errors
- CNI / DNS issues

HOW TO CHECK
- ☑ Check node status
- ☑ Check kube-system pods
- ☑ Check events & logs
- ☑ Check CNI & CoreDNS
- ☑ Check cluster upgrades

HOW TO FIX
- ⌘ Restart failed pods
- ⌘ Fix node / capacity issues
- ⌘ Correct CNI / DNS configs
- ⌘ Update cluster / add-ons
- ⌘ Check container registry access

4. AZURE MONITOR (OBSERVABILITY)

COMMON ISSUES
- No metrics / logs
- Alerts not firing
- Data ingestion failure
- Workbook errors
- Diagnostic settings missing

HOW TO CHECK
- ☑ Check data sources
- ☑ Check diagnostic settings
- ☑ Check log ingestion status
- ☑ Check alert rules
- ☑ Check workspace health

HOW TO FIX
- ⌘ Enable diagnostic settings
- ⌘ Fix data source / AMA agent
- ⌘ Recreate alerts
- ⌘ Check workspace retention
- ☑ Validate permissions

5. LOAD BALANCER (TRAFFIC MANAGEMENT)

COMMON ISSUES
- Backend not reachable
- Health probe failing
- SNAT port exhaustion
- High latency / timeout
- Rule / Listener issues

HOW TO CHECK
- ☑ Check backend pool health
- ☑ Check health probes
- ☑ Check NSG / UDR
- ☑ Check SNAT connections
- ☑ Check listeners & rules

HOW TO FIX
- ⌘ Fix backend / probes
- ⌘ Update NSG / routes
- ⌘ Add outbound SNAT (NAT GW)
- ☑ Tune timeouts / rules
- ☑ Review session persistence

AZURE TROUBLESHOOTING PROCESS

1. **IDENTIFY** — Understand the symptoms & impact
2. **ISOLATE** — Narrow down the affected component
3. **COLLECT EVIDENCE** — Gather logs, metrics, and events
4. **ANALYZE** — Find the root cause
5. **REMEDIATE** — Apply the fix and validate
6. **VALIDATE** — Confirm service is healthy
7. **DOCUMENT** — Document what happened & what you fixed

ESSENTIAL AZURE TOOLS

Azure Portal	Visual management for all resources
Azure CLI	Command line management
Azure PowerShell	Automation & scripting
Azure Monitor	Logs, metrics, alerts, dashboards
Network Watcher	Network diagnostics & tests
Log Analytics	Centralized log analysis

PRO TIP

Check the basics first:
- Is the resource up?
- Can you reach it?
- Are logs showing errors?
- What changed recently?

COMMON AZURE CLI TROUBLESHOOTING COMMANDS

VIRTUAL MACHINES

```
az vm list --output table
az vm get-instance-view -g <rg> -n <vm>
az vm boot-diagnostics get-boot-log
  -g <rg> -n <vm>
az vm run-command invoke
  -g <rg> -n <vm> --command-id RunShellScript
  --scripts "df -h; free -m"
```

VIRTUAL NETWORKS

```
az network watcher test-connectivity
  -g <rg> --source-resource <sourceId>
  --dest-resource <destId> --protocol Tcp
  --port <port>
az network watcher show-effective-route-table
  -g <rg> --resource <nicId>
az network watcher show-next-hop
  -g <rg> --ip-address <ip>
```

AKS

```
az aks get-credentials
  -g <rg> -n <cluster>
az aks show -g <rg> -n <cluster>
kubectl get nodes -o wide
kubectl get pods -A
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns>
```

AZURE MONITOR

```
az monitor metrics list-definitions
  --resource <id>
az monitor metrics list
  --resource <id> --metric <name>
az monitor log-analytics query
  --workspace <id> --analytics-query
  "Heartbeat | summarize count() by
  bin(TimeGenerated, 5m)"
```

LOAD BALANCER

```
az network lb show -g <rg> -n <lb>
az network lb probe show
  -g <rg> --lb-name <lb> -n <probe>
az network lb show-backend-health
  -g <rg> --lb-name <lb>
az network watcher test-lb
  -g <rg> --lb-name <lb> --protocol Tcp
  --port <port>
```

QUICK CHECKLIST BY SERVICE

SERVICE	QUICK CHECKS
VM	Status, NSG, Extensions, Disk, Boot diagnostics
VNET	Routes, NSG, DNS, Peering, Connectivity
AKS	Nodes, Pods, Events, CNI, CoreDNS, Upgrades
MONITOR	Data sources, Diagnostic settings, Alerts, Logs
LB	Backend health, Probes, Rules, SNAT, NSG

COMMON ERROR MESSAGES

✕ VM not reachable	NSG / route / VM stopped
✕ Subnet not reachable	Route / UDR / NSG issue
✕ AKS nodes NotReady	Node pool / capacity / CNI
✕ No logs in workspace	Diagnostic settings off
✕ Backend unhealthy	Probe failing / App down
✕ SNAT port exhausted	Too many outbound connections

ESCALATION LEVELS

- **L1 – On-Call Engineer** — Initial triage & mitigation
- **L2 – Platform / DevOps** — Deep investigation & fix
- **L3 – Azure / Microsoft Support** — Escalate with evidence package

EVIDENCE TO COLLECT

- ☑ Resource IDs & Region
- ☑ Exact error messages
- ☑ Time of incident (UTC)
- ☑ Screenshots / Logs
- ☑ Steps to reproduce
- ☑ Impact / Business effect

OBSERVE. INVESTIGATE. UNDERSTAND. RESOLVE. VALIDATE. DOCUMENT.

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

Google Cloud

GOOGLE CLOUD TROUBLESHOOTING

REMEMBER
Observe.
Investigate.
Identify.
Resolve.
Validate.

Great engineers don't guess. They investigate, collect evidence, and fix the root cause.

1. COMPUTE ENGINE (VIRTUAL MACHINES)

COMMON ISSUES
- Instance not reachable
- High CPU / Memory
- Disk I/O issues
- Startup / Boot failures
- Metadata / Service issues

HOW TO CHECK
- ☑ Check instance status
- ☑ Check serial console output
- ☑ Check CPU / Memory metrics
- ☑ Check disk and IOPS
- ☑ Check VPC & firewall rules

HOW TO FIX
- ✖ Restart or stop / start instance
- ✖ Fix firewall rules / routes
- ✖ Resize instance / add disk
- ✖ Repair boot disk / image
- ✖ Use serial console for recovery

2. GKE (GOOGLE KUBERNETES ENGINE)

COMMON ISSUES
- Nodes NotReady
- Pods Pending / CrashLoopBackOff
- Image pull / registry errors
- DNS / CoreDNS issues
- Service / Ingress not working

HOW TO CHECK
- ☑ Check cluster / node status
- ☑ Check pod events & logs
- ☑ Check system pods
- ☑ Check network policies
- ☑ Check Ingress / Services

HOW TO FIX
- ✖ Repair or upgrade node pool
- ✖ Fix resource requests / limits
- ✖ Correct image / registry access
- ✖ Restart CoreDNS / metrics
- ✖ Fix Service / Ingress config

3. CLOUD LOAD BALANCING (NETWORKING)

COMMON ISSUES
- Backend unhealthy
- 5xx / 502 / 504 errors
- SSL / TLS handshake errors
- Health check failures
- Wrong routing / URL map

HOW TO CHECK
- ☑ Check backend health
- ☑ Check health check logs
- ☑ Check URL map / rules
- ☑ Check SSL certificate
- ☑ Check firewall rules

HOW TO FIX
- ✖ Fix app / backend issues
- ✖ Update health check config
- ✖ Correct URL map / backend svc
- ✖ Renew or fix SSL cert
- ✖ Allow LB health check IPs

4. CLOUD STORAGE (OBJECT STORAGE)

COMMON ISSUES
- Access denied
- Objects not visible
- Wrong region / location
- Slow uploads / downloads
- Lifecycle / retention issues

HOW TO CHECK
- ☑ Check bucket permissions
- ☑ Check object ACL / IAM
- ☑ Check bucket location
- ☑ Check request logs
- ☑ Check lifecycle policies

HOW TO FIX
- ✖ Fix IAM / ACL permissions
- ✖ Use correct bucket location
- ✖ Adjust lifecycle rules
- ✖ Enable / review logging
- ✖ Correct retention settings

5. IAM (IDENTITY & ACCESS MANAGEMENT)

COMMON ISSUES
- Permission denied
- Service account issues
- Roles not working
- API not enabled
- Cross-project access issues

HOW TO CHECK
- ☑ Check IAM policy bindings
- ☑ Check service account roles
- ☑ Check audit logs
- ☑ Check API status
- ☑ Check organization policy

HOW TO FIX
- ✖ Grant correct roles
- ✖ Fix service account scopes
- ✖ Enable required APIs
- ☑ Update policy bindings
- ☑ Adjust organization policy

GOOGLE CLOUD TROUBLESHOOTING WORKFLOW

1. **IDENTIFY** — Understand the symptoms & impact
2. **ISOLATE** — Narrow down the affected service or component
3. **COLLECT EVIDENCE** — Gather logs, metrics, events
4. **ANALYZE** — Find the root cause
5. **RESOLVE** — Apply the fix and validate
6. **VALIDATE** — Confirm service is healthy
7. **DOCUMENT** — Document what happened & what you fixed

USEFUL GCP TOOLS

Cloud Console	Web UI for all resources
gcloud CLI	Command line management
Cloud Logging	Logs, Audit, Events
Cloud Monitoring	Metrics, Alerts, Dashboards
Cloud Trace	Request latency analysis
Error Reporting	Application error tracking

PRO TIP

Always start with the basics:
- ☑ Is the resource up?
- ☑ Can it reach?
- ☑ Are firewall rules blocking?
- ☑ What changed recently?

ESSENTIAL GCP CLI COMMANDS (USE --HELP FOR MORE OPTIONS)

COMPUTE ENGINE
```
gcloud compute instances list
gcloud compute instances describe VM-NAME
gcloud compute ssh VM-NAME
gcloud compute start VM-NAME
gcloud compute reset VM-NAME
gcloud compute firewall-rules list
```

GKE
```
gcloud container clusters list
gcloud container clusters describe CLUSTER
gcloud container node-pools list --cluster CLUSTER
kubectl get nodes
kubectl get pods -A
kubectl describe pod POD-NAME -n NAMESPACE
```

CLOUD LOAD BALANCING
```
gcloud compute backend-services list
gcloud compute backend-services get-health BACKEND
gcloud compute health-checks list
gcloud compute url-maps list
gcloud compute target-proxies list
gcloud compute forwarding-rules list
```

CLOUD STORAGE
```
gsutil ls -b gs://BUCKET-NAME
gsutil ls gs://BUCKET-NAME
gsutil stat gs://BUCKET-NAME/OBJECT
gsutil acl get gs://BUCKET-NAME
gsutil mb -l LOCATION gs://BUCKET-NAME
gsutil cp file.txt gs://BUCKET-NAME/
```

IAM
```
gcloud projects get-iam-policy PROJECT-ID
gcloud iam service-accounts list
gcloud projects add-iam-policy-binding
PROJECT-ID
gcloud auth list
gcloud projects list
gcloud services list --enabled
```

COMMON ERROR MESSAGES

- ✖ PERMISSION_DENIED — You do not have required permissions.
- ✖ RESOURCE_NOT_FOUND — The resource does not exist.
- ✖ FAILED_PRECONDITION — The resource is not in a valid state.
- ✖ QUOTA_EXCEEDED — Usage exceeds your project quota.
- ✖ UNAVAILABLE — Service is temporarily unavailable.
- ✖ DEADLINE_EXCEEDED — Request timed out.

ROOT CAUSE CHECKLIST

- ☑ Recent changes / deployments
- ☑ Resource status & health
- ☑ Network connectivity
- ☑ IAM roles & permissions
- ☑ Quotas & limits
- ☑ Logs & audit trails
- ☑ Configuration correctness
- ☑ External dependencies

GCP SERVICES QUICK REFERENCE

Compute Engine	Virtual machines & scalable infrastructure
GKE	Managed Kubernetes for containers
Cloud Load Balancing	Global HTTP(S) & TCP/UDP load balancing
Cloud Storage	Durable & highly available object storage
IAM	Manage identities, access & permissions
Cloud VPC	Isolated private network in the cloud
Cloud Monitoring	Metrics, alerts & uptime monitoring
Cloud Logging	Centralized logging & analysis

EVIDENCE TO COLLECT

- ☑ Logs (Cloud Logging / Application)
- ☑ Metrics (Monitoring / Dashboards)
- ☑ Events (Audit Logs / Activity)
- ☑ Configuration / Settings
- ☑ Network traces / Connectivity
- ☑ Screenshots / Error messages
- ☑ Timeline of events

OBSERVE DEEPLY. INVESTIGATE SMARTLY. RESOLVE CORRECTLY. VALIDATE ALWAYS.

FOLLOW VERIQTA

YouTube Veriqta · GitHub Veriqta · LinkedIn Veriqta · X Veriqta · Instagram Veriqta · Telegram Veriqta

★ SOLVE PROBLEMS. DELIVER RELIABILITY. BUILD TRUST. ★

TERRAFORM TROUBLESHOOTING

Terraform

Troubleshoot Smart.
Fix the root cause.
Prevent repeat.

Great engineers don't guess.
They investigate, collect evidence, and fix the root cause.

1. STATE ISSUES (STATE FILE PROBLEMS)

COMMON ISSUES
- State file missing or corrupted
- Cannot find resource in state
- Remote state unreachable
- State file manually modified
- Backend permissions errors

HOW TO CHECK
- ☑ Verify backend configuration
- ☑ Check state file existence
- ☑ Validate backend access
- ☑ Review file permissions
- ☑ Run terraform state list

HOW TO FIX
- ⌘ Restore from backup
- ⌘ Reconfigure backend
- ⌘ Fix permissions
- ⌘ Import missing resources
- ⌘ Remove manual edits

2. DRIFT ISSUES (CONFIG VS REALITY)

COMMON ISSUES
- Resource changed outside Terraform
- Manual changes in console
- Missing resources in config
- Tags or settings drift
- Unexpected diffs in plan

HOW TO CHECK
- ☑ Run terraform plan
- ☑ Review drift in plan output
- ☑ Use terraform state show
- ☑ Compare tags & settings
- ☑ Check last applied changes

HOW TO FIX
- ⌘ Revert manual changes
- ⌘ Update configuration
- ⌘ Import existing resources
- ⌘ Run terraform apply
- ⌘ Enforce policy / guardrails

3. LOCKING ISSUES (STATE LOCK PROBLEMS)

COMMON ISSUES
- State lock already held
- State lock after crash
- Cannot acquire lock
- Concurrent operations
- Force unlock needed

HOW TO CHECK
- ☑ Review lock info in backend
- ☑ Check who holds the lock
- ☑ Check lock age / timestamp
- ☑ Validate no active runs
- ☑ Check CI/CD running jobs

HOW TO FIX
- ⌘ Wait for lock release
- ⌘ Kill stale process / job
- ⌘ Force unlock (carefully)
- ⌘ Improve locking process
- ⌘ Enable lock timeouts

4. PLAN FAILURES (PLAN PHASE ERRORS)

COMMON ISSUES
- Syntax errors
- Provider errors
- Authentication failures
- Data source failures
- Invalid reference / index
- Policy / validation failures

HOW TO CHECK
- ☑ Review error message
- ☑ Run terraform validate
- ☑ Check credentials
- ☑ Check provider version
- ☑ Enable TF_LOG=DEBUG

HOW TO FIX
- ⌘ Fix syntax / references
- ⌘ Update provider
- ⌘ Fix auth / permissions
- ⌘ Check data sources
- ⌘ Adjust policies / rules

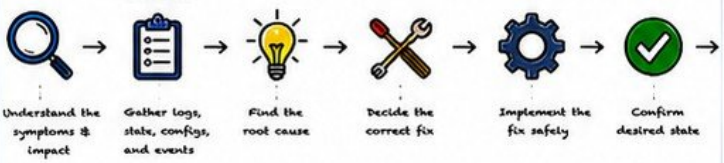
5. APPLY FAILURES (APPLY PHASE ERRORS)

COMMON ISSUES
- Resource creation failed
- Quota / limit exceeded
- Dependency errors
- Timeouts
- Partial apply / unknown state
- Permission denied

HOW TO CHECK
- ☑ Read full error output
- ☑ Check resource events (cloud)
- ☑ Verify dependencies
- ☑ Check quotas / limits
- ☑ Review recent changes

HOW TO FIX
- ⌘ Fix root cause
- ⌘ Adjust quotas / limits
- ⌘ Add explicit depends_on
- ⌘ Increase timeouts
- ⌘ Re-run apply safely

TERRAFORM TROUBLESHOOTING WORKFLOW



USEFUL TERRAFORM COMMANDS

terraform init	Initialize working directory
terraform validate	Validate configuration
terraform fmt	Format configuration files
terraform plan	Preview changes
terraform apply	Apply changes
terraform destroy	Destroy infrastructure
terraform state list	List resources in state
terraform state show <addr>	Show resource in state
terraform state rm <addr>	Remove from state
terraform import <addr> <id>	Import existing resource
terraform taint <addr>	Mark resource for recreation
terraform untaint <addr>	Remove taint
terraform providers	Show provider requirements
terraform graph	Generate dependency graph
terraform workspace list	List workspaces
terraform console	Interactive console

PRO TIP

Always run plan before apply.
- ☑ Automate state backups.
- ☑ Use remote state with locking.
- ☑ Review before you approve.
- ☑ Small changes, frequent applies.

STATE BACKEND OPTIONS

- S3 + DynamoDB (AWS)
- GCS + Cloud Storage (GCP)
- Azure Storage + Blob (Azure)
- HashiCorp Terraform Cloud
- Consul / HTTP Backend

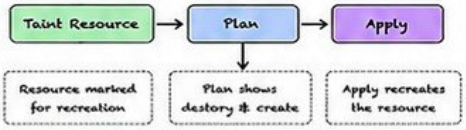
LOCKING MECHANISMS

- S3 + DynamoDB locking
- GCS locking
- Azure Blob Lease
- Terraform Cloud locking
- Consul sessions

DEBUGGING WITH TF_LOG

TF_LOG=ERROR	Show only errors
TF_LOG=WARN	Warnings and errors
TF_LOG=INFO	General information
TF_LOG=DEBUG	Detailed debugging
TF_LOG=TRACE	Very detailed (verbose)

RESOURCE TAINT & RECREATION



COMMON ERROR MESSAGES & MEANINGS

Error: Failed to acquire lock	State is locked by another process.
Error: Unsupported argument	Invalid argument in configuration.
Error: Reference to undeclared resource	Resource not defined or wrong name.
Error: Provider configuration not present	Missing or incorrect provider config.
Error: Error refreshing state	Cannot read resources from cloud.
Error: 403 / Permission denied	Insufficient permissions.
Error: Rate exceeded	API rate limit or quota exceeded.

ROOT CAUSE CHECKLIST

- ☑ Is state file accessible and valid?
- ☑ Are credentials and permissions correct?
- ☑ Is there drift or manual changes?
- ☑ Are provider versions compatible?
- ☑ Are quotas and limits sufficient?
- ☑ Are dependencies and references correct?
- ☑ Are network and connectivity working?
- ☑ Are policies or guardrails blocking?

BEST PRACTICES

- ☑ Store state remotely with locking
- ☑ Enable state versioning & backups
- ☑ Use workspaces for environments
- ☑ Enforce code reviews
- ☑ Automate plan in CI/CD
- ☑ Validate before apply
- ☑ Follow least privilege IAM
- ☑ Document everything

REMEMBER

- ☑ Investigate.
- ☑ Collect evidence.
- ☑ Identify root cause.
- ☑ Fix permanently.
- ☑ Automate to prevent repeat.

GOOD INFRASTRUCTURE IS WRITTEN ONCE. GREAT INFRASTRUCTURE IS TROUBLESHOOTED WELL. DOCUMENT. AUTOMATE. REPEAT.

LOGGING STRATEGY

Log with purpose. Correlate everything. Centralize, analyze, and act fast.

You can't fix what you can't see.
Good logging turns incidents into insights.

1. STRUCTURED LOGS
(CONSISTENT & MACHINE READABLE)

WHY IT MATTERS
- Easy to parse
- Searchable
- Supports automation
- Enables analytics

BEST PRACTICES
- ☑ Use JSON format
- ☑ Use consistent fields
- ☑ Use proper data types
- ☑ Avoid free text
- ☑ Include context

EXAMPLE (JSON LOG)

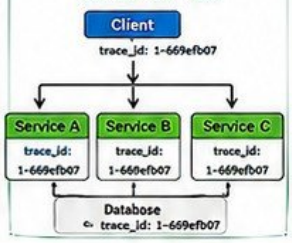
```
{
  "timestamp": "2025-07-27T10:15:30Z",
  "level": "ERROR",
  "service": "payment-service",
  "env": "prod",
  "message": "Payment failed",
  "trace_id": "1-669efb07-8b2f3f1a2d3c4e5f",
  "span_id": "8b3f3f1a2d3c4e5f",
  "user_id": "u12345",
  "amount": 199.99,
  "currency": "USD",
  "error_code": "PAY_402",
  "stack": "...."
}
```

2. CORRELATION IDS
(CONNECT THE DOTS)

WHY IT MATTERS
- Trace requests across microservices
- Faster root cause identification
- Essential for distributed systems

BEST PRACTICES
- ☑ Generate at request entry
- ☑ Propagate across services
- ☑ Log trace_id in every log
- ☑ Use W3C Trace Context
- ☑ Include span_id if available

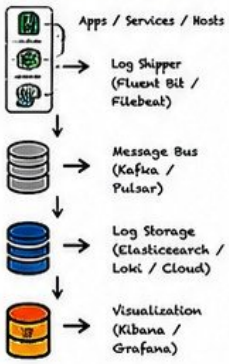
CORRELATION FLOW



3. CENTRALIZED LOGGING
(ONE PLACE, ALL LOGS)

WHY IT MATTERS
- Single source of truth
- Easier search & dashboards
- Better retention & security
- Faster incident response

COMMON ARCHITECTURES



4. LOG ANALYSIS
(FIND INSIGHTS FAST)

WHY IT MATTERS
- Detect issues early
- Understand trends
- Reduce MTTR
- Improve system reliability

KEY ANALYSIS AREAS
- ☑ Error patterns
- ☑ Latency spikes
- ☑ Top slow endpoints
- ☑ User / business impact
- ☑ Security & anomalies

EXAMPLE DASHBOARD PANELS

5. LOG RETENTION & GOVERNANCE
(KEEP WHAT MATTERS)

WHY IT MATTERS
- Meet compliance
- Control storage cost
- Ensure availability
- Support audits

BEST PRACTICES
- ☑ Define retention policy
- ☑ Archive old logs
- ☑ Encrypt logs in transit and at rest
- ☑ Restrict access (RBAC)
- ☑ Tag logs with metadata

RETENTION EXAMPLE
- Hot (7–15 days)
- Warm (15–90 days)
- Cold (90–365 days)
- Archive (1+ year)

END-TO-END LOGGING PIPELINE (EXAMPLE)

APPLICATION	AGENT / SHIPPER	TRANSPORT	STORAGE	ANALYSIS & VISUALIZATION	ALERTING & INTEGRATION
• App generates structured logs • Include trace_id • JSON format	• Collect logs • Add host metadata • Buffer & forward	• Reliable delivery • Kafka / Pub/Sub • Handle spikes	• Indexing • Compression • Replication	• Search & query • Dashboards • Alerts	• Alerts (PagerDuty, Slack) • Ticket creation • Automation

IMPORTANT LOG FIELDS
- ☑ timestamp
- ☑ level (DEBUG/INFO/WARN/ERROR)
- ☑ service
- ☑ environment
- ☑ message
- ☑ trace_id
- ☑ span_id
- ☑ user_id / account_id
- ☑ request_id
- ☑ source_ip
- ☑ error_code
- ☑ duration_ms

COMMON COMMANDS

VIEW LOGS IN REAL TIME

```
kubectl logs -f deployment/app -n prod
```

FILTER BY CONTAINER

```
kubectl logs -f pod/app-pod -c app -n prod
```

SEARCH LOGS (ELASTICSEARCH EXAMPLE)

```
GET /logs-*/_search
{
  "query": {
    "bool": {
      "must": [
        {"match": {"service": "payment-service"}},
        {"range": {"@timestamp": {"gte": "now-15m"}}},
        {"match": {"level": "ERROR"}}
      ]
    }
  },
  "sort": [{"@timestamp": "desc"}],
  "size": 50
}
```

LOG ANALYSIS QUERIES (EXAMPLES)

ERRORS OVER TIME

```
level:ERROR | stats count() by bin(5m)
```

TOP 10 ERROR MESSAGES

```
level:ERROR | stats count() by message | sort - count | head 10
```

SLOW REQUESTS (>2s)

```
duration_ms > 2000 | stats avg(duration_ms) by endpoint |
 sort - avg(duration_ms)
```

TOP SERVICES BY LATENCY

```
stats avg(duration_ms) by service | sort - avg(duration_ms)
```

USERS AFFECTED

```
level:ERROR | stats dc(user_id) as users_affected
```

TOOLING OPTIONS.

OPEN SOURCE
- ELK Stack (Elasticeearch, Logstash, Kibana)
- Grafana + Loki
- Promtail / Fluent Bit / Filebeat
- OpenSearch + Dashboards

CLOUD MANAGED

AWS	Azure	GCP
• CloudWatch Logs • OpenSearch Service • Kinesis Firehose • S3 Glacier	• Azure Monitor Logs • Log Analytics • Event Hubs • Storage Accounts	• Cloud Logging • BigQuery • Pub/Sub • Cloud Storage

SENIOR ENGINEER LOGGING CHECKLIST
- ☑ Logs are structured (JSON)
- ☑ Correlation IDs are implemented
- ☑ All services log at every level
- ☑ Logs include context fields
- ☑ Centralized logging is in place
- ☑ Retention policy is defined
- ☑ Access is restricted (RBAC)
- ☑ Logs are encrypted
- ☑ Dashboards cover key metrics
- ☑ Alerts are configured
- ☑ Runbooks reference log queries
- ☑ Incident playbooks use logs
- ☑ Log pipeline is monitored
- ☑ Log costs are optimized
- ☑ Regular review & cleanup
- ☑ Test logging during incidents

SENIOR PRINCIPLE

Good logging is not about collecting more logs.
It is about collecting the right logs and making them actionable.

LOG EVERYTHING. CORRELATE EVERYTHING. CENTRALIZE EVERYTHING. ANALYZE EVERYTHING. IMPROVE EVERYTHING.

FOLLOW VERIQTA
YouTube Veriqta
GitHub Veriqta
LinkedIn Veriqta
X Veriqta
Instagram Veriqta
Telegram Veriqta

BETTER LOGS. FASTER ANSWERS. STRONGER SYSTEMS.

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

METRICS TROUBLESHOOTING

METRICS

Good metrics tell you what is happening.
Great metrics help you understand why and what to do next.

Measure what matters.
Alert with intent.
Act before users feel it.

1. GOLDEN SIGNALS (WHAT TO WATCH)

- **Latency** — How fast is it?
- **Traffic** — How much is happening?
- **Errors** — What is failing?
- **Saturation** — Are we close to capacity?

WHY THEY MATTER

- ☑ Simple to understand
- ☑ Map to user experience
- ☑ Work at every layer
- ☑ Power fast decisions

2. SLIs (SERVICE LEVEL INDICATORS)

SLIs are quantitative measures of your service performance.

EXAMPLES

- Latency (p95) < 300ms
- Availability > 99.9%
- Error rate < 1%
- Success rate of checkout > 99%

GOOD SLI PRACTICES

- ☑ User-focused
- ☑ Measurable
- ☑ Clear & unambiguous
- ☑ Based on real traffic
- ☑ Owned by teams

3. SLOs (SERVICE LEVEL OBJECTIVES)

SLOs define the target for each SLI over a specific time window.

EXAMPLES

SLI	SLO TARGET	WINDOW
Availability	99.9%	30 days
Latency (p95)	< 300ms	7 days
Error Rate	< 1%	7 days
Success Rate	> 99%	30 days

SLO BEST PRACTICES

- ☑ Set based on data
- ☑ Challenging but achievable
- ☑ Reviewed regularly
- ☑ Aligned with business goals

4. ERROR BUDGETS (YOUR RISK ACCOUNT)

The error budget is the amount of unreliability you can spend.

FORMULA

Error Budget % = 100% - SLO Target %

EXAMPLE

SLO: 99.9% availability
Error Budget: 0.1%
In 30 days → 43.2 minutes downtime allowed

5. USING ERROR BUDGETS

IF BUDGET IS HEALTHY

- Ship new features
- Experiment
- Increase velocity

IF BUDGET IS LOW

- Slow down releases
- Improve reliability
- Fix issues

IF BUDGET IS EXHAUSTED

- Freeze changes
- Focus on recovery
- Communicate impact

The error budget drives engineering decisions.

METRICS TROUBLESHOOTING WORKFLOW

1. DETECT	2. IDENTIFY	3. ANALYZE	4. ISOLATE	5. RESOLVE	6. REVIEW
• Alert triggered • SLO at risk • User report	• What changed? • Which SLI? • Impact scope?	• Check golden signals • Drill down metrics • Correlate events	• Service / region • Dependency • Recent deploy?	• Fix / rollback • Validate metrics • Restore SLO	• Root cause • Action items • Prevent repeat

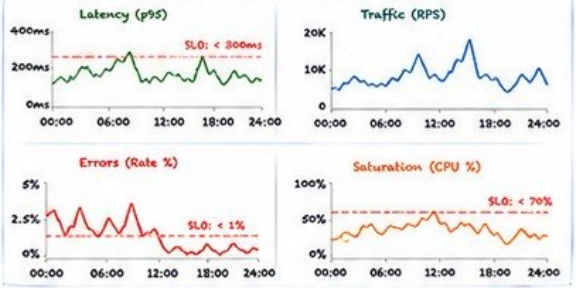
WHAT EACH GOLDEN SIGNAL TELLS YOU

SIGNAL	WHEN IT SPIKES	WHEN IT DROPS
Latency	Slowness, resource contention, dependency latency	Less traffic, brownout, partial outage
Traffic	Traffic spike, loops, retry storms	Traffic drop, outage, routing issue
Errors	Failures, exceptions, timeouts, bad deploys	Traffic drop, feature turned off
Saturation	High CPU / memory, queue build up	Under-utilization, scale down

PRACTICAL SLI EXAMPLES

Web API	Availability, p95 Latency, Error Rate, Success Rate
Checkout Service	Success Rate, Latency, Payment Error Rate
Database	Query Latency, Connections, Error Rate, CPU
Queue / Worker	Processing Latency, Queue Depth, Failure Rate
Frontend (Web)	Page Load Time (p95), JS Errors, Availability

EXAMPLE DASHBOARD (GOLDEN SIGNALS)



POPULAR METRICS TOOLS

Prometheus	Open source monitoring & alerting
Grafana	Dashboards & visualization
Datadog	Metrics, logs, traces, APM
CloudWatch	AWS native monitoring
Azure Monitor	Azure native monitoring
Google Cloud Monitoring	GCP native monitoring

USEFUL PROMQL QUERIES

```
# Error rate (5m)
sum(rate(http_requests_total{status=~"5.."}[5m]))
  /
sum(rate(http_requests_total[5m])) * 100

# p95 Latency (5m)
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket[5m]))
  by (le))

# CPU Utilization
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m]))
  * 100)
  by (instance)
```

ALERT EXAMPLES (GOOD ALERTS)

- ☑ High latency p95 > 300ms for 5m
- ☑ Error rate > 1% for 5m
- ☑ Availability < 99.9% (burn rate 14x)
- ☑ CPU utilization > 80% for 10m
- ☑ Queue depth > 1000 for 5m

Alert with context, runbook link, and dashboard link.

COMMON METRICS ANTI-PATTERNS

- ✗ Too many metrics, no clear purpose
- ✗ Alerts on symptoms, not causes
- ✗ No SLOs or undefined targets
- ✗ Ignoring saturation (until it's too late)
- ✗ Alerts with high noise
- ✗ No ownership for metrics
- ✗ "Set and forget" dashboards

PRO TIP

- ☑ Instrument for users, not for engineers.
- ☑ Review SLOs quarterly.
- ☑ Use error budgets to drive decisions.
- ☑ Automate and validate your telemetry.

METRICS MATURITY CHECKLIST

- ☑ Golden signals defined and monitored
- ☑ SLIs defined and documented
- ☑ SLOs set with clear targets
- ☑ Error budgets tracked
- ☑ Dashboards for insights (not just pretty charts)
- ☑ Alerts are actionable and low noise
- ☑ Post-incident review improves SLOs

FOLLOW VERIQTA

YouTube Veriqta · GitHub Veriqta · LinkedIn Veriqta · X Veriqta · Instagram Veriqta · Telegram Veriqta

★ MEASURE. UNDERSTAND. IMPROVE. DELIVER RELIABILITY. ★

Prometheus

PROMETHEUS TROUBLESHOOTING

Great monitoring isn't magic.
It's metrics scraped, stored, and alerted the right way.

Observe.
Investigate.
Understand.
Fix. Validate.

1. SCRAPING ISSUES (METRICS NOT COLLECTED)

COMMON CAUSES
- Wrong scrape_config
- Target down / timeout
- Firewall / network issue
- TLS / auth misconfig
- High scrape latency

HOW TO CHECK
- ☑ Check /targets page
- ☑ Check Prometheus logs
- ☑ Test target endpoint
- ☑ Verify relabel_configs
- ☑ Validate TLS settings

HOW TO FIX
- ✖ Fix job / metrics_path
- ✖ Open firewall / ports
- ✖ Increase scrape_timeout
- ✖ Fix TLS / auth
- ✖ Optimize target latency

2. TARGETS ISSUES (DOWN OR UNHEALTHY)

TARGET STATES
- UP — Target is healthy
- DOWN — Target unreachable
- UNKNOWN — Target status unknown

HOW TO CHECK
- ☑ Visit: /targets
- ☑ Check last scrape time
- ☑ Check error message
- ☑ Verify service discovery
- ☑ Test from Prometheus pod

COMMON FIXES
- ✖ Fix network / DNS
- ✖ Correct metrics endpoint
- ✖ Check service discovery
- ✖ Fix auth / TLS
- ✖ Increase timeouts

3. RECORDING RULE ISSUES (PRE-COMPUTED METRICS)

COMMON CAUSES
- Rule syntax error
- Missing metrics in query
- High cardinality explosion
- Expensive query
- Rule not loaded

HOW TO CHECK
- ☑ Check /rules page
- ☑ Check rule group status
- ☑ Check Prometheus logs
- ☑ Test expression manually
- ☑ Verify metrics existence

HOW TO FIX
- ✖ Fix query syntax
- ✖ Optimize / reduce cardinality
- ✖ Use rate() for counters
- ✖ Add missing metrics
- ✖ Reload / fix rule file

4. ALERT RULE ISSUES (ALERTS NOT FIRING)

COMMON CAUSES
- Alert expression wrong
- Labels / annotations wrong
- For duration too short/long
- Alertmanager not routing
- Silenced / inhibited alert

HOW TO CHECK
- ☑ Check /alerts page
- ☑ Check rule evaluation
- ☑ Check Alertmanager logs
- ☑ Check silences / inhibit rules
- ☑ Test alert expression

HOW TO FIX
- ✖ Fix alert expression
- ✖ Adjust "for" duration
- ✖ Fix labels / annotations
- ✖ Fix Alertmanager routing
- ✖ Remove silences if needed

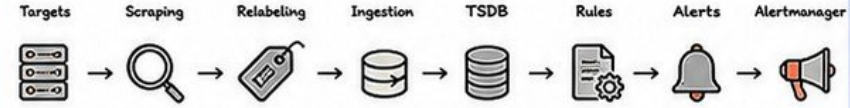
5. TSDB ISSUES (STORAGE PROBLEMS)

COMMON CAUSES
- Disk full
- High churn / cardinality
- Compactions taking long
- Corrupted blocks
- IO / performance issues

HOW TO CHECK
- ☑ Check TSDB status
- ☑ Check disk usage
- ☑ Check compaction time
- ☑ Check Prometheus logs
- ☑ Check cardinality

HOW TO FIX
- ✖ Free up disk space
- ✖ Reduce retention
- ✖ Optimize cardinality
- ✖ Repair / delete corrupt blocks
- ✖ Improve storage / IO

PROMETHEUS METRICS PIPELINE



Targets	Scraping	Relabeling	Ingestion	TSDB	Rules	Alerts	Alertmanager
• Apps	• HTTP	• Filter	• Validate	• Time Series	• Recording	• Firing	• Route
• Services	• HTTPS	• Transform	• Normalize	• Chunks	• Alerting	• Pending	• Group
• Exporters	• gRPC	• Drop / Keep	• Store	• Blocks		• Resolved	• Notify

/TARGETS PAGE EXPLAINED

JOB	INSTANCE	STATE	LAST SCRAPE	ERROR
node-exporter	10.0.0.10:9100	UP	2.134s ago	-
my-app	10.0.0.20:8080	DOWN	-	connect refused
db-exporter	10.0.0.30:9187	UP	1.452s ago	-

STATE MEANING

UP	Target is healthy and metrics are being scraped
DOWN	Prometheus cannot reach the target
UNKNOWN	Prometheus cannot determine the target status

USEFUL PROMQL FOR TROUBLESHOOTING

```
# Targets down
up == 0

# Targets by job
sum by(job)(up)

# High scrape duration
scrape_duration_seconds{job=~".*"}

# Scrape errors
rate(scrape_series_added{job=~".*"}[5m]) == 0
    up == 1

# Missing metrics (expected vs actual)
absent(up{job="my-app"})
```

RECORDING RULES EXAMPLE

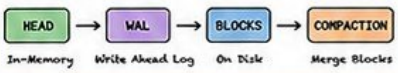
```
groups:
- name: service.rules
  interval: 30s
  rules:

  - record: job:http_requests:rate5m
    expr: sum(rate(http_requests_total[5m])) by (job)

  - record: job:cpu_usage:avg5m
    expr: avg(rate(process_cpu_seconds_total[5m]))
    by (job)
```

ALERT RULES EXAMPLE

```
groups:
- name: service.alerts
  rules:
  - alert: HighErrorRate
    expr: job:http_requests:rate5m /
      job:http_requests:rate5m offset 1h > 0.05
    for: 10m
    labels:
      severity: critical
    annotations:
      summary: "High error rate on {{ $labels.job }}"
      description: "Error rate > 5% for 10m"
```

TSDB STORAGE OVERVIEW

- Data stored in blocks (2h blocks)
- Head (in-memory) → WAL → Blocks
- Compaction merges blocks
- Retention deletes old blocks



TSDB HEALTH CHECKS

- ☑ Check disk space & inode usage
- ☑ Monitor compaction duration
- ☑ Monitor WAL size
- ☑ Monitor memory usage
- ☑ Monitor series cardinality
- ☑ Check for "out of order" samples
- ☑ Use Prometheus built-in metrics

IMPORTANT PROMETHEUS METRICS

up – Target health (1=up, 0=down)
scrape_duration_seconds – Scrape time
scrape_samples_scraped – Samples scraped
scrape_samples_post_metric_relabeling – After relabel
prometheus_tsdb_storage_blocks_bytes – Disk usage
prometheus_rule_evaluation_failures_total – Rule failures
prometheus_notifications_failed_total – Alert failures

PRO TIPS

- ☑ Keep scrape interval >= 10s for stability
- ☑ Avoid high cardinality (labels explode cost)
- ☑ Use recording rules for heavy queries
- ☑ Alert on monitoring (monitor your monitor)

GOOD METRICS. CLEAN PROMETHEUS. FAST ALERTS. HAPPY ENGINEERS.

FOLLOW VERIQTA

YouTube Veriqta · GitHub Veriqta · LinkedIn Veriqta · X Veriqta · Instagram Veriqta · Telegram Veriqta

MEASURE EVERYTHING. UNDERSTAND DEEPLY. IMPROVE CONTINUOUSLY.

GRAFANA TROUBLESHOOTING

Visualize truth.
Troubleshoot fast.
Make data useful.
Tell the right story.

Great dashboards don't fix bad data.
Understand. Diagnose. Visualize. Act.

1. DASHBOARDS ISSUES (VISUALIZATION PROBLEMS)

COMMON ISSUES
- Panel shows No data
- Wrong time range
- Panels not loading
- Missing / broken panels
- Incorrect panel type
- Overloaded dashboard

HOW TO CHECK
- ☑ Check time range
- ☑ Refresh dashboard
- ☑ Check panel query
- ☑ Validate data source
- ☑ Check panel settings
- ☑ Browser console (F12)

HOW TO FIX
- ✖ Set correct time range
- ✖ Fix query or filter
- ✖ Re-select data source
- ✖ Adjust interval
- ✖ Fix panel visualization
- ✖ Simplify heavy dashboards

2. QUERIES ISSUES (QUERY PROBLEMS)

COMMON ISSUES
- No data / Empty result
- Slow queries
- Wrong results
- Query timeout
- Interval mismatch
- High cardinality query

HOW TO CHECK
- ☑ Run query in Explore
- ☑ Check time range & interval
- ☑ Verify metric name & labels
- ☑ Check data source health
- ☑ Review query inspector
- ☑ Check for regex / typos

HOW TO FIX
- ✖ Correct metric / labels
- ✖ Adjust time range or interval
- ✖ Use filters / aggregation
- ✖ Add rate() for counters
- ✖ Optimize query
- ✖ Reduce cardinality

3. VARIABLES ISSUES (TEMPLATE VARIABLES)

COMMON ISSUES
- Variable not populating
- Wrong values
- Multi-value not working
- All option missing
- Variable not refreshing
- Chained variables failing

HOW TO CHECK
- ☑ Open dashboard variables
- ☑ Check variable query
- ☑ Verify data source
- ☑ Check regex / formatting
- ☑ Test in Explore
- ☑ Review variable settings

HOW TO FIX
- ✖ Fix variable query
- ✖ Add "All" option
- ✖ Enable Multi-value
- ✖ Set correct refresh
- ✖ Fix regex / capture groups
- ✖ Check dependency order

4. DATA SOURCES ISSUES (CONNECTIVITY & DATA)

COMMON ISSUES
- Data source down
- Authentication failed
- Permission denied
- SSL / TLS errors
- Wrong URL / endpoint
- Slow / timeout

HOW TO CHECK
- ☑ Test connection
- ☑ Check credentials
- ☑ Check URL & port
- ☑ Check SSL settings
- ☑ Review logs
- ☑ Check network access

HOW TO FIX
- ✖ Fix credentials / token
- ✖ Correct URL / endpoint
- ✖ Enable / fix SSL
- ✖ Update permissions
- ✖ Check firewall / SG
- ✖ Restart data source

5. ALERTING ISSUES (ALERT PROBLEMS)

COMMON ISSUES
- Alerts not firing
- Alerts firing too often
- No notifications
- Wrong thresholds
- Alert stuck firing
- Notification failures

HOW TO CHECK
- ☑ Check alert rule
- ☑ Check query & data
- ☑ Review alert state
- ☑ Check contact points
- ☑ Check notification logs
- ☑ Check silences / mute

HOW TO FIX
- ✖ Fix query / threshold
- ✖ Adjust evaluation interval
- ✖ Fix notification channel
- ✖ Check routing / policies
- ✖ Remove or adjust silences
- ✖ Test notification

GRAFANA TROUBLESHOOTING WORKFLOW

1. OBSERVE	2. IDENTIFY	3. ISOLATE	4. TEST	5. RESOLVE	6. VALIDATE
• What is wrong? • What changed? • Impact?	• Dashboard? • Query? • Data source? • Alert?	• Time range • Variables • Data source • Query	• Explore • Query inspector • Test connection	• Fix root cause • Apply change • Test again	• Verify data • Verify alerts • Confirm fix

USEFUL GRAFANA SHORTCUTS

R	Refresh dashboard
t	Toggle time picker
e	Edit panel
d	Duplicate panel
p	Panel menu
i	Inspect → Query
Ctrl+s	Save dashboard
Esc	Exit / Close

COMMON "NO DATA" CHECKLIST

- ☑ Correct time range?
- ☑ Data source working?
- ☑ Query returns data in Explore?
- ☑ Variables populated?
- ☑ Panel interval reasonable?
- ☑ Any filters hiding data?
- ☑ Permissions to data source?
- ☑ Network / firewall issues?

QUERY INSPECTOR (USE IT!)

```
{
  "datasource": "Prometheus",
  "queries": [
    {
      "expr": "rate(http_requests_total[5m])",
      "intervalMs": 15000,
      "legendFormat": "{{ method }} - {{ instance }}"
    }
  ],
  "range": {
    "from": "2025-07-27T10:00:00Z",
    "to": "2025-07-27T11:00:00Z",
    "raw":{ "from": "now-1h", "to": "now" }
  }
}
```

Use the inspector to see the exact query, response, timings and errors.

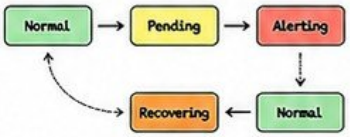
EXPLORE LIKE A PRO

1. Select the right data source
2. Set the same time range
3. Run the query
4. Add labels one by one
5. Use functions (rate, sum, avg)
6. Validate results
7. Save as panel

Example PromQL:

```
sum(rate(http_requests_total[5m])) by (instance)

topk(10, sum(rate(http_requests_total[5m])) by (path))
```

ALERT STATE FLOW



- Normal: Healthy
- Pending: Condition met, waiting for duration
- Alerting: Firing
- Recovering: Condition cleared, waiting to resolve

ALERT TESTING

- ▷ Test rule in Explore
- ▷ Use "Test rule" button
- ▷ Verify state transitions
- ▷ Check notification channel
- ▷ Check notification logs
- ▷ Ensure message is useful

Good alerts are:
- ☑ Actionable
- ☑ Specific
- ☑ Timely
- ☑ Noise-free

BEST PRACTICES

- ☑ Keep dashboards simple and focused
- ☑ Use consistent naming & units
- ☑ Use template variables wisely
- ☑ Limit panel queries per dashboard
- ☑ Record important metrics
- ☑ Test alerts regularly
- ☑ Document dashboards & alerts

PERFORMANCE TIPS

- ☑ Use $__interval and $__rate_interval
- ☑ Avoid high cardinality queries
- ☑ Use recording rules for heavy queries
- ☑ Limit dashboard auto refresh
- ☑ Use caching & CDN for Grafana
- ☑ Split large dashboards

COMMON ERROR MESSAGES

- ✖ No data → Check query, time range, data source
- ✖ Data source is not working → Test connection
- ✖ Error parsing query → Check syntax
- ✖ Permission denied → Check credentials
- ✖ Timeout → Check network / query performance
- ✖ Alert rule error → Check expression & labels

PRO TIP

Dashboards answer questions.
Alerts tell you when to act.
Together they protect your systems.

VISUALIZE CLEARLY. TROUBLESHOOT SMARTLY. ALERT ONLY WHAT MATTERS. DELIVER VALUE.

FOLLOW VERIQTA
YouTube Veriqta
GitHub Veriqta
LinkedIn Veriqta
X Veriqta
Instagram Veriqta
Telegram Veriqta

BETTER DASHBOARDS. BETTER INSIGHTS. FASTER TROUBLESHOOTING. STRONGER SYSTEMS.

DISTRIBUTED TRACING

Logs tell you WHAT happened.
Metrics show you WHEN it happened.
Traces show you WHY it happened.

Follow the request.
Connect the dots.
Find the root cause.
Fix what matters.

1. OPENTELEMETRY (STANDARD FOR TELEMETRY)

WHY IT MATTERS
- Vendor neutral standard
- Unified metrics, logs, traces
- Automatic instrumentation
- Ecosystem & portability

HOW TO CHECK
- ☑ Check OTel collector health
- ☑ Verify exporters configured
- ☑ Check service instrumentation
- ☑ Validate sampling settings
- ☑ Review resource attributes

HOW TO FIX
- ✖ Fix collector pipeline
- ✖ Correct exporter endpoint
- ✖ Instrument missing services
- ✖ Adjust sampling rate
- ✖ Add proper resource attrs

2. JAEGER (TRACE VISUALIZATION)

WHY IT MATTERS
- End-to-end trace visibility
- Service dependency graph
- Latency and error tracking
- Root cause identification

HOW TO CHECK
- ☑ Check Jaeger UI status
- ☑ Verify data is received
- ☑ Check sampling rate impact
- ☑ Validate storage backend
- ☑ Review retention/TTL

HOW TO FIX
- ✖ Ensure agents/collectors run
- ✖ Fix network to collector
- ✖ Increase sampling if needed
- ✖ Repair storage issues
- ✖ Tune retention policies

3. TEMPO (SCALABLE TRACE STORAGE)

WHY IT MATTERS
- Highly scalable storage
- Cost effective for traces
- Integrated with Grafana
- Built for observability scale

HOW TO CHECK
- ☑ Check Tempo ingesters
- ☑ Verify trace ingestion rate
- ☑ Check storage (S3/GCS)
- ☑ Review compactor status
- ☑ Validate retention config

HOW TO FIX
- ✖ Scale ingesters
- ✖ Fix object storage access
- ✖ Adjust retention period
- ✖ Resolve compaction issues
- ✖ Tune limits & quotas

4. TRACE ANALYSIS (DEEP INSIGHTS)

WHY IT MATTERS
- Find latency hotspots
- Identify failing components
- Understand dependencies
- Improve user experience

HOW TO CHECK
- ☑ Analyze latency breakdown
- ☑ Filter by errors/status
- ☑ Check slow spans
- ☑ Review service map
- ☑ Use tag/attribute filters

HOW TO FIX
- ✖ Optimize slow services
- ✖ Fix downstream failures
- ✖ Remove unnecessary calls
- ✖ Add timeouts & retries
- ✖ Improve resource usage

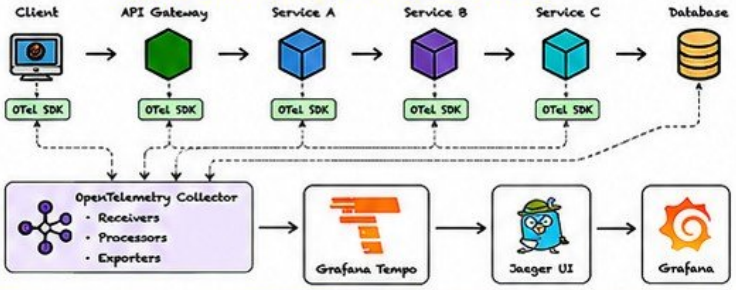
5. ROOT CAUSE ISOLATION (FIND THE REAL ISSUE)

WHY IT MATTERS
- Reduce MTTR
- Stop guessing
- Fix the right problem
- Prevent repeat incidents

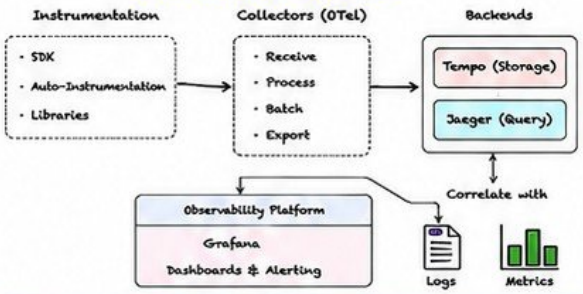
HOW TO CHECK
- ☑ Follow trace from start
- ☑ Identify first failing span
- ☑ Check error tags/events
- ☑ Correlate with logs/metrics
- ☑ Validate external deps

HOW TO FIX
- ✖ Fix failing component
- ✖ Improve error handling
- ✖ Add circuit breaker
- ✖ Set proper timeouts
- ✖ Add alerts & safeguards

DISTRIBUTED TRACING END-TO-END PIPELINE



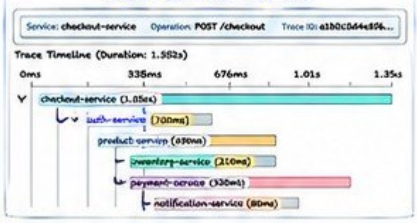
REFERENCE ARCHITECTURE



OPENTELEMETRY (OTEL) INSTRUMENTATION EXAMPLE (PYTHON)

```python
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter \
    import OTLPSpanExporter
from opentelemetry.sdk.trace.export import BatchSpanProcessor

trace.set_tracer_provider(TracerProvider())
tracer = trace.get_tracer(__name__)
otlp_exporter = OTLPSpanExporter(endpoint="http://otel-collector:4317")
trace.get_tracer_provider().add_span_processor(
    BatchSpanProcessor(otlp_exporter)
)
@tracer.start_as_current_span("process_order")
def process_order(order_id):
    with tracer.start_as_current_span("validate") as span:
        span.set_attribute("order.id", order_id)
        # validation logic
    with tracer.start_as_current_span("charge_payment"):
        # payment logic
    return "ok"
```

JAEGER UI - WHAT YOU SEE



Span Details (payment-service)

Key	Value
span.kind	client
http.method	POST
http.status_code	200
db.system	postgresql
net.peer.name	payment-db
error	false

Tags

payment.id	pay_12345
amount	99.99
currency	USD
region	us-east-1

TRACE ANALYSIS CHECKLIST

- ☑ Is latency high?
- ☑ Which span is slowest?
- ☑ Where does the time go?
- ☑ Any errors in the trace?
- ☑ Are there timeouts?
- ☑ External dependency issues?
- ☑ Excessive retries?
- ☑ Service instance unhealthy?
- ☑ Resource exhaustion?
- ☑ Is sampling too low?
- ☑ Are tags/attributes useful?
- ☑ Can we reproduce?

ROOT CAUSE ISOLATION WORKFLOW

USEFUL JAEGER / TEMPO QUERIES (GRAFANA EXPLORE)

```
# Find traces by service        service.name = "checkout-service"
# Find traces by operation      name = "POST /checkout"
# Error traces                  status.code = "ERROR"
# Slow traces (>500ms)          duration > 500ms
# Trace by ID                   traceID = "a1b2c3d4e5f6...."
```

BEST PRACTICES

- ☑ Use consistent naming
- ☑ Instrument all services
- ☑ Propagate context (trace IDs)
- ☑ Set appropriate sampling
- ☑ Add meaningful attributes
- ☑ Monitor collector & backend
- ☑ Correlate with logs & metrics
- ☑ Create useful dashboards
- ☑ Alert on SLO violations
- ☑ Review traces regularly

COMMON PITFALLS

- ✖ No instrumentation
- ✖ Sampling rate too low
- ✖ Missing context propagation
- ✖ Too many high-cardinality tags
- ✖ Storage retention too short

PRO TIP

A trace is a story of a request. Read the story from left to right. The first failing span is usually the root cause.

TRACE EVERYTHING. SEE THE FULL STORY. FIND THE ROOT CAUSE. FIX WHAT MATTERS. IMPROVE CONTINUOUSLY.

FOLLOW VERIQTA

YouTube Veriqta · GitHub Veriqta · LinkedIn Veriqta · X Veriqta · Instagram Veriqta · Telegram Veriqta

OBSERVE. CORRELATE. INVESTIGATE. UNDERSTAND. RESOLVE.

IAM & ACCESS

IAM AND ACCESS PROBLEMS

The right access. To the right people. For the right reason.
Secure by design. Least privilege by default.

Access is power.
Control it.
Audit it.
Protect it.

1. AUTHENTICATION (VERIFY WHO YOU ARE)

WHAT IT IS
Proving the identity of a user, service or workload.
Examples: passwords, MFA, tokens, certificates.

COMMON ISSUES
- Invalid credentials
- Expired tokens / keys
- MFA not working
- Clock skew
- Wrong identity provider

HOW TO CHECK
- ☑ Verify sign-in method
- ☑ Check MFA status
- ☑ Validate tokens / keys
- ☑ Check time sync (NTP)
- ☑ Review IdP / SSO logs

HOW TO FIX
- ✖ Reset credentials
- ✖ Re-sync time
- ✖ Re-issue tokens / keys
- ✖ Configure MFA correctly
- ✖ Check IdP configuration

2. AUTHERIZATION (WHAT YOU ARE ALLOWED TO DO)

WHAT IT IS
Determining what actions an identity is allowed to perform on which resources.

COMMON ISSUES
- Access denied errors
- Missing permissions
- Explicit deny
- Wrong policy attached
- Permissions boundary blocked

HOW TO CHECK
- ☑ Review attached policies
- ☑ Check explicit deny
- ☑ Validate resource ARN
- ☑ Simulate policy
- ☑ Review permission boundary

HOW TO FIX
- ✖ Attach correct policy
- ✖ Update policy rules
- ✖ Remove explicit deny
- ✖ Fix resource ARN
- ✖ Adjust permission boundary

3. RBAC (ROLE BASED ACCESS CONTROL)

WHAT IT IS
Grant permissions based on roles within a system (users, groups, service accounts).

COMMON ISSUES
- User not in correct role
- Role binding missing
- ClusterRole not applied
- Wrong namespace scope
- Group sync issues

HOW TO CHECK
- ☑ Review role / group mapping
- ☑ Check role bindings
- ☑ Verify scope (ns / cluster)
- ☑ Inspect service accounts
- ☑ Check group sync (IdP)

HOW TO FIX
- ✖ Add user to role
- ✖ Create / fix role binding
- ✖ Apply correct ClusterRole
- ✖ Set right namespace
- ✖ Sync groups from IdP

4. LEAST PRIVILEGE (ONLY WHAT'S NEEDED)

WHAT IT IS
Grant the minimum permissions required to perform a task. Nothing more.

COMMON ISSUES
- Overly permissive policies
- Admin access everywhere
- Wildcard permissions
- Shared accounts
- No access reviews

HOW TO CHECK
- ☑ Review policy scope
- ☑ Look for * (wildcards)
- ☑ Check last accessed data
- ☑ Review admin usage
- ☑ Verify resource scope

HOW TO FIX
- ✖ Remove unnecessary perms
- ✖ Scope down resources
- ✖ Avoid wildcards
- ✖ Use roles, not users
- ✖ Review access regularly

5. PERMISSION DEBUGGING (FIND THE REAL CAUSE)

WHAT IT IS
Systematically identifying why an access request is allowed or denied.

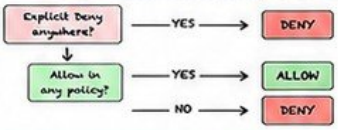
COMMON ISSUES
- Access denied with no clue
- Conflicting policies
- SCP / org policy blocked
- Resource policy deny
- Condition mismatch

HOW TO CHECK
- ☑ Read full error message
- ☑ Check CloudTrail / audit logs
- ☑ Simulate the request
- ☑ Review all policy types
- ☑ Check resource policies

HOW TO FIX
- ✖ Identify the deny
- ✖ Fix policy / condition
- ✖ Update resource policy
- ✖ Adjust SCP / org policy
- ✖ Validate and retest

ACCESS REQUEST FLOW

POLICY EVALUATION LOGIC (AWS EXAMPLE)



POLICY TYPES (CHECK IN THIS ORDER)

1. Organization SCPs
2. Permission Boundaries
3. Session Policies
4. Identity-based Policies
5. Resource-based Policies

An explicit DENY in any of the above overrides any ALLOW.

COMMON ACCESS DENIED ERRORS

- ✖ AccessDenied: User is not authorized to perform: s3:ListBucket
- ✖ UnauthorizedOperation: You are not authorized to perform this operation
- ✖ InsufficientPermissions: Missing required permissions
- ✖ Forbidden: Resource policy denies the request
- ✖ K8s Forbidden: User "system:anonymous" cannot get resource "pods"

USEFUL AWS COMMANDS

```
# Who am I?
aws sts get-caller-identity

# Simulate policy
aws iam simulate-principal-policy \
    --policy-source-arn arn:aws:iam::123456789012:user/John \
    --action-names s3:ListBucket ec2:StartInstances \
    --resource-arns arn:aws:s3:::my-bucket

# List attached policies
aws iam list-attached-user-policies --user-name John

# Get IAM policy
aws iam get-policy-version --policy-arn arn:aws:iam::123456789012:
policy/MyPolicy --version-id v1
```

USEFUL KUBECTL COMMANDS

```
# Can I do this?
kubectl auth can-i get pods
kubectl auth can-i create deployments -n my-ns

# Who am I?
kubectl auth whoami
kubectl auth whoami --show-groups

# Describe access
kubectl describe role my-role -n my-ns
kubectl describe rolebinding my-rb -n my-ns
```

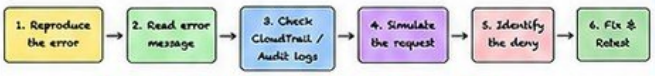
ACCESS REVIEW CHECKLIST

- ☑ Review admin users
- ☑ Remove unused access
- ☑ Check wildcard usage
- ☑ Review last accessed data
- ☑ Validate role & group mapping
- ☑ Check external access (federation)
- ☑ Review service accounts / keys
- ☑ Audit changes regularly

PRO TIP

- ☑ Grant access by role, not by user.
- ☑ Automate access reviews.
- ☑ Use just-in-time access where possible.
- ☑ Log, alert and investigate.

PERMISSION DEBUGGING WORKFLOW



Always find the DENY. That's the real problem.

BEST PRACTICES

- ☑ Use least privilege
- ☑ Use roles over users
- ☑ Enable MFA
- ☑ Rotate keys & tokens
- ☑ Centralize identity (SSO / IdP)
- ☑ Monitor & audit continuously
- ☑ Document access & ownership

COMMON PITFALLS

- ✖ Over-permissive admin access
- ✖ Sharing credentials
- ✖ Not reviewing access
- ✖ Hardcoded keys in apps
- ✖ Ignoring audit logs
- ✖ No separation of duties
- ✖ Access creep over time

REMEMBER

Security is not a product.
It's a practice.
Access is not permanent.
Review it.
Prove it.
Protect it.

LEAST PRIVILEGE. STRONG AUTHENTICATION. CLEAN POLICIES. CONTINUOUS AUDIT. ZERO TRUST.

SECRETS

SECRETS MANAGEMENT

Protect secrets like passwords, tokens and keys.
Store securely. Access safely. Rotate regularly.

No hardcode.
No plain text.
Least privilege.
Rotate always.

1. VAULT (ENTERPRISE SECRETS)

HashiCorp Vault

WHY IT MATTERS
- Centralized secrets store
- Dynamic secrets
- Fine-grained access
- Audit and versioning

HOW TO CHECK
- ☑ Vault server status
- ☑ Unseal status
- ☑ Auth method health
- ☑ Secret path access
- ☑ Token TTL and policies

HOW TO FIX
- ✖ Restart Vault if unhealthy
- ✖ Re-unseal with key shares
- ✖ Fix auth backend config
- ✖ Correct policy or role
- ✖ Renew or replace token

2. KUBERNETES SECRETS (IN-CLUSTER SECRETS)

WHY IT MATTERS
- Store secrets in cluster
- Mount as files or env
- RBAC controlled
- Integrated with workloads

HOW TO CHECK
- ☑ Secret exists in namespace
- ☑ Correct type (Opaque, TLS)
- ☑ Mounted in pod
- ☑ RBAC permissions
- ☑ Encoded (base64) OK

HOW TO FIX
- ✖ Recreate or update secret
- ✖ Fix serviceaccount access
- ✖ Mount using correct path
- ✖ Restart pod to reload
- ✖ Use External Secrets (if any)

3. AWS SECRETS MANAGER (MANAGED SECRETS)

WHY IT MATTERS
- Fully managed service
- Automatic rotation
- Fine-grained IAM
- Integrated with AWS services

HOW TO CHECK
- ☑ Secret exists
- ☑ Version stage (AWSCURRENT)
- ☑ KMS key access
- ☑ IAM permissions
- ☑ Rotation configuration

HOW TO FIX
- ✖ Update secret value
- ✖ Fix IAM policy
- ✖ Grant KMS decrypt
- ✖ Re-run rotation
- ✖ Check VPC endpoint access

4. ROTATION (KEEP SECRETS FRESH)

WHY IT MATTERS
- Limits blast radius
- Reduces long-term risk
- Meets compliance
- Protects from leaked creds

HOW TO CHECK
- ☑ Rotation enabled
- ☑ Last rotated time
- ☑ Next rotation time
- ☑ Rotation Lambda/Job health
- ☑ Secrets still valid

HOW TO FIX
- ✖ Enable rotation
- ✖ Fix rotation function
- ✖ Update interval
- ✖ Test rotation
- ✖ Revoke old credentials

5. ACCESS FAILURES (TROUBLESHOOT ACCESS)

WHY IT MATTERS
- Apps fail to start
- Security or config issue
- Expired or revoked access
- Misconfigured permissions

HOW TO CHECK
- ☑ Error message details
- ☑ Identity / ServiceAccount
- ☑ Role / Policy / IAM
- ☑ Network connectivity
- ☑ Secret path / name

HOW TO FIX
- ✖ Fix permissions
- ✖ Renew token / creds
- ✖ Update role bindings
- ✖ Open network / endpoint
- ✖ Correct secret reference

SECRETS MANAGEMENT ARCHITECTURE

SECRETS ACCESS FLOW (EXAMPLE)

1. Application requests secret
2. Authenticate identity
3. Authorize access (policy / role)
4. Retrieve secret from store
5. Decrypt (if encrypted)
6. Return secret to application
7. Log access and audit

KUBERNETES SECRETS EXAMPLE

```
# Create secret
kubectl create secret generic db-secret \
   --from-literal=username=veriqta_app \
   --from-literal=password='S3cureP@ss!'

# Use in pod (env)
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: db-secret
        key: password

# Decode secret
kubectl get secret db-secret -o yaml
# data is base64 encoded
```

VAULT QUICK CHECKS

```
# Vault status
vault status

# List secrets
vault kv list secret/

# Read secret
vault kv get secret/app/config

# Check token
vault token lookup
```

AWS SECRETS MANAGER COMMANDS (AWS CLI)

```
# Get secret
aws secretsmanager get-secret-value \
   --secret-id mysecret
# Describe secret
aws secretsmanager describe-secret \
   --secret-id mysecret
# List secrets
 aws secretsmanager list-secrets
# Rotate secret
aws secretsmanager rotate-secret \
   --secret-id mysecret
```

COMMON ACCESS FAILURE ERRORS

- ✖ AccessDeniedException
- ✖ User is not authorized to perform: secretsmanager:GetSecretValue
- ✖ permission denied
- ✖ Forbidden
- ✖ Secret not found
- ✖ Invalid token
- ✖ Vault is sealed
- ✖ Kubernetes secret not found

PERMISSION DEBUGGING CHECKLIST

- ☑ Identify the calling identity
- ☑ Check role / policy
- ☑ Validate resource ARN / path
- ☑ Check conditions (MFA, IP, VPC)
- ☑ Verify trust relationship
- ☑ Test with minimal policy
- ☑ Confirm network access
- ☑ Check service quotas / limits

BEST PRACTICES

- ☑ Never hardcode secrets
- ☑ Use least privilege access
- ☑ Enable audit logging
- ☑ Encrypt at rest and in transit
- ☑ Rotate secrets automatically
- ☑ Use short-lived credentials
- ☑ Monitor access and alerts
- ☑ Document secret owners

SECRETS ROTATION EXAMPLE (FLOW)

Automate rotation wherever possible.
Test rotation before enforcing.

EXAMPLE: VAULT DYNAMIC DB SECRET

```
# Enable database secrets engine
 vault secrets enable database

# Read dynamic credentials
 vault read database/creds/app-role'
# Response includes:
   username, password, ttl

# Vault will revoke automatically on TTL expiry
```

PRO TIP

- ★ Short TTL
- ★ Automatic rotation
- ★ Monitor access
- ★ Alert on failures
- ★ Secure everything

MANAGE SECRETS. REDUCE RISK. AUTOMATE ROTATION. BUILD TRUST.

FOLLOW VERIQTA

YouTube Veriqta · GitHub Veriqta · LinkedIn Veriqta · X Veriqta · Instagram Veriqta · Telegram Veriqta

★ SECURE SECRETS. PROTECT ACCESS. STAY AHEAD. ★

SECURITY

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

PRODUCTION SECURITY INCIDENTS

Detect early.
Respond fast.
Contain damage.
Recover stronger.

Security incidents are inevitable.
Preparation, detection, and response reduce damage.

1. UNAUTHORIZED ACCESS (ACCESS INCIDENT)

COMMON INDICATORS

- Login from unusual location
- Impossible travel
- Privilege escalation
- New admin/user created
- Access outside normal hours

WHAT TO CHECK

- ☑ IAM audit logs
- ☑ CloudTrail / Activity logs
- ☑ VPN / Bastion logs
- ☑ SSO / IdP logs
- ☑ API access logs

IMMEDIATE ACTIONS

- ✖ Disable compromised account
- ✖ Revoke active sessions
- ✖ Reset credentials
- ✖ Review roles/permissions
- ✖ Notify security team

2. MALWARE INDICATORS (MALICIOUS ACTIVITY)

COMMON INDICATORS

- Unexpected process
- Outbound connections to IPs
- File integrity changes
- High CPU / memory usage
- Suspicious scheduled tasks

WHAT TO CHECK

- ☑ EDR / AV alerts
- ☑ Process list & network
- ☑ DNS query logs
- ☑ File integrity monitoring
- ☑ System logs (Linux/Windows)

IMMEDIATE ACTIONS

- ✖ Isolate affected host
- ✖ Kill malicious process
- ✖ Block malicious IPs
- ✖ Capture forensic data
- ✖ Escalate to incident response

3. CREDENTIAL COMPROMISE (STOLEN / LEAKED CREDS)

COMMON INDICATORS

- Login failures spike
- Access from new device
- Password spray / brute force
- Secrets exposed in code/repos
- API key / token misuse

WHAT TO CHECK

- ☑ Auth logs (success/failure)
- ☑ Git / Secrets scanning
- ☑ Cloud provider audit logs
- ☑ Token / Key usage
- ☑ Third-party app access

IMMEDIATE ACTIONS

- ✖ Reset passwords / keys
- ✖ Invalidate tokens
- ✖ Revoke API keys
- ✖ Force MFA re-auth
- ✖ Check for lateral movement

4. RESPONSE WORKFLOW (INCIDENT LIFECYCLE)

KEY PRINCIPLE

Speed + Accuracy + Communication
minimize impact and recovery time.

SECURITY INCIDENT RESPONSE WORKFLOW (DETAILED)

DETECT	TRIAGE	CONTAIN	ERADICATE	RECOVER	REVIEW
• Alert triggered	• Validate alert	• Isolate systems	• Remove threat	• Restore services	• Root cause
• Monitor / SIEM	• Assess severity	• Block access	• Close gaps	• Monitor closely	• Lessons learned
• User report	• Identify scope	• Preserve evidence	• Patch systems	• Validaat integrity	• Improve controls

COMMUNICATION PLAN

- Notify right people fast
- Keep stakeholders updated
- Use clear, simple language
- Document all actions

LOGS & EVIDENCE TO COLLECT

- CudTrail / Activity Logs
- System & Auth Logs
- Network Flow Logs
- Endpoint / EDR Logs
- DNS / Proxy Logs
- Application Logs
- Firewall / WAF Logs
- Screenshots / Timeline

Preserve evidence before making changes.
Maintain chain of custody.

COMMON ATTACK VECTORS

- Phishing / Social Engineering
- Weak / Stolen Credentials
- Unpatched Vulnerabilities
- Misconfigured Services
- Exposed Secrets / Keys
- Malicious Insider
- Third-Party Compromise
- Supply Chain Attacks

EXAMPLE: UNAUTHORIZED ACCESS FLOW

LOG EXAMPLE (CLOUDTRAIL)

```
{
  "eventTime": "2025-07-27T10:15:322",
  "eventName": "ConsoleLogin",
  "userIdentity": {"type": "IAMUser", "userName": "alice"},
  "sourceIPaddress": "203.0.113.45",
  "userAgent": "Mozilla/5.0",
  "errorCode": "-"
}
```

Look for: unusual IP, new device, impossible travel.

QUICK CHECKLIST

- ☑ Monitor alerts & dashboards
- ☑ Enforce MFA everywhere
- ☑ Rotate credentials regularly
- ☑ Use least privilege access
- ☑ Scan for exposed secrets
- ☑ Patch systems on time
- ☑ Backup & test recovery
- ☑ Run incident drills

Security is not a product.
It's a process.
Stay sharp. Stay secure.

BEST PRACTICES

- ☑ Assume breach mindset
- ☑ Detect early, respond fast
- ☑ Automate response where possible
- ☑ Principle of least privilege
- ☑ Encrypt data in transit & at rest
- ☑ Regular security assessments
- ☑ Train your team
- ☑ Improve continuously

USEFUL TOOLS

- SIEM: Splunk, Elastic, Sentinel
- EDR: CrowdStrike, Defender, Falco
- IDS/IPS: Suricata, Snort
- Cloud: GuardDuty, Security Center
- Secrets: Vault, AWS SM, K8s Secrets
- Scanners: Trivy, Snyk, Nessus
- WAF: Cloudflare, AWS WAF

INCIDENT SEVERITY GUIDE

- 🔴 CRITICAL - Major outage / data breach
- 🟠 HIGH - Potential data exposure
- 🟡 MEDIUM - Limited impact
- 🟢 LOW - Minimal risk

Classify quickly.
Respond appropriately.

REMEMBER

Strong prevention is good.
Fast detection is better.
Great response is best.

Security is everyone's responsibility.

DETECT. CONTAIN. ERADICATE. RECOVER. LEARN. IMPROVE.

FOLLOW VERIQTA

YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

★ SECURE SYSTEMS. PROTECT DATA. BUILD TRUST. ★

INCIDENT

MAJOR INCIDENT RESPONSE

When systems fail, how you respond defines your reliability.
A fast, coordinated response minimizes impact and builds trust.

Be prepared.
Communicate.
Decide fast.
Recover safely.

1. ROLES & RESPONSIBILITIES

KEY ROLES

- Incident Commander (IC)
- Operations Lead
- Communications Lead
- Technical Lead
- Scribe / Note Taker
- Stakeholder Liaison

WHAT TO CHECK

- ☑ Roles are defined
- ☑ On-call is reachable
- ☑ Escalation paths clear
- ☑ Runbooks are updated
- ☑ Backups & contacts ready

BEST PRACTICE

One incident.
One commander.
No confusion.

2. COMMUNICATION

KEY ACTIONS

- Open incident channel
- Notify the right people
- Share updates regularly
- Use clear, simple language
- Document everything

COMMUNICATION PLAN

- ☑ Incident channel (Slack/Teams)
- ☑ Status update frequency
- ☑ Stakeholder list (internal)
- ☑ Customer communication plan
- ☑ Post-incident report

BEST PRACTICE

Communicate early.
Communicate often.
Over-communicate.

3. SEVERITY LEVELS

SEVERITY GUIDE

LEVEL	NAME	IMPACT	RESPONSE
P1	CRITICAL	Widespread	Immediate
P2	HIGH	Major impact	< 30 min
P3	MEDIUM	Partial impact	< 2 hrs
P4	LOW	Minor impact	< 8 hrs

WHAT TO CHECK

- ☑ Severity reflects user impact
- ☑ Clear impact description
- ☑ Response time targets set
- ☑ Escalation thresholds defined
- ☑ Reviewed after each incident

4. ESCALATION

ESCALATION FLOW

1. Detect & acknowledge
2. Engage responder
3. Escalate to leadership
4. Engage specialists
5. Executive escalation (if needed)

WHAT TO CHECK

- ☑ Escalation matrix exists
- ☑ Contacts are up to date
- ☑ Time to escalate is short
- ☑ Clear decision owners
- ☑ Escalation is documented

5. DECISION MAKING

DECISION PRINCIPLES

- Act on facts, not assumptions
- Focus on impact reduction
- Use data and telemetry
- Choose reversible actions first
- Document decisions & reasons

WHAT TO CHECK

- ☑ Runbooks guide decisions
- ☑ Blast radius understood
- ☑ Risks vs benefits evaluated
- ☑ Rollback plan ready
- ☑ Decisions communicated

BEST PRACTICE

Good decisions
Reduce MTTR.
Great decisions
Prevent recurrence.

MAJOR INCIDENT RESPONSE WORKFLOW

1. DETECT	2. ACKNOWLEDGE	3. TRIAGE	4. CONTAIN	5. RESOLVE	6. RECOVER	7. REVIEW
• Alerts trigger	• Validate the alert	• Identify impact	• Stop the spread	• Fix the root cause	• Monitor closely	• Post-incident review
• User reports	• Create incident	• Collect data	• Isolate systems	• Verify the fix	• Scale & stabilize	• Root cause analysis
• Synthetic checks	• Assign severity	• Identify scope	• Mitigate impact	• Run validations	• Inform stakeholders	• Lessons learned
• Anomaly detected	• Notify channel	• Find initial cause	• Workaround if possible	• Restore services	• Return to normal	• Update runbooks

INCIDENT COMMANDER CHECKLIST

- ☑ Declare incident
- ☑ Assess severity
- ☑ Assign roles
- ☑ Set communication cadence
- ☑ Validate impact
- ☑ Approve major actions
- ☑ Escalate when needed
- ☑ Close incident
- ☑ Approve post-incident review

SEVERITY LEVEL EXAMPLES

P1 - CRITICAL (Immediate)
System down for most users.
Revenue loss / Data at risk.
All hands on deck.

P2 - HIGH (<30 min)
Major features unavailable.
Workarounds exist.
High user impact.

P3 - MEDIUM (<2 hrs)
Partial degradation.
Some users impacted.
Monitor closely.

P4 - LOW (<8 hrs)
Minor issues / cosmetic.
Low user impact.
Standard process.

COMMUNICATION TEMPLATES (EXAMPLES)

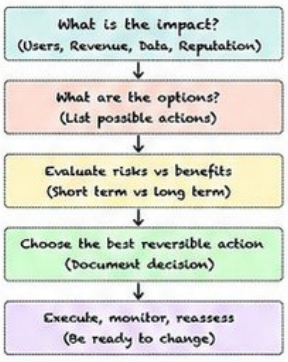
INITIAL UPDATE (Within 15 minutes)
"We are investigating an issue affecting {system}.
Impact: {describe}. We will update again at {time}."

REGULAR UPDATE
"Current status: {update}. Impact: {impact}.
Next update at {time} or sooner if major changes occur."

RESOLUTION UPDATE
"The issue has been resolved. We are monitoring the system closely. Thank you for your patience."

POST-INCIDENT SUMMARY
"Root cause: {cause}. Actions taken: {actions}.
Preventive actions: {next steps}. Full report to follow."

DECISION MAKING FRAMEWORK



KEY METRICS TO TRACK

- MTTA - Mean Time To Acknowledge
- MTTR - Mean Time To Resolve
- MTTC - Mean Time To Communicate
- Change Failure Rate
- Incident Frequency
- Customer Impact (Users affected)
- Post-incident Action Completion

"INCIDENTS WILL HAPPEN. PREPARATION DECIDES THE OUTCOME." BE CALM. BE CLEAR. LEAD. SOLVE. IMPROVE.

FOLLOW VERIQTA

YouTube Veriqta · GitHub Veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram Veriqta

PREPARE NOW. RESPOND FAST. RECOVER STRONGER.

RCA

ROOT CAUSE ANALYSIS

Find the real cause. Fix it right. Prevent it forever.
RCA turns incidents into learning and reliability.

Find the root.
Fix the system.
Prevent recurrence.
Drive reliability.

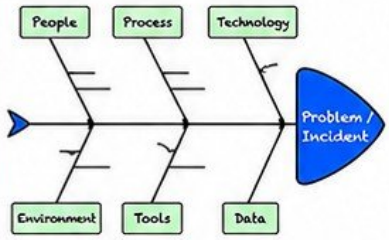
1. FIVE WHYS (ITERATIVE QUESTIONING)

- Why #1 — Why did this happen?
- Why #2 — Why did that happen?
- Why #3 — Why did that happen?
- Why #4 — Why did that happen?
- Why #5 — Why did that happen?

KEY POINTS

- Keep asking why.
- Go beyond symptoms.
- Stop at the root cause.
- Validate the cause.

2. FISHBONE DIAGRAM (CAUSE CATEGORIZATION)



COMMON CATEGORIES

- People
- Process
- Technology
- Environment
- Tools
- Data

USE FOR

- ☑ Complex incidents with multiple factors
- ☑ Team brainstorming
- ☑ Visualizing cause relationships

3. TIMELINE ANALYSIS (EVENT SEQUENCE)

09:00	Deployment started
09:05	New version enabled for 10% traffic
09:07	Error rate increased
09:09	Alerts triggered
09:10	Auto rollback started
09:14	Rollback completed
09:20	Service recovered

KEY POINTS

- Build accurate timeline
- Include all relevant events
- Use logs, metrics, and alerts
- Identify the turning point

4. CORRECTIVE ACTIONS (FIX & PREVENT)

IMMEDIATE FIX
Resolve the issue and restore service.

CORRECTIVE ACTION
Fix the root cause permanently.

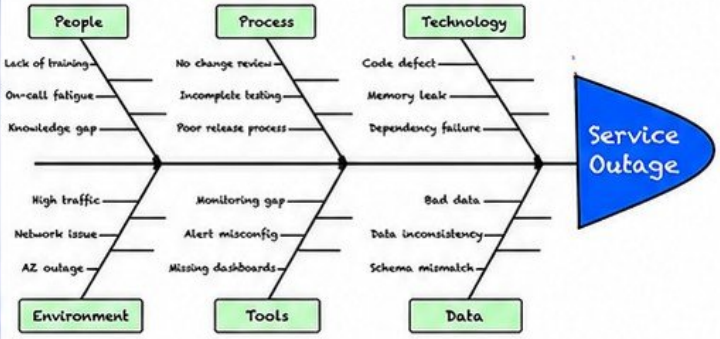
PREVENTIVE ACTION
Improve systems to prevent recurrence.

FOLLOW UP
Verify effectiveness and monitor.

KEY PRINCIPLE

Fix the cause, not the symptom.
Improve the system, not just today.

FISHBONE DIAGRAM EXAMPLE (WEB SERVICE OUTAGE)



FIVE WHYS EXAMPLE

Why #1	Why did the service crash?	→ Because the application ran out of memory.
Why #2	Why did it run out of memory?	→ Because of a memory leak in the caching module.
Why #3	Why was there a memory leak?	→ Because objects were not released after cache expiry.
Why #4	Why were objects not released?	→ Because the eviction logic was missing in edge cases.
Why #5	Why was the logic missing?	→ Because test cases did not cover those scenarios.

ROOT CAUSE: Incomplete test coverage allowed a memory leak in edge cases leading to service crash.

TIMELINE ANALYSIS TEMPLATE

TIME (UTC)	EVENT	SOURCE	IMPACT
09:00	Deployment started	CI/CD Pipeline	None
09:05	New version 1.4.2	Kubernetes	10% traffic
09:07	Error rate spiked	Prometheus	Users failing
09:09	High latency alert	Grafana Alert	Degraded
09:10	Auto rollback started	Argo Rollouts	Mitigating
09:14	Rollback completed	Argo Rollouts	Recovering
09:20	Service healthy	Synthetic Check	Resolved

KEY POINTS

- Use single source of truth for time (UTC).
- Correlate logs, metrics, and alerts.
- Identify the turning point.

RCA REPORT TEMPLATE

1	Incident Summary	What happened, when, impact.
2	Detection	How it was detected.
3	Timeline	Key events in order.
4	Impact	Users, services, revenue, SLAs.
5	Root Cause	The underlying cause.
6	Contributing Factors	What made it worse.
7	Corrective Actions	Fix the root cause.
8	Preventive Actions	Prevent recurrence.

QUALITY CHECK

- ☑ Evidence backed
- ☑ Specific and factual
- ☑ Action oriented
- ☑ No blame or assumptions
- ☑ Clear owners and dates
- ☑ Follow up and verify

ACTION TRACKING

ACTION	OWNER	PRIORITY	DUE DATE	STATUS
Fix memory leak	Dev Team	P0	2025-07-30	Done
Add missing tests	QA Team	P1	2025-08-02	In Progress
Improve monitoring	SRE Team	P1	2025-08-05	Planned
Review release process	Platform Team	P2	2025-08-10	Planned
Postmortem follow up	Incident Lead	P2	2025-08-15	Pending

BEST PRACTICES

- ☑ Focus on systems, not people.
- ☑ Collect facts, not opinions.
- ☑ RCA is a team activity.
- ☑ Document actions and follow through.
- ☑ Measure improvement over time.

RCA IS NOT ABOUT BLAME. IT IS ABOUT LEARNING. IT IS ABOUT IMPROVING. IT IS ABOUT RELIABILITY.

REVIEW

POST-INCIDENT REVIEWS

We don't waste incidents. We learn from them.
Every incident is an opportunity to build a better, more reliable system.

Learn together.
Share openly.
Improve always.
Build reliability.

1. BLAMELESS CULTURE
(FOCUS ON SYSTEMS, NOT PEOPLE)

WHY IT MATTERS
- Encourages honesty
- Surfaces real issues
- Reduces fear and hiding
- Builds trust and teamwork
- Drives real improvements

HOW TO PRACTICE
- ☑ Assume good intent
- ☑ Focus on process and systems
- ☑ Avoid blame and finger pointing
- ☑ Thank people for reporting
- ☑ Celebrate learning

KEY PRINCIPLE
Blame individuals for actions, not for outcomes. Fix the system.

2. ACTION ITEMS
(TURN INSIGHTS INTO ACTION)

WHAT TO IDENTIFY
- What went wrong
- Why it happened
- What worked well
- What needs to change
- Who will do what, by when

ACTION ITEM TEMPLATE

Action	What will be done?
Owner	Who is responsible?
Priority	P0 / P1 / P2 / P3
Due Date	When will it be done?
Status	Open / In Progress / Done

KEY PRINCIPLE
Track to closure. Done > Documented.

3. KNOWLEDGE SHARING
(MAKE LEARNING VISIBLE)

HOW TO SHARE
- Write clear post-incident report
- Share in team channels
- Update runbooks and docs
- Create training or brown bags
- Share metrics and trends

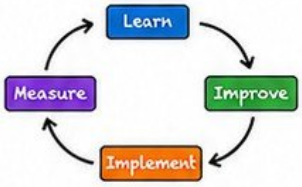
SHARE CHECKLIST
- ☑ Incident summary
- ☑ Timeline
- ☑ Root cause
- ☑ Impact
- ☑ What we learned
- ☑ Actions and owners
- ☑ Preventive changes

KEY PRINCIPLE
If we don't share it, we will repeat it.

4. CONTINUOUS IMPROVEMENT
(GET BETTER EVERY TIME)

IMPROVEMENT AREAS
- Monitoring and alerting
- Process and automation
- Architecture and design
- Tools and observability
- People and skills

CONTINUOUS IMPROVEMENT LOOP



KEY PRINCIPLE
Small improvements, compounded over time, create resilience.

POST-INCIDENT REVIEW PROCESS

1. **PREPARE**
   - Gather data
   - Review alerts
   - Invite right people
   - Set agenda
2. **REVIEW**
   - Walk through timeline
   - Discuss impact
   - Identify root cause
   - What went well?
   - What didn't?
3. **IDENTIFY ACTIONS**
   - Define corrective actions
   - Define preventive actions
   - Assign owners
   - Set priorities
   - Set due dates
4. **DOCUMENT**
   - Write clear report
   - Include evidence
   - Record decisions
   - Update runbooks
5. **SHARE**
   - Share with team
   - Share key learnings
   - Update knowledge base
   - Create short summary
6. **TRACK & CLOSE**
   - Track action items
   - Review progress
   - Close actions
   - Verify effectiveness

POST-INCIDENT REPORT OUTLINE
- ☑ Incident Summary
- ☑ Impact
- ☑ Timeline
- ☑ Root Cause Analysis
- ☑ What Went Well
- ☑ What Didn't Go Well
- ☑ Corrective Actions
- ☑ Preventive Actions
- ☑ Action Items
- ☑ Lessons Learned
- ☑ Appendices / Evidence

EXAMPLE: INCIDENT TIMELINE

09:00	Alert triggered (High error rate)
09:02	On-call acknowledged
09:05	Triage started
09:12	Root cause identified
09:18	Mitigation applied
09:25	Service recovered
09:40	Monitoring normal
10:15	Incident declared resolved
11:00	Post-incident review scheduled

ACTION PRIORITY GUIDE

P0	CRITICAL	Must fix immediately.
P1	HIGH	Significant risk / impact.
P2	MEDIUM	Important but not urgent.
P3	LOW	Nice to have / improvements.

ACTION TRACKING BOARD (EXAMPLE)

ID	ACTION	OWNER	PRIORITY	DUE DATE	STATUS
A-01	Add alert for queue depth	Dev Team	P1	2025-07-28	In Progress
A-02	Increase DB connection pool	Platform Team	P0	2025-07-26	Done
A-03	Update runbook	SRE Team	P2	2025-08-02	Open
A-04	Add integration tests	QA Team	P2	2025-08-05	Open

BLAMELESS LANGUAGE EXAMPLES

INSTEAD OF THIS	SAY THIS INSTEAD
✗ Who broke this?	✓ What happened?
✗ Why did you do that?	✓ What caused this?
✗ This is your fault.	✓ What can we learn?
✗ You should have known.	✓ How can we improve?
✗ We need to blame someone.	✓ What will we do differently?

REMEMBER
- Incidents are inevitable.
- Improvement is optional.
- A strong team learns, adapts, and prevents recurrence.

LEARN. SHARE. IMPROVE. REPEAT.

EVERY INCIDENT IS A LESSON. EVERY LESSON MAKES US STRONGER.

PERFORMANCE

PERFORMANCE TROUBLESHOOTING

Fast systems are designed. Reliable systems are measured.
Understand capacity. Find bottlenecks. Optimize resources.

Measure first.
Analyze deeply.
Optimize smartly.
Scale intentionally.

1. CAPACITY (PLAN FOR DEMAND)

WHAT TO CHECK
- Current usage vs capacity
- Growth trends
- Peak vs average load
- Resource limits
- Headroom / buffer

KEY METRICS
- ☑ CPU utilization (%)
- ☑ Memory utilization (%)
- ☑ Disk IOPS / throughput
- ☑ Network in/out (Mbps)
- ☑ Request rate (RPS / TPS)

BEST PRACTICE
Keep 20–30% headroom for critical resources.

2. SCALING (SCALE THE RIGHT WAY)

SCALING OPTIONS
- Vertical scaling (more power)
- Horizontal scaling (more nodes)
- Auto scaling (react to load)
- Database read replicas
- Caching and CDN

WHAT TO CHECK
- ☑ Scaling policies
- ☑ Trigger metrics
- ☑ Cooldown and limits
- ☑ Instance / pod health
- ☑ Load balancer distribution

BEST PRACTICE
Scale early, not after users complain.

3. BOTTLENECKS (FIND THE LIMIT)

COMMON BOTTLRECKS
- CPU saturation
- Memory pressure / swapping
- Disk I/O / latency
- Network latency / packet loss
- Database slow queries
- Lock contention
- External service latency
- Thread / connection limits

HOW TO FIND
- ☑ Check metrics & dashboards
- ☑ Use tracing / profiling
- ☑ Review logs & slow queries
- ☑ Load test & stress test
- ☑ Correlate across components

BEST PRACTICE
Fix the biggest bottleneck first, not the easiest.

4. RESOURCE OPTIMIZATION (DO MORE WITH LESS)

OPTIMIZATION AREAS
- Right-sizing instances / pods
- Optimize code & algorithms
- Cache frequently used data
- Optimize database queries
- Reduce over-provisioning
- Tune configuration

WHAT TO CHECK
- ☑ Resource requests/limits
- ☑ Connection pool settings
- ☑ JVM / runtime tuning
- ☑ Garbage collection
- ☑ I/O scheduler & filesystem

BEST PRACTICE
Continuously review and remove waste.

PERFORMANCE TROUBLESHOOTING WORKFLOW

1. DETECT → **2. VALIDATE** → **3. ISOLATE** → **4. ANALYZE** → **5. IDENTIFY ROOT CAUSE** → **6. RESOLVE** → **7. REVIEW & IMPROVE**

- Alert triggered
- User complaints
- SLO / SLA breach

- Confirm issue
- Check dashboards
- Define impact

- Find affected component
- Narrow scope

- Check metrics
- Logs & traces
- Identify patterns

- Find bottleneck
- Understand why
- Correlate data

- Apply fix
- Optimize or scale
- Verify improvement

- Post-incident review
- Update/runbooks
- Prevent recurrence

TOOLS YOU WILL USE
- ☑ Prometheus / Grafana
- ☑ CloudWatch / Datadog
- ☑ Kibana / Loki / ELK
- ☑ Jaeger / OpenTelemetry
- ☑ Top / htop / iostat
- ☑ nmon / vmstat / sar
- ☑ EXPLAIN / Slow logs
- ☑ Load testing tools
- ☑ Application profiling

PERFORMANCE QUICK CHECKS

LAYER	WHAT TO CHECK	COMMANDS / TOOLS EXAMPLES
Application	Response time, errors, CPU	Top, Htop, pprof, APM traces
Database	Slow queries, connections, locks	EXPLAIN, slow log, pg_stat_activity
System	CPU, memory, load, processes	top, vmstat, free -m, uptime
Disk	I/O, latency, queue depth	iostat -x, df -h, iotop
Network	Latency, packets, bandwidth	ping, traceroute, netstat, iftop
Infrastructure	Instance health, autoscaling	CloudWatch, kubectl top, alerts

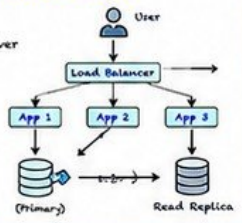
BOTTLENECK INDICATORS
- High CPU + high load → CPU bound
- High memory usage + swapping → Memory bound
- High disk latency / queue depth → Disk I/O bound
- High network latency / retransmits → Network bound
- Slow DB queries / high locks → Database bound
- High thread count / queue size → Concurrency bound

CAPACITY PLANNING BASICS

METRIC	FORMULA / GUIDELINE	EXAMPLE
CPU Headroom	Target Utilization < 70%	If avg = 50%, Headroom = 50%
Memory Headroom	Free Memory > 30%	If used = 60%, Headroom = 40%
Disk Usage	Free Space > 20%	If used = 75%, Free = 25%
Network	Utilization < 70%	If in = 200 Mbps of 1 Gbps

Capacity Rule of Thumb
Required Capacity = (Current Usage × Growth Factor × Safety Factor)
Safety Factor: 1.2 to 1.5

EXAMPLE: HIGH RESPONSE TIME INVESTIGATION
1. Alert: API p95 latency > 2s
2. Check dashboards → High CPU on app server
3. Verify logs → GC pauses detected
4. Check DB → Slow queries found
5. Optimize query & add index
6. Scale app replicas to 6
7. Latency back to normal (p95 < 500ms)
8. Document and share learnings



PERFORMANCE TUNING CHECKLIST
- ☑ Baseline metrics collected
- ☑ Bottleneck identified
- ☑ Root cause confirmed
- ☑ Quick wins applied
- ☑ Scaling policy reviewed
- ☑ Resource limits adjusted
- ☑ Load tested after changes
- ☑ Monitoring and alerts updated
- ☑ Documentation updated

BEST PRACTICES
- ★ Know your baseline.
- ★ Monitor everything that matters.
- ★ Alert on trends, not just thresholds.
- ★ Scale based on metrics, not guesses.
- ★ Optimize before you scale.
- ★ Test under load, not in theory.
- ★ Review and improve continuously.

MEASURE EVERYTHING. UNDERSTAND THE SYSTEM. ELIMINATE WASTE. DELIVER PERFORMANCE.

FOLLOW VERIQTA
YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

★ FAST SYSTEMS ARE BUILT. RELIABLE SYSTEMS ARE EARNED. ★

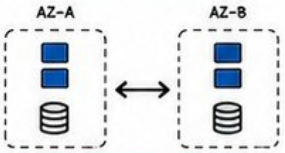
HA RELIABILITY

HIGH AVAILABILITY FAILURES

Availability is a feature. Resilience is a design.
Plan for failure. Test failover. Recover fast.

Design for failure.
Test relentlessly.
Recover automatically.
Learn and improve.

1. MULTI-AZ ARCHITECTURE (ZONE RESILIENCE)

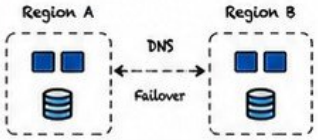


KEY COMPONENTS
- Load balancer across AZs
- App tier in multiple AZs
- Database Multi-AZ
- Shared, durable storage
- Health checks & auto recovery

WHAT TO CHECK
- ☑ AZ health
- ☑ Subnet & route tables
- ☑ Load balancer targets
- ☑ DB failover status
- ☑ Cross-AZ connectivity

BEST PRACTICE
Design so one AZ can fail without impacting users.

2. MULTI-REGION ARCHITECTURE (GLOBAL RESILIENCE)

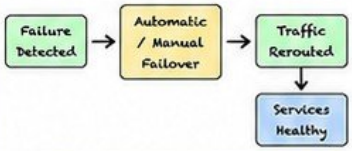


KEY COMPONENTS
- Global DNS with health checks
- Replicated data (async / sync)
- Region-independent services
- Traffic routing & failover
- Consistent backups

WHAT TO CHECK
- ☑ DNS health & latency
- ☑ Replication status & lag
- ☑ Cross-region networking
- ☑ Data consistency
- ☑ Failover runbooks

BEST PRACTICE
Automate failover.
Test regularly. Document clearly.

3. FAILOVER MECHANISMS (FAST RECOVERY)



MECHANISMS
- Health checks (L7 / synthetic)
- Auto scaling group replacement
- Database failover (Multi-AZ)
- DNS failover / Global Accelerator
- Queue based buffering

WHAT TO CHECK
- ☑ Health check thresholds
- ☑ Failover time (RTO)
- ☑ Data loss risk (RPO)
- ☑ Failover automation
- ☑ Alerts & notifications

BEST PRACTICE
Keep RTO and RPO as low as possible.

4. DISASTER SCENARIOS (PREPARE FOR WORST CASE)

- Region Outage
- Network Outage
- Data Center Failure
- Corruption / Data Loss
- Human / Security Incident

WHAT TO PREPARE
- ☑ RTO / RPO targets
- ☑ DR runbooks
- ☑ Backup strategy
- ☑ Communication plan
- ☑ Regular DR drills

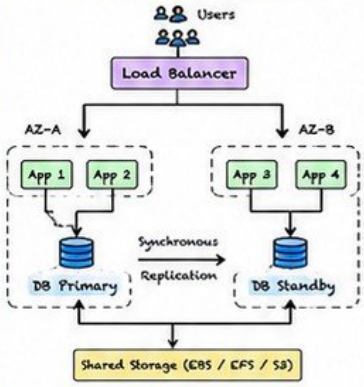
BEST PRACTICE
Assume everything can fail.
Prepare, test, and improve.

HIGH AVAILABILITY FAILURE WORKFLOW

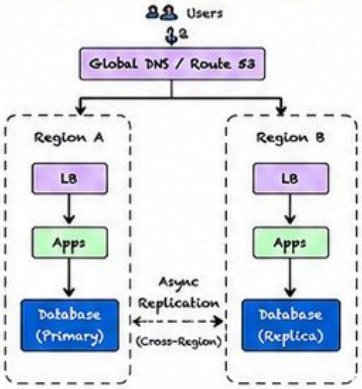
1. DETECT	2. IDENTIFY	3. ASSESS	4. FAILOVER	5. RECOVER	6. REVIEW
• Alerts trigger	• Which component	• Severity level	• Trigger failover	• Restored services	• Root cause analysis
• Health checks fail	• Which AZ / Region	• Users affected	• Route traffic	• Monitor closely	• Lessons learned
• User reports	• Total impact	• RTO / RPO impact	• Promote resources	• Validate data	• Update runbooks
• Monitoring detects	• Time of failure	• Escalate if needed	• Verify health	• Confirm stability	• Improve & test

FAILURE DETECTION METHODS
- ☑ CloudWatch / Prometheus
- ☑ Synthetic monitoring
- ☑ Application health checks
- ☑ Database replication lag
- ☑ Network monitoring
- ☑ User experience alerts
- ☑ Auto scaling group events

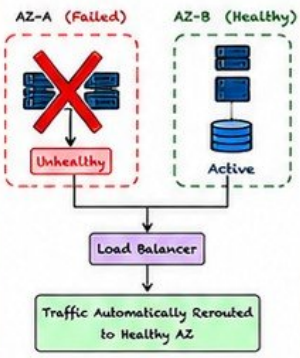
MULTI-AZ ARCHITECTURE EXAMPLE



MULTI-REGION ARCHITECTURE EXAMPLE



FAILOVER SCENARIO: AZ FAILURE



HA CHECKLIST

ARCHITECTURE
- ☑ Multi-AZ deployment
- ☑ Multi-region strategy
- ☑ No single points of failure
- ☑ Stateless application design

DATA PROTECTION
- ☑ Automated backups
- ☑ Replication enabled
- ☑ Regular restore testing
- ☑ Encryption at rest & in transit

OPERATIONS
- ☑ Monitoring & alerting
- ☑ Auto recovery enabled
- ☑ Runbooks documented
- ☑ DR drills executed

RELIABILITY TARGETS
- ☑ RTO defined
- ☑ RPO defined
- ☑ SLAs documented
- ☑ Capacity for failover

RTO vs RPO

RTO (Recovery Time Objective)
- Maximum tolerable downtime
- How fast you must recover

RPO (Recovery Point Objective)
- Maximum tolerable data loss
- How much data you can lose

COMMON FAILURE CAUSES
- AZ or Region outage
- Network partition
- Hardware failure
- Software / config error
- Data corruption
- Human error
- Security attack

REAL-WORLD IMPACT EXAMPLES
- Slow failover = more downtime
- Data loss = business impact
- Outages = revenue loss
- Users leave = trust lost
- Resilience = competitive advantage

KEY PRINCIPLE
You can't prevent everything.
But you can design so failures don't become disasters.

DISASTER RECOVERY DRILL
- ☑ Simulate real failure
- ☑ Measure RTO / RPO
- ☑ Validate runbooks
- ☑ Test backups & restore
- ☑ Document results
- ☑ Fix improvements

YOU CAN'T ELIMINATE FAILURE, BUT YOU CAN ENGINEER RESILIENCE. DESIGN WELL. TEST OFTEN. RECOVER FAST. IMPROVE ALWAYS.

DR RESILIENCE

DISASTER RECOVERY TROUBLESHOOTING

When disaster strikes, speed matters. Plan, test, and recover with confidence.
Understand your RTO. Protect your RPO. Validate your backups. Test your recoveries.

Prepare today.
Test often.
Recover faster.
Build resilience.

1. RTO (RECOVERY TIME OBJECTIVE)

Maximum tolerable time to restore systems.

WHAT TO CHECK

- ☑ RTO documented for all critical systems
- ☑ Realistic and tested
- ☑ Dependencies included
- ☑ Communication plan defined

TROUBLESHOOTING

- RTO missed in tests
- Missing runbooks
- Manual steps slowing recovery
- Unclear ownership
- Infrastructure not ready

BEST PRACTICE

Design for automated recovery. Test to prove it.

2. RPO (RECOVERY POINT OBJECTIVE)

Maximum tolerable data loss.

WHAT TO CHECK

- ☑ RPO defined per system
- ☑ Backup frequency meets RPO
- ☑ Replication lag within limits
- ☑ Data loss impact understood

TROUBLESHOOTING

- Backups running late
- Replication lag high
- Failed or skipped backups
- Retention too short
- No transaction logs

BEST PRACTICE

Align backup frequency with business tolerance.

3. BACKUP VALIDATION

Backups are only useful if they can be restored.

WHAT TO CHECK

- ☑ Backup job success
- ☑ Backup size trend (normal range)
- ☑ Checksums / integrity
- ☑ Restore test results
- ☑ Offsite / immutable copy

TROUBLESHOOTING

- Corrupted backups
- Incomplete backups
- Wrong backup chain
- Permissions / access issues
- Storage space full

BEST PRACTICE

Validate backups regularly. Don't assume.

4. RECOVERY TESTING

Test recoveries before you need them.

WHAT TO CHECK

- ☑ DR test frequency
- ☑ Test environment ready
- ☑ Runbooks up to date
- ☑ Dependencies tested
- ☑ Results documented

TROUBLESHOOTING

- Tests not executed
- Runbooks outdated
- Environment not matching production
- Hidden dependencies
- Unrealistic test scope

BEST PRACTICE

Test, learn, and improve continuously.

5. COMMON DR FAILURES

- ✖ Backups exist but restore fails
- ✖ RTO / RPO not realistic
- ✖ Dependencies not included
- ✖ Runbooks missing or outdated
- ✖ Network / IAM not ready in DR
- ✖ No monitoring after recovery
- ✖ Untested assumptions

BEST PRACTICE

Assume failure. Design for recovery.

DISASTER RECOVERY WORKFLOW

1. DETECT	2. ASSESS	3. ACTIVATE DR PLAN	4. RECOVER	5. VALIDATE	6. COMMUNICATE	7. REVIEW & IMPROVE
• Alerts trigger	• Identify affected systems	• Notify stakeholders	• Restore infrastructure	• Health checks	• Update stakeholders	• Post-incident review
• Monitor dashboards	• Check RTO / RPO	• Declare disaster	• Restore / failover data	• Data validation	• Share status	• Lessons learned
• User reports	• Review last good backup	• Start DR runbook	• Validate services	• Performance checks	• Confirm partial / full recovery	• Update runbooks
• Uncfrar impact	• Decide recovery strategy	• Assign roles	• Bring systems online	• User acceptance	• Post-recovery report	• Improve and test again

DR READINESS CHECKLIST

- ☑ RTO / RPO defined
- ☑ Backups automated
- ☑ Backups validated
- ☑ DR runbooks tested
- ☑ Dependencies mapped
- ☑ DR environment ready
- ☑ Monitoring in DR
- ☑ Team trained
- ☑ Regular DR tests
- ☑ Continuous improvement

RTO / RPO EXAMPLE TABLE

SYSTEM	RTO	RPO	BUSINESS IMPACT
Web Application	2 hrs	15 min	High - Customer facing
Database (OLTP)	1 hr	5 min	Very High - Revenue
File Storage	4 hrs	1 hr	Medium - Operations
Analytics DB	6 hrs	2 hrs	Low - Reporting
Logs / Monitoring	12 hrs	4 hrs	Low - Observability

BACKUP VALIDATION CHECKLIST

- ☑ Backup job completed successfully
- ☑ Backup size within expected range
- ☑ Verify backup integrity (checksum)
- ☑ Test restore of critical data
- ☑ Verify restore to alternate location
- ☑ Validate backup retention policy
- ☑ Confirm offsite / immutable copy
- ☑ Document results and issues

RECOVERY TESTING TYPES

TABLETOP TEST
Discussion based, no systems used

WALKTHROUGH TEST
Review runbooks step-by-step

SIMULATION TEST
Partial execution in test env

FULL FAILOVER TEST
Complete recovery to DR site

RTO / RPO CALCULATION GUIDE

RTO Considerations:
- Time to detect
- Time to failover
- Time to restore services
- Time to validate

Total RTO = D + F + R + V

RPO Considerations:
- Backup frequency
- Replication lag
- Transaction log availability
- Consistency model

Total RPO = Max Data Loss Tolerance

DISASTER SCENARIOS & SYMPTOMS

Region Outage	All services unreachable, status-health fails
AZ Failure	Partial outage, some dependencies failing
Database Corruption	Errors on queries, application failures
Backup Failure	Restore fails, files missing or corrupted
Network Outage	High latency, timeouts, packet loss

DR RUNBOOK (EXAMPLE SNIPPET)

1. Declare disaster and notify stakeholders
2. Verify scope and impact
3. Choose recovery site / region
4. Provision infrastructure (IaC / Templates)
5. Restore or failover data (meet RPO)
6. Start application services
7. Validate end-to-end functionality
8. Communicate and close incident

DR TESTING FREQUENCY GUIDE

Critical Systems	Monthly
High Impact	Every 2 Months
Medium Impact	Quarterly
Low Impact	Every 6 Months

After major changes, always test.

KEY PRINCIPLES

Plan for the worst.

- ☑ Test like you mean it.
- ☑ Automate everything possible.
- ☑ Measure RTO and RPO.
- ☑ Validate before you need it.
- ☑ Communicate clearly.
- ☑ Learn and improve always.

DISASTERS HAPPEN. RESILIENCE IS A CHOICE. PLAN AHEAD. TEST OFTEN. RECOVER FASTER.

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

PRODUCTION DEBUGGING TOOLKIT

DEBUG TOOLKIT

The right command at the right time saves hours.
Observe. Inspect. Verify. Correlate. Fix.

> Great engineers debug with clarity, not guesswork. Measure. Validate, Understand.

1. ESSENTIAL LINUX COMMANDS

SYSTEM INFO

```
uname -a          # Kernel info
uptime            # System uptime
hostnamectl       # Host details
df -h             # Disk usage
free -h           # Memory usage
top               # Real-time process
htop              # Better top
vmstat 1 5        # System stats
iostat -xz 1 5    # Disk I/O stats
```

PROCESS & SERVICES

```
ps aux            # All processes
pstree -p         # Process tree
ss -tulnp         # Listening ports
systemctl status <svc>    # Service
journalctl -u <svc> -f    # Logs follow
```

LOGS & FILES

```
tail -f /var/log/syslog          # Syslog
tail -f /var/log/messages        # Messages
grep -i "error" /var/log/*       # Search
less /var/log/secure             # Auth logs
cat /etc/os-release              # OS info
```

FILESYSTEM & PERMISSIONS

```
ls -lah           # List detailed
du -sh *          # Folder sizes
find / -type f -name <f>  # Find file
chmod / chown     # Permissions
```

ARCHIVES

```
tar -czvf backup.tar.gz /data  # Create
tar -xzvf backup.tar.gz        # Extract
```

2. KUBERNETES COMMANDS

kubectl

CLUSTER INFO

```
kubectl cluster-info
kubectl get nodes -o wide
kubectl get componentstatuses
```

WORKLOADS

```
kubectl get pods -A
kubectl get pods -A -o wide
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns>
kubectl logs -f <pod> -n <ns>
kubectl exec -it <pod> -n <ns> -- sh
```

DEPLOYMENTS & SERVICES

```
kubectl get deploy -A
kubectl describe deploy <deploy> -n <ns>
kubectl rollout status deploy/<deploy> -n <ns>
kubectl get svc -A
kubectl describe svc <svc> -n <ns>
kubectl get endpoints <svc> -n <ns>
```

CONFIG & STORAGE

```
kubectl get configmap -A
kubectl describe configmap <cm> -n <ns>
kubectl get secret -A
kubectl describe secret <secret> -n <ns>
kubectl get pvc -A
kubectl describe pvc <pvc> -n <ns>
```

TROUBLESHOOTING

```
kubectl get events -A --sort-by='.lastTimestamp'
kubectl describe node <node>
kubectl top nodes
kubectl top pods -A
```

3. DOCKER COMMANDS

docker

CONTAINERS

```
docker ps -a
docker inspect <container>
docker logs <container>
docker logs -f <container>
docker exec -it <container> sh
docker stats
```

IMAGES

```
docker images
docker inspect <image>
docker rmi <image>
```

NETWORKING

```
docker network ls
docker network inspect <net>
docker exec <container> ping <ip>
```

VOLUMES

```
docker volume ls
docker volume inspect <volume>
```

SYSTEM

```
docker info
docker system df
docker system prune -a
```

BUILD & RUN

```
docker build -t <image> .
docker run -d --name <name> \
 -p <host>:<container> <image>
```

4. CLOUD CLI TOOLS

AWS CLI

```
aws sts get-caller-identity
aws configure list
aws ec2 describe-instances
aws ec2 describe-azs
aws s3 ls
aws s3api head-object
  --bucket <b> --key <k>
aws cloudwatch describe-alarms
aws rds describe-db-instances
aws iam list-users
```

AZURE CLI

```
az account show
az vm list -o table
az vm show -g <rg> -n <vm>
az storage account list
az network vnet list
az monitor metrics list
```

GCP CLI

```
gcloud auth list
gcloud compute instances list
gcloud compute zones list
gcloud storage ls
gcloud sql instances list
gcloud logging read
"resource.type=\"gce_instance\""
```

GENERAL

```
curl -I <url>          # HTTP HEAD
nslookup <domain>      # DNS lookup
dig <domain>           # DNS query
```

5. NETWORK UTILITIES

CONNECTIVITY

```
ping <host>
ping -c 4 <host>
traceroute <host>
mtr <host>
tracepath <host>
```

DNS TOOLS

```
nslookup <domain>
dig <domain>
dig +short <domain>
host <domain>
```

PORT & SOCKET

```
ss -tulnp              # All sockets
netstat -tulnp         # Legacy
lsof -i :<port>        # Process using port
nc -vz <host> <port>   # Check port
telnet <host> <port>   # Test port
```

HTTP / API

```
curl <url>
curl -v <url>
curl -X GET <url> -H "Auth: token"
wget <url>
```

PACKET ANALYSIS

```
tcpdump -i any host <ip>
tcpdump -i any port <port>
```

PRODUCTION DEBUGGING WORKFLOW

1. **IDENTIFY**
   - What is broken?
   - Impact & scope
   - When started?
   - Any recent change?
2. **OBSERVE**
   - Collect metrics
   - Check dashboards
   - Review alerts
   - Gather logs
3. **ISOLATE**
   - Narrow down component
   - Check dependencies
   - Reproduce if possible
   - Validate assumptions
4. **VERIFY**
   - Run targeted checks
   - Validate configuration
   - Test connectivity
   - Confirm root cause
5. **CORRELATE**
   - Correlate logs & metrics
   - Build timeline
   - Identify pattern
   - Confirm impact
6. **RESOLVE**
   - Implement fix
   - Validate recovery
   - Monitor closely
   - Communicate status
7. **IMPROVE**
   - Document learnings
   - Update runbooks
   - Improve alerts
   - Prevent recurrence

QUICK DEBUG CHECKLIST

- [x] Reproduce the issue
- [x] Check recent changes
- [x] Validate configuration
- [x] Check resource usage
- [x] Review logs & metrics
- [x] Check dependencies
- [x] Test connectivity
- [x] Confirm root cause
- [x] Implement fix
- [x] Monitor & verify
- [x] Document & improve

LOG LOCATIONS (COMMON)

SERVICE / APP	LOG LOCATION
System Logs	/var/log/syslog or /var/log/messages
Auth Logs	/var/log/auth.log or /var/log/secure
Nginx	/var/log/nginx/access.log / error.log
Apache	/var/log/httpd/access_log / error_log
Docker	/var/lib/docker/containers/<id>/*.log
Kubernetes	kubectl logs <pod> -n <ns>
Application	/var/log/<app>/<app>.log

EXIT CODES (COMMON)

CODE	MEANING
0	Success
1	General error
2	Misuse / incorrect usage
126	Command invoked cannot execute
127	Command not found
130	Script terminated by Ctrl+C
137	Killed (OOM / SIGKILL)
143	Terminated (SIGTERM)

RESOURCE THRESHOLDS (GUIDELINE)

RESOURCE	WARNING	CRITICAL
CPU Usage	> 70%	> 90%
Memory Usage	> 75%	> 90%
Disk Usage	> 80%	> 95%
Inodes Usage	> 80%	> 95%
Load Average	> CPUs	> 2 x CPUs
Network Errors	> 1%	> 5%
Pod Restarts	> 3 / hr	> 10 / hr

HANDY ONE-LINERS

Find large files:
```
find / -type f -size +100M -exec ls -lh {} \;
```
Find and kill process by port:
```
lsof -ti :8080 | xargs -r kill -9
```
Top 10 memory consumers:
```
ps aux --sort=-%mem | head -10
```
Top 10 CPU consumers:
```
ps aux --sort=-%cpu | head -10
```
Disk usage by folder:
```
du -sh /* | sort -hr | head -10
```
Check pod resources (K8s):
```
kubectl top pods -A --sort-by=memory
```

PRO TIP: Don't guess. Collect evidence. Every command you run should answer a question. Good debugging is 80% gathering the right data and 20% fixing the issue.

OBSERVE -> UNDERSTAND -> RESOLVE -> IMPROVE

FOLLOW VERIQTA

YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

★ THE BEST TOOL IN PRODUCTION IS A CALM MIND AND A GOOD TOOLKIT. ★

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

TROUBLESHOOT SMARTER

ENTERPRISE TROUBLESHOOTING CHECKLISTS

Use these checklists during incidents to investigate faster, avoid blind spots, and restore services with confidence.

Follow the process.
Check everything.
Document clearly.
Learn always.

1. LINUX CHECKLIST

SYSTEM HEALTH
- ☑ CPU usage (top, htop)
- ☑ Memory usage (free -h)
- ☑ Disk usage (df -h)
- ☑ Inode usage (df -i)
- ☑ Load average (uptime)
- ☑ System logs (journalctl -xe)

SERVICES & PROCESSES
- ☑ Critical services status (systemctl --failed)
- ☑ Running processes (ps aux / top)
- ☑ Zombie processes (ps aux | grep Z)
- ☑ Port listeners (ss -tulnp)

LOGS & FILES
- ☑ Application logs
- ☑ /var/log/syslog or /var/log/messages
- ☑ Kernel logs (dmesg)
- ☑ Recent changes (last, history)
- ☑ Permissions & ownership

KEY TIP:
Start with system health, then narrow down.

2. NETWORKING CHECKLIST

CONNECTIVITY
- ☑ Can I reach the host? (ping)
- ☑ DNS resolution working? (nslookup / dig)
- ☑ Route to destination? (traceroute / tracepath)
- ☑ Latency normal? (ping / mtr)

PORTS & SERVICES
- ☑ Is the port open? (nc -vz / telnet)
- ☑ Service listening? (ss -tulnp)
- ☑ Firewall rules (iptables / nftables / ufw)
- ☑ Security Groups / NACLs (Cloud)

NETWORK CONFIGURATION
- ☑ IP address & subnet correct?
- ☑ Default gateway reachable?
- ☑ DNS servers reachable?
- ☑ MTU issues? (ping -M do -s 1472 <ip>)

KEY TIP:
Validate path end-to-end, don't assume the network.

3. KUBERNETES CHECKLIST

CLUSTER HEALTH
- ☑ Cluster status (kubectl get nodes)
- ☑ System pods healthy (kubectl get pods -A)
- ☑ Events issues (kubectl get events --sort-by=.lastTimestamp)
- ☑ Kubelet status on nodes

WORKLOADS
- ☑ Pod status (kubectl get pods -A)
- ☑ Pod logs (kubectl logs <pod> -n <ns>)
- ☑ Pod describe (kubectl describe pod <pod> -n <ns>)
- ☑ CrashLoopBackOff / ImagePullBackOff?

NETWORKING
- ☑ Services & Endpoints (kubectl get svc,endpoints -A)
- ☑ Ingress status (kubectl get ingress -A)
- ☑ DNS resolution (kubectl exec -it <pod> -- nslookup <svc>)
- ☑ NetworkPolicies affecting traffic?

KEY TIP:
Check Events and describe — they tell the real story.

4. CLOUD CHECKLIST

REGION & AVAILABILITY
- ☑ Region status (AWS: Health Dashboard)
- ☑ AZ health (EC2 / RDS / ALB status)
- ☑ Service limits (Quotas / Limits)

COMPUTE
- ☑ Instance status (running / terminated)
- ☑ CPU / Memory metrics
- ☑ Auto Scaling events
- ☑ Load Balancer health checks

STORAGE & DATABASE
- ☑ EBS / EFS / S3 availability
- ☑ RDS status & connections
- ☑ Backup status & snapshots
- ☑ Disk usage & IOPS / throughput

SECURITY & ACCESS
- ☑ IAM permissions
- ☑ Security Groups / NACLs
- ☑ KMS / Secrets access

KEY TIP:
Check CloudWatch metrics and service health first.

5. CI/CD CHECKLIST

PIPELINE HEALTH
- ☑ Pipeline status (success / failed / running)
- ☑ Failed stage logs
- ☑ Agent / Runner health
- ☑ Trigger / Webhook issues

CODE & ARTIFACTS
- ☑ Source code checkout successful?
- ☑ Build logs / Compile errors
- ☑ Artifact published correctly?
- ☑ Dependency scan / Security scan

DEPLOYMENT
- ☑ Deployment logs
- ☑ Environment variables / Secrets
- ☑ Health checks after deployment
- ☑ Rollback history (last successful build)

KEY TIP:
Reproduce in lower environment before making changes.

6. SECURITY CHECKLIST

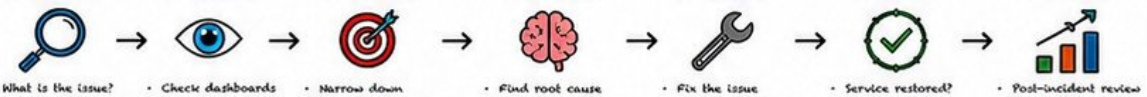
ACCESS & AUTHENTICATION
- ☑ Unexpected user logins
- ☑ Failed login attempts
- ☑ MFA / SSO issues
- ☑ Privilege escalation attempts

SYSTEM & ENDPOINTS
- ☑ Malware / Suspicious processes
- ☑ File integrity changes
- ☑ Open ports & listening services
- ☑ Unexpected outbound connections

DATA & LOGS
- ☑ Critical logs reviewed (auth, syslog, app)
- ☑ Data exfiltration indicators
- ☑ Secrets / Keys exposure
- ☑ Alerting / SIEM events reviewed

KEY TIP:
Contain first, then investigate. Preserve evidence.

TROUBLESHOOTING WORKFLOW (FOLLOW EVERY TIME)



1. IDENTIFY	2. OBSERVE	3. ISOLATE	4. ANALYZE	5. RESOLVE	6. VERIFY	7. IMPROVE
• What is the issue?	• Check dashboards	• Narrow down	• Find root cause	• Fix the issue	• Service restored?	• Post-incident review
• Impact and scope	• Review alerts	• Eliminate non-issues	• Correlate data	• Validate the fix	• No side effects?	• Create action items
• When did it start?	• Collect logs	• Reproduce if possible	• Use the checklists	• Monitor closely	• Users confirmed?	• Update runbooks
• Who is affected?	• Note symptoms	• Validate assumptions	• Prove the cause	• Document actions	• Close the incident	• Share learnings

INCIDENT SEVERITY GUIDE

- P1 **Critical** — Major outage, many users
- P2 **High** — Partial outage, high impact
- P3 **Medium** — Degraded performance
- P4 **Low** — Minimal impact / inquiry

Senior engineers don't guess. They observe, validate, and solve with a process.
Check everything. Document everything. Learn from every incident.

Build reliability. Earn trust.

FOLLOW VERIQTA

YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

OBSERVE. INVESTIGATE. RESOLVE. IMPROVE. THAT'S ENGINEERING.

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

REAL CASES
REAL LESSONS

REAL PRODUCTION CASE STUDIES

Real incidents. Real impact. Real lessons.
Study the failure. Apply the lesson. Prevent the next one.

Every outage teaches something. Document today, Prevent tomorrow.

1. KUBERNETES OUTAGE

Pod CrashLoopBackOff across cluster

WHAT HAPPENED

After a config update, multiple deployments (dev, staging, prod) Pods entered CrashLoopBackOff state across several namespaces.

IMPACT

- 65% of services unavailable
- 42 minutes of downtime

ROOT CAUSE

Invalid environment variable in ConfigMap referenced by multiple deployments.

HOW WE INVESTIGATED

1. Checked alerts (Pod restarts spiked)
2. kubectl get pods -A
3. kubectl describe pod <pod>
4. Checked recent ConfigMap changes
5. Rollback to previous ConfigMap version

FIX

Reverted ConfigMap, redeployed workloads, and added validation in CI pipeline.

LESSONS LEARNED

- Validate configs before rolling out
- Use config checks in CI/CD
- Enable alerts on Pod restarts

2. DATABASE OUTAGE

PostgreSQL primary node crashed

WHAT HAPPENED

Primary DB node ran out of disk space and crashed. Applications lost connectivity.

IMPACT

- 100% write operations down
- 37 minutes downtime

ROOT CAUSE

Autovacuum was disabled. Disk filled up due to transaction bloat.

HOW WE INVESTIGATED

1. App errors: DB connection failed
2. Checked RDS status (primary down)
3. Verified disk metrics (100% used)
4. Checked logs (could not extend file)
5. Failed over to standby (Multi-AZ)

FIX

Enabled autovacuum, cleaned bloat, increased storage, added disk alarms.

LESSONS LEARNED

- Monitor disk usage and inode usage
- Never disable autovacuum
- Test failover regularly

3. DNS FAILURE

Internal DNS service became unavailable

WHAT HAPPENED

CoreDNS pods crashed due to high memory usage, causing DNS resolution failures.

IMPACT

- Services could not resolve names
- 28 minutes downtime

ROOT CAUSE

Memory leak in CoreDNS plugin + insufficient resources.

HOW WE INVESTIGATED

1. Users reported timeouts
2. nslookup failed internally
3. Checked CoreDNS pod status
4. Reviewed pod logs and metrics
5. OOMKilled events found

FIX

Increased memory limits, upgraded CoreDNS, and restarted pods.

LESSONS LEARNED

- Monitor CoreDNS resource usage
- Set proper requests and limits
- Use horizontal autoscaling

4. CI/CD FAILURE

Pipelines stuck – deployments blocked

WHAT HAPPENED

Build pipelines started failing in the test stage. No new code could be deployed.

IMPACT

- All deployments blocked
- 3 hours delay

ROOT CAUSE

Expired API token in secrets manager used by CI runners.

HOW WE INVESTIGATED

1. Pipelines failing at same step
2. Checked job logs (401 Unauthorized)
3. Verified secrets (token expired)
4. Checked secret rotation policy
5. Updated token and re-ran pipeline

FIX

Rotated token, updated secret, restarted runners, and re-ran jobs.

LESSONS LEARNED

- Monitor secret/token expiration
- Use automated rotation
- Add pipeline health checks

5. CLOUD NETWORKING INCIDENT

Intermittent connectivity between subnets

WHAT HAPPENED

Applications in private subnets could not reach some external services intermittently.

IMPACT

- Partial outage for users
- 1 hour total impact

ROOT CAUSE

NAT Gateway in one AZ failed. Route table was misconfigured to single NAT.

HOW WE INVESTIGATED

1. Users reported intermittent timeouts
2. Checked VPC flow logs (packets dropped)
3. Verified NAT Gateway status (unhealthy)
4. Reviewed route tables
5. Traffic routed to single AZ NAT

FIX

Added NAT Gateway in each AZ, updated route tables for high availability.

LESSONS LEARNED

- Use NAT Gateway per AZ
- Enable VPC flow logs
- Build for failure, not hope

CASE STUDY TAKEAWAYS

Observe
Collect facts and evidence before jumping to conclusions.

Analyze
Find the real root cause, not just the symptoms.

Fix & Validate
Fix the issue and confirm the system is healthy.

Learn & Document
Write it down. Share it. Prevent it from happening again.

INCIDENT RESPONSE LIFECYCLE (APPLY TO EVERY CASE)

1. DETECT	2. TRIAGE	3. CONTAIN	4. INVESTIGATE	5. RESOLVE	6. COMMUNICATE	7. REVIEW
• Alerts	• Validate issue	• Stop the impact	• Find root cause	• Implement fix	• Update stakeholders	• Post-incident review
• User reports	• Determine impact	• Isolate affected	• Collect evidence	• Restore services	• Share timelines	• Lessons learned
• Monitoring	• Gather data	• Apply temporary fix	• Analyze logs/metrics	• Validate fully	• Be transparent	• Action items

POST-INCIDENT CHECKLIST

- [x] Incident timeline documented
- [x] Root cause identified
- [x] Action items created
- [x] Monitoring improved
- [x] Runbooks updated
- [x] Stakeholders informed
- [x] Knowledge shared

★ REAL ENGINEERS LEARN MORE FROM OUTAGES THAN FROM UPTIME. DOCUMENT. LEARN. IMPROVE. THAT'S HOW SYSTEMS GET STRONGER. ★

FOLLOW VERIQTA

YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

★ INCIDENTS WILL HAPPEN. RESILIENCE IS BUILT BY HOW YOU RESPOND. ★

THINK
INVESTIGATE
RESOLVE

SENIOR ENGINEER TROUBLESHOOTING PLAYBOOKS

Use a process. Ask the right questions.
Collect evidence. Find the real cause. Recover with confidence.

Good engineers fix.
Senior engineers
prevent recurrence.
Document everything. ☺

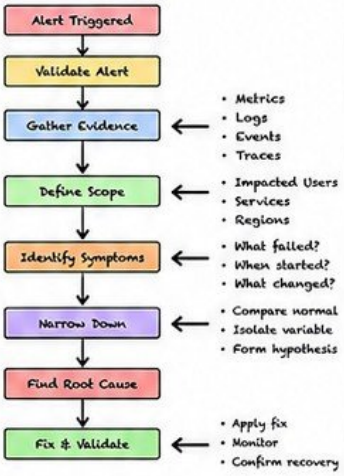
1. DECISION TREES

Use quick decisions
to narrow the problem.

★ Use decision trees to act fast
and avoid random troubleshooting.

2. INVESTIGATION FLOWCHARTS

Follow structured steps.
Don't skip evidence.



★ Follow the flow. Don't guess.
Evidence > Assumptions.

3. ESCALATION GUIDANCE

Escalate early with context,
not noise.

P0	Critical	War Room Incident Commander + Major Impact
P1	High	Service Owner + Cross-functional Support
P2	Medium	Team Lead + Subject Matter Experts
P3	Low	On-call Engineer + Local Troubleshooting

ESCALATE WHEN:

- ☑ Impact is high or growing
- ☑ You lack access or expertise
- ☑ Recovery time is at risk
- ☑ You are unsure after initial checks

★ Provide: Impact, Symptoms,
What you tried, Evidence collected.

4. RECOVERY STRATEGIES

Restore service. Verify.
Stabilize. Prevent repeat.

ROLLBACK
- Revert to last known good
- Validate functionality
- Monitor after rollback

FAILOVER
- Switch to healthy instance
- Promote standby
- Validate traffic flow

SCALE OUT
- Add capacity
- Balance load
- Monitor utilization

RESTART / RECYCLE
- Restart service / pods
- Recycle instances
- Clear stuck states

DATA RECOVERY
- Restore from backup
- Point-in-time recovery
- Validate data integrity

FEATURE TOGGLE
- Disable failing feature
- Reduce blast radius
- Restore core service

★ Recover fast, then stabilize,
then improve.

5. INCIDENT INVESTIGATION WORKFLOW

1. DETECT	2. VALIDATE	3. GATHER EVIDENCE	4. ANALYZE	5. TEST THEORY	6. ROOT CAUSE	7. RESOLVE	8. REVIEW
• Alerts • User reports • Monitoring	• Is it real? • Impact check • Avoid noise	• Logs • Metrics • Events • Traces	• Identify patterns • Compare baseline • Form hypothesis	• Prove / disprove • Reproduce • Narrow scope	• Confirm cause • Document • Share	• Apply fix • Validate • Monitor	• Post-incident • Lessons learned • Improve

INVESTIGATION PRINCIPLES

- ☑ Start broad, then narrow down
- ☑ Trust evidence, not assumptions
- ☑ Change one thing at a time
- ☑ Document everything as you go

KEY QUESTIONS TO ASK

- What changed?
- What is the impact?
- What did it start?
- What is different from normal?
- What does the data say?

EVIDENCE COLLECTION ORDER

1. Metrics & Dashboards
2. Logs & Events
3. Traces & Requests
4. Configurations & Changes
5. Dependencies & External Services

6. TROUBLESHOOTING PLAYBOOK INDEX

Linux Playbook → Page 59
Kubernetes Playbook → Page 60
Database Playbook → Page 61
Networking Playbook → Page 62
CI/CD Playbook → Page 63
Cloud Playbook → Page 64
Security Playbook → Page 65
Application Playbook → Page 66
Performance Playbook → Page 67

7. COMMON ESCALATION PATHS

8. RECOVERY CHECKLIST

- ☑ Service is reachable
- ☑ Core functionality works
- ☑ Error rate is back to normal
- ☑ Latency is within SLO
- ☑ Dependencies are healthy
- ☑ Data integrity is verified
- ☑ Monitoring & alerts updated
- ☑ Stakeholders notified
- ☑ Post-incident review scheduled

9. SENIOR ENGINEER MINDSET

- ☑ Stay calm. Lead the process.
- ☑ Ask better questions.
- ☑ Dig deep, not wide.
- ☑ Think in systems.
- ☑ Document for the next person.
- ☑ Prevent repeat, not just fix.
- ☑ Own the outcome.
- ☑ Teach and share.

TROUBLESHOOTING MANTRA

OBSERVE → QUESTION → INVESTIGATE → ANALYZE → DECIDE → ACT → VALIDATE → IMPROVE

REMEMBER
Every incident is a learning opportunity. Every fix should make us stronger.

FOLLOW VERIQTA

YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

★ PROCESS. PATIENCE. PRECISION. THAT'S HOW SENIOR ENGINEERS WIN. ★

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

READY.
RELIABLE.
RESILIENT.

PRODUCTION READINESS CHECKLIST

Check everything.
Miss nothing.
Ship confidently.
Sleep peacefully.

A production system is not ready until it is observable, secure, resilient, tested, documented, and automated.
Use this checklist before every release.

1. MONITORING

- ☑ Health checks configured
- ☑ Metrics collection enabled
- ☑ Dashboards created
- ☑ Alerts defined & tested
- ☑ SLOs / SLIs established
- ☑ Uptime monitoring enabled
- ☑ Dependency monitoring on
- ☑ Synthetic tests configured

KEY TIP:
If you can't see it, you can't fix it.

2. LOGGING

- ☑ Centralized logging enabled
- ☑ Structured logs (JSON)
- ☑ Log levels standardized
- ☑ Key events are logged
- ☑ Log retention policy set
- ☑ Sensitive data masked
- ☑ Log search / alerts ready
- ☑ Log pipeline monitored

KEY TIP:
Good logs turn mystery into clarity.

3. SECURITY

- ☑ Security groups / firewalls
- ☑ Least privilege access
- ☑ MFA enabled for users
- ☑ Secrets in Vault / KMS
- ☑ Dependencies scanned
- ☑ Container images scanned
- ☑ RBAC / IAM reviewed
- ☑ Penetration testing done
- ☑ Security headers enabled
- ☑ Vulnerability remediation

KEY TIP:
Secure by design.
Verify by default.

4. BACKUPS

- ☑ Automated backups enabled
- ☑ Backup encryption on
- ☑ Backup retention policy set
- ☑ Backup stored off-site / off-region
- ☑ Restore tested successfully
- ☑ Point-in-time recovery tested
- ☑ Backup monitoring & alerts
- ☑ Disaster recovery documented

KEY TIP:
Backups you don't test don't exist.

5. CAPACITY

- ☑ Current usage monitored
- ☑ Capacity thresholds defined
- ☑ Scaling strategy documented
- ☑ Auto-scaling configured
- ☑ Load testing completed
- ☑ Resource limits set
- ☑ Performance baseline captured
- ☑ Growth forecast reviewed

KEY TIP:
Plan for growth, not just for today.

6. DOCUMENTATION

- ☑ Architecture documented
- ☑ Runbooks created
- ☑ API documentation updated
- ☑ Operational procedures
- ☑ Environment diagrams
- ☑ Incident response plan
- ☑ On-call contacts listed
- ☑ Change log / release notes

KEY TIP:
Document today.
Thank yourself tomorrow.

7. AUTOMATION

- ☑ Infrastructure as Code
- ☑ Configuration as Code
- ☑ CI/CD pipeline automated
- ☑ Repeatable deployments
- ☑ Automated rollbacks
- ☑ Automated backups
- ☑ Policy as Code enforced
- ☑ Drift detection enabled
- ☑ ChatOps / Notifications
- ☑ Automation monitored

KEY TIP:
Automate the repeatable.
Reduce human error.

8. TESTING

- ☑ Unit tests passing
- ☑ Integration tests passing
- ☑ End-to-end tests passing
- ☑ Performance tests done
- ☑ Security tests done
- ☑ Chaos / resilience tests
- ☑ UAT completed
- ☑ Release candidate verified
- ☑ Smoke tests automated
- ☑ Go/No-Go criteria defined

KEY TIP:
Test like it's production.
Because it is.

PRODUCTION READINESS GATE

1. PLAN	2. VALIDATE	3. TEST	4. APPROVE	5. DEPLOY	6. REVIEW
• Define scope	• Check every item	• Run all tests	• Stakeholder sign-off	• Deploy confidently	• Monitor & validate
• Identify risks	• Validate configs	• Load & security	• Go/No-Go decision	• Monitor closely	• Collect feedback
• Review checklist	• Verify dependencies	• UAT & smoke tests	• Change window set	• Keep rollback ready	• Improve continuously

QUICK CHECK BOX

- ☑ All critical alerts work
- ☑ Dashboards show healthy
- ☑ Backups tested
- ☑ Access is secure
- ☑ Rollback tested
- ☑ Documentation updated
- ☑ Stakeholders notified
- ☑ System is Go for Prod

FINAL SAY BEFORE GO-LIVE

If any critical item is unchecked, the system is NOT ready.
Fix it, verify it, and then ship it with confidence.

SECURE + OBSERVABLE + AUTOMATED + TESTED = READY

★ PRODUCTION IS EARNED, NOT HOPED FOR. CHECK IT ALL. TRUST IT ALL. OWN IT ALL. ★

FOLLOW VERIQTA

YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

★ PREPARE. VERIFY. RELEASE. REPEAT. ★

★ VERIQTA ★

LEARN • BUILD • OPERATE • SCALE

YOUR ROADMAP TO MASTERY

DEVOPS TROUBLESHOOTING MASTERY ROADMAP

Master the art of troubleshooting. Solve problems faster.
Think like a senior engineer. Build systems that never stop.

Keep learning.
Keep building.
Keep solving.
Keep improving.

That's how engineers level up.

1. CORE SKILLS REVIEW

- Linux & Shell Scripting
- Networking Fundamentals
- Docker & Containers
- Kubernetes Operations
- Cloud Platforms (AWS/Azure/GCP)
- CI/CD & Automation
- Monitoring & Observability
- Security & Compliance
- Scripting & Programming
- Systems Design Basics

★ FOCUS: Strong foundations make troubleshooting easy.

2. TROUBLESHOOTING METHODOLOGY

1. **IDENTIFY**
   Understand the problem
   Collect symptoms
2. **OBSERVE**
   Check metrics, logs, events, and alerts
3. **ISOLATE**
   Narrow down the blast radius
4. **ANALYZE**
   Find the root cause (not just the cause)
5. **RESOLVE**
   Fix the issue
   Implement correctly
6. **VERIFY**
   Confirm the fix
   Validate completely
7. **DOCUMENT**
   Capture learnings
   Share with team

★ FOLLOW THE PROCESS. DON'T SKIP STEPS.

3. CONTINUOUS LEARNING PATH

- **Daily:** Read docs. Explore tools. Practice 30–60 mins.
- **Weekly:** Build something new. Solve real problems.
- **Monthly:** Deep dive into one technology.
- **Quarterly:** Work on complex projects.
- **Annually:** Level up your architecture skills.
- **Always:** Learn from incidents. Reflect. Improve.

★ ENGINEERS NEVER STOP LEARNING. GROW. ADAPT. EVOLVE.

4. ENTERPRISE ENGINEERING PRINCIPLES

- ☑ Reliability over speed
- ☑ Observability over assumptions
- ☑ Automation over manual work
- ☑ Security by design
- ☑ Design for failure (not success)
- ☑ Document everything
- ☑ Test in production-like environment
- ☑ Monitor everything that matters
- ☑ Simplicity over complexity
- ☑ Ownership over blame
- ☑ Blameless culture
- ☑ Continuous improvement mindset

★ BUILD SYSTEMS THAT SCALE. BUILD CULTURE THAT LASTS.

5. RECOMMENDED LABS (PRACTICE MAKES YOU BETTER)

LINUX LABS

1. Log analysis
2. Process troubleshooting
3. Disk & memory issues
4. Services & logs
5. Shell scripting automation

NETWORKING LABS

1. DNS troubleshooting
2. Connectivity analysis
3. Packet capture (tcpdump)
4. Firewall & NACL labs
5. Load balancer issues

DOCKER LABS

1. Container failures
2. Image issues
3. Storage problems
4. Network issues
5. Multi-container apps

KUBERNETES LABS

1. Pod failures
2. Scheduling issues
3. Networking issues
4. Storage issues
5. Rollout failures

CLOUD LABS

1. VPC & networking
2. IAM & permissions
3. Auto scaling issues
4. RDS / DB failures
5. Cost & performance

CI/CD LABS

1. Pipeline failures
2. Artifact issues
3. Deployment failures
4. Secrets management
5. Rollback strategies

★ BUILD LABS. BREAK THINGS. FIX THEM. REPEAT.

6. FINAL GUIDANCE: BECOME PRODUCTION-READY

- Solve real problems. Not just tutorials.
- Understand systems end-to-end.
- Ask the right questions.
- Collect evidence before making changes.
- Validate assumptions.
- Communicate clearly.
- Document your work.
- Learn from every incident.
- Share knowledge with the team.
- Take ownership. Deliver results.

★ BE CURIOUS. BE CONSISTENT. BE RELIABLE. THAT'S WHAT SENIOR ENGINEERS DO.

7. TROUBLESHOOTING MATURITY LEVELS

- LEVEL 5 ENGINEER — Designs resilient systems. Prevents issues at scale
- LEVEL 4 EXPERT — Solves complex problems. Optimizes & automates
- LEVEL 3 ADVANCED — Troubleshoots independently. Understands root cause
- LEVEL 2 INTERMEDIATE — Follows process. Needs some guidance
- LEVEL 1 BEGINNER — Needs step-by-step help. Learning the basics

★ KEEP CLIMBING. LEVEL UP EVERY DAY. ★

8. QUICK REFERENCE CHEAT SHEET

Check Alerts	→	Is something failing?
Check Metrics	→	Is it resource related?
Check Logs	→	What errors exist?
Check Events	→	Any warnings or failures?
Check Config	→	Any recent changes?
Check Dependencies	→	Is upstream healthy?
Reproduce	→	Can you reproduce it?
Fix & Verify	→	Did the fix work?
Document	→	What did we learn?

★ FOLLOW THE FLOW. STAY CALM. SOLVE SMART. ★

TROUBLESHOOTING IS A SKILL. MASTERY IS A JOURNEY.
KEEP PRACTICING. KEEP SOLVING. KEEP BUILDING. YOU'VE GOT THIS!

FOLLOW VERIQTA

YouTube veriqta · GitHub veriqta · LinkedIn veriqta · X veriqta · Instagram veriqta · Telegram veriqta

★ SOLVE TODAY. IMPACT TOMORROW. BUILD FOR SCALE. ENGINEER FOR THE FUTURE. ★